
ARCHITECTURAL DESIGN PATTERNS HANDBOOK

Complete Guide for Interview Preparation with Diagrams

Version 1.0

About This Book

Dear Reader,

This book is specially created for developers and architects who want to master architectural design patterns before their technical interviews. Every concept is explained in simple, clear English with real-world examples. Each pattern includes a diagram to help you visualize the structure and flow, making it easier to understand and remember.

Architectural patterns provide the foundation for building scalable, maintainable, and resilient software systems. They define the high-level structure of your application, how components interact, and how data flows through the system. Whether you're building a monolith, microservices, or event-driven architecture, understanding these patterns is essential for making the right architectural decisions.

This book covers all the essential architectural patterns you need to know for your interviews. Take your time reading this book. Read one pattern per day. Study the diagrams. Practice explaining each pattern out loud. By the time you finish, you will be completely ready to answer any architectural pattern question in your interview.

Let's begin your journey to becoming an architecture expert.

TABLE OF CONTENTS

PART ONE: INTRODUCTION TO ARCHITECTURAL PATTERNS

- Chapter 1: What Are Architectural Patterns?
- Chapter 2: Why Architectural Patterns Matter
- Chapter 3: Classification of Architectural Patterns

PART TWO: LAYERED ARCHITECTURE PATTERNS

- Chapter 4: Layered (N-Tier) Architecture
- Chapter 5: Hexagonal Architecture (Ports & Adapters)
- Chapter 6: Onion Architecture
- Chapter 7: Clean Architecture

PART THREE: MICROSERVICES ARCHITECTURE PATTERNS

- Chapter 8: Monolithic Architecture
- Chapter 9: Microservices Architecture
- Chapter 10: Service-Oriented Architecture (SOA)
- Chapter 11: Self-Contained Systems (SCS)
- Chapter 12: Space-Based Architecture

PART FOUR: EVENT-DRIVEN PATTERNS

- Chapter 13: Event-Driven Architecture (EDA)
- Chapter 14: Event Sourcing
- Chapter 15: Command Query Responsibility Segregation (CQRS)
- Chapter 16: Publish-Subscribe Architecture

PART FIVE: MESSAGE-BASED PATTERNS

- Chapter 17: Message Bus Architecture
- Chapter 18: Message Queue Architecture
- Chapter 19: Enterprise Service Bus (ESB)

PART SIX: DISTRIBUTED SYSTEMS PATTERNS

- Chapter 20: Peer-to-Peer Architecture
- Chapter 21: Master-Slave Architecture
- Chapter 22: Shared-Nothing Architecture

- Chapter 23: Sharding Pattern

PART SEVEN: MODERN ARCHITECTURE PATTERNS

- Chapter 24: Serverless Architecture
- Chapter 25: Event-Driven Microservices
- Chapter 26: Stream Processing Architecture
- Chapter 27: Data Mesh Architecture
- Chapter 28: Cell-Based Architecture

PART EIGHT: CROSS-CUTTING PATTERNS

- Chapter 29: Circuit Breaker
- Chapter 30: API Gateway
- Chapter 31: Strangler Pattern
- Chapter 32: Sidecar Pattern
- Chapter 33: Ambassador Pattern

PART NINE: COMPARISON AND INTERVIEW PREPARATION

- Chapter 34: Pattern Comparison Table
- Chapter 35: How to Answer Architecture Questions
- Chapter 36: Common Interview Questions with Examples

[Page 3]

PART ONE: INTRODUCTION TO ARCHITECTURAL PATTERNS

Chapter 1: What Are Architectural Patterns?

Understanding the Basic Concept

An architectural pattern is a reusable solution to a recurring problem in software architecture. It provides a high-level blueprint for structuring software systems, defining how components interact, how data flows, and how the system meets its non-functional

requirements.

The Difference from Design Patterns

Aspect	Architectural Patterns	Design Patterns
Scope	Whole system	Individual components
Level	High-level structure	Implementation details
Granularity	Coarse-grained	Fine-grained
Decisions	Technology, deployment	Code organization
Examples	Microservices, Layered	Singleton, Factory

Think of it this way. If building a house:

- **Architectural Pattern** is the overall style (Victorian, Modern, Ranch)
- **Design Pattern** is how you build specific parts (door mechanism, window frame)

Key Characteristics of Architectural Patterns

1. **High-Level Abstraction:** They describe system structure, not implementation details.

2. **Reusable:** Solutions proven effective across many systems.

3. **Constraints:** They impose structure and rules on the system.

4. **Trade-offs:** Each pattern has advantages and disadvantages.

5. **Non-Functional Focus:** Address scalability, maintainability, performance.

Why Understanding Architecture Matters

1. **Foundation for Decisions:** Architecture shapes everything that follows.

2. **Team Communication:** Common vocabulary for discussing system structure.

3. **Non-Functional Requirements:** Architecture determines scalability, availability.

4. **Evolution Path:** Architecture defines how system can grow.

5. **Interview Essential:** Architecture questions are core to senior roles.

[Page 4]

Chapter 2: Why Architectural Patterns Matter

The Impact of Architecture

Poor architectural choices have severe consequences:

1. **Technical Debt:** Hard to change, costly to maintain.

2. **Scalability Limits:** System can't handle growth.

3. **Team Bottlenecks:** Teams step on each other's toes.

4. **Deployment Nightmares:** Complex, risky deployments.

5. **Vendor Lock-in:** Hard to migrate away from chosen technology.

The Benefits of Using Patterns

1. Proven Solutions

Architectural patterns represent collective wisdom from thousands of systems. When you choose microservices, you benefit from lessons learned by Netflix, Amazon, and others.

2. Common Vocabulary

When you say "We're using a layered architecture," everyone understands:
- Where to put code (presentation, business, data)
- How dependencies flow (downward)
- Where to make changes

3. Focus on Trade-offs

Patterns make trade-offs explicit:

- Monolith: Simple but less scalable
- Microservices: Scalable but complex
- Event-Driven: Decoupled but eventual consistency

4. Risk Reduction

Using proven patterns reduces the risk of architectural failure. You're not inventing new approaches for common problems.

5. Team Alignment

Patterns give teams a shared understanding of system structure, enabling parallel work and clear ownership boundaries.

When to Choose a Pattern

- **Start Simple:** Begin with simpler patterns (layered, monolith)
- **Evolve When Needed:** Add complexity when justified by pain points
- **Match to Problem:** Different patterns solve different problems
- **Consider Team:** Pattern must match team size and experience
- **Plan for Change:** Architecture will evolve; make it changeable

---

[Page 5]

Chapter 3: Classification of Architectural Patterns

The Seven Categories

Category 1: Layered Architecture Patterns

These patterns organize code into layers with clear responsibilities. They separate concerns and enforce dependency rules.

- Layered (N-Tier) Architecture
- Hexagonal Architecture (Ports & Adapters)
- Onion Architecture

- Clean Architecture

Category 2: Microservices Architecture Patterns

These patterns focus on building systems as independent, deployable services.

- Monolithic Architecture
- Microservices Architecture
- Service-Oriented Architecture (SOA)
- Self-Contained Systems (SCS)
- Space-Based Architecture

Category 3: Event-Driven Patterns

These patterns use events to decouple producers from consumers.

- Event-Driven Architecture (EDA)
- Event Sourcing
- Command Query Responsibility Segregation (CQRS)
- Publish-Subscribe Architecture

Category 4: Message-Based Patterns

These patterns use message brokers for communication between components.

- Message Bus Architecture
- Message Queue Architecture
- Enterprise Service Bus (ESB)

Category 5: Distributed Systems Patterns

These patterns address the challenges of building systems across multiple nodes.

- Peer-to-Peer Architecture
- Master-Slave Architecture
- Shared-Nothing Architecture
- Sharding Pattern

Category 6: Modern Architecture Patterns

These patterns emerged with cloud computing and new technologies.

- Serverless Architecture
- Event-Driven Microservices
- Stream Processing Architecture
- Data Mesh Architecture
- Cell-Based Architecture

Category 7: Cross-Cutting Patterns

These patterns address concerns across the entire architecture.

- Circuit Breaker
- API Gateway
- Strangler Pattern
- Sidecar Pattern
- Ambassador Pattern

In the following chapters, we will explore each of these patterns in detail, with definitions, explanations, use cases, advantages, disadvantages, and diagrams to help you visualize each pattern.

[Page 6]

PART TWO: LAYERED ARCHITECTURE PATTERNS

Chapter 4: Layered (N-Tier) Architecture

Detailed Explanation

The Layered Architecture pattern organizes software into horizontal layers, where each layer has a specific responsibility. Requests flow from the top layer down to the bottom layer, with each layer only communicating with the layer directly below it.

Understanding the Problem

As applications grow, code becomes tangled:

- Business logic mixed with presentation code
- Database logic scattered everywhere
- Changes in one area break unrelated features
- Hard to test, hard to maintain

The Solution

Separate concerns into distinct layers with clear boundaries:

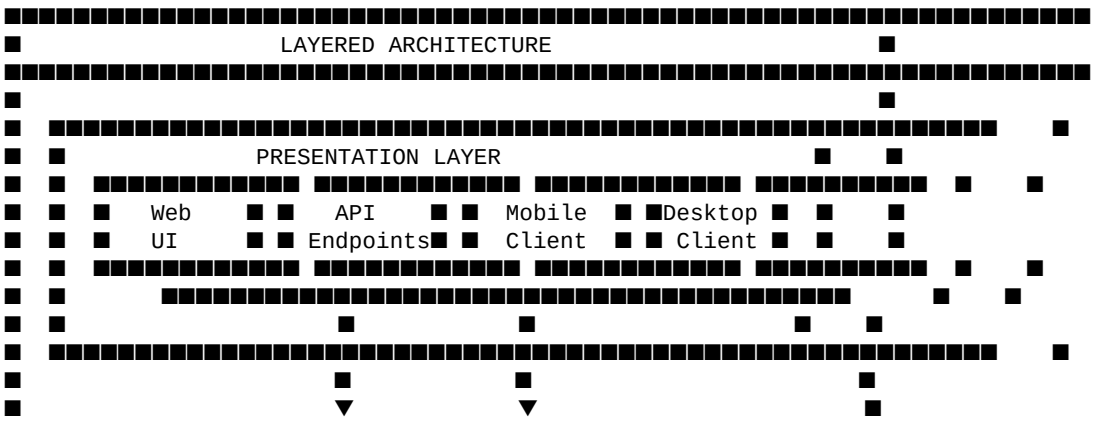
Typical Layers:

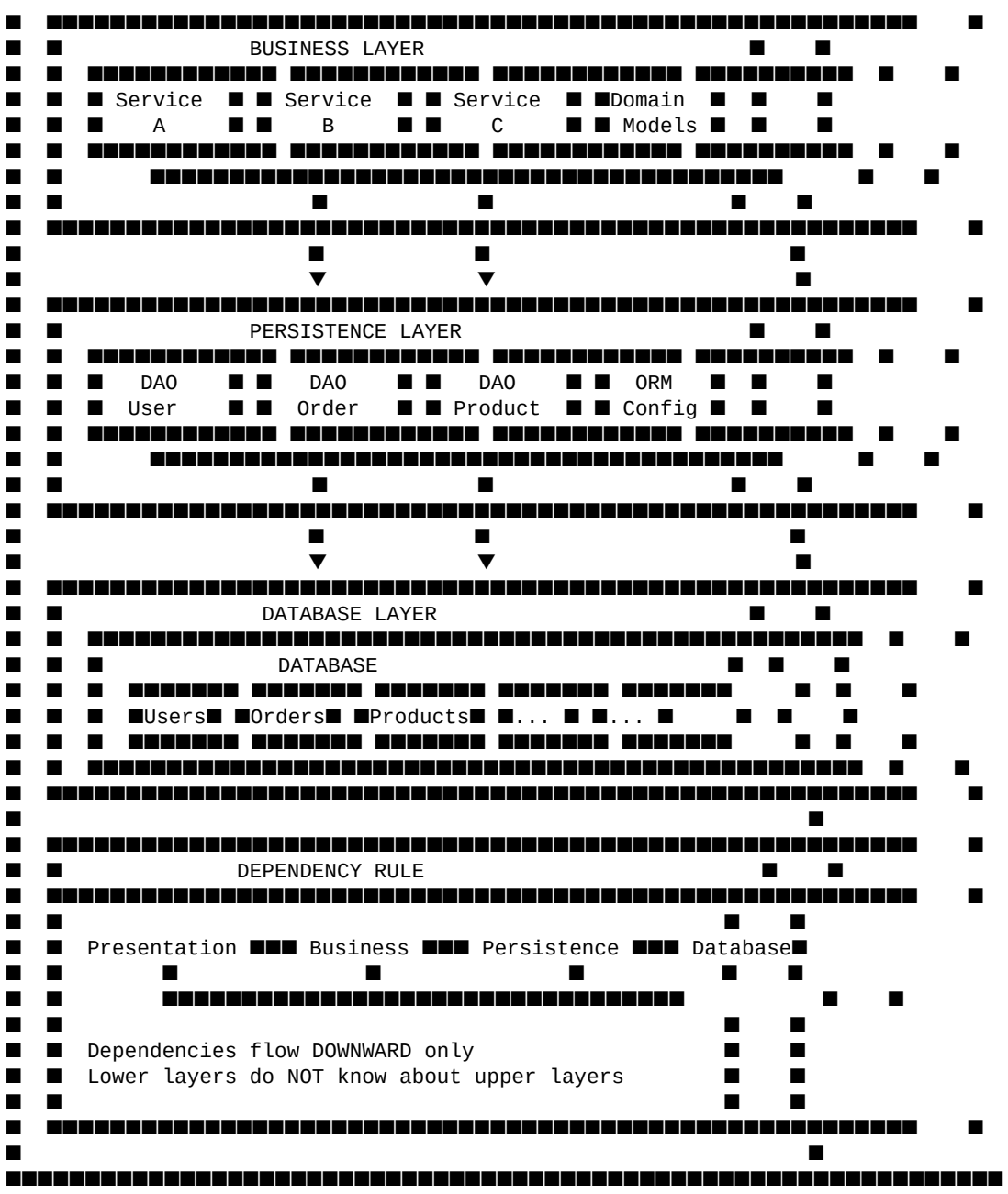
1. **Presentation Layer:** User interface, handles user input, displays output
2. **Business Layer:** Business logic, rules, calculations
3. **Persistence Layer:** Data access, database operations
4. **Database Layer:** Actual database storage

Layer Communication Rules:

- Each layer only talks to the layer directly below it
- No skipping layers (though sometimes allowed for performance)
- Dependencies flow downward
- Upper layers depend on lower layers

Diagram: Layered Architecture





Implementation Example

```
// ========== PRESENTATION LAYER ==========
@RestController
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/users/{id}")
    public UserDto getUser(@PathVariable Long id) {
        User user = userService.getUser(id);
        return new UserDto(user.getId(), user.getName(), user.getEmail());
    }
```

```
    @PostMapping("/users")
    public UserDto createUser(@RequestBody CreateUserRequest request) {
        User user = userService.createUser(request.getName(), request.getEmail());
        return new UserDto(user.getId(), user.getName(), user.getEmail());
    }
}

// ========== BUSINESS LAYER ==========
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User getUser(Long id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
    }

    public User createUser(String name, String email) {
        // Business validation
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidUserException("Name cannot be empty");
        }
        if (!email.contains("@")) {
            throw new InvalidUserException("Invalid email format");
        }

        // Check for duplicate email
        if (userRepository.existsByEmail(email)) {
            throw new DuplicateEmailException(email);
        }

        User user = new User(name, email);
        return userRepository.save(user);
    }
}

// ========== PERSISTENCE LAYER ==========
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}

// ========== DOMAIN MODEL ==========
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    protected User() {}

    public User(String name, String email) {
        this.name = name;
```

```
            this.email = email;
        }

        // Getters
        public Long getId() { return id; }
        public String getName() { return name; }
        public String getEmail() { return email; }
    }
```

When to Use Layered Architecture

Suitable Situations:

1. For simple business applications with clear separation of concerns.

2. When the team is familiar with traditional enterprise development.

3. For applications that don't require complex domain logic.

4. When rapid development is more important than long-term flexibility.

5. For internal tools and line-of-business applications.

Real-World Use Cases:

- **CRUD Applications:** Simple data entry and reporting
- **Content Management Systems:** CMS platforms
- **E-commerce Basics:** Product catalogs, simple shopping carts
- **Internal Dashboards:** Admin interfaces, reporting tools
- **Prototypes:** Quickly building proof-of-concepts

Advantages of Layered Architecture

1. **Separation of Concerns:** Each layer has a clear responsibility.

2. **Testability:** Layers can be tested independently.

3. **Maintainability:** Changes isolated to specific layers.

4. **Familiarity:** Widely understood pattern.

5. **Reusability:** Lower layers can be reused across applications.

Disadvantages of Layered Architecture

1. **Rigid Structure:** Changes may ripple through layers.

2. **Performance Overhead:** Extra layers add latency.

3. **Leaky Abstractions:** Lower layer details may leak upward.

4. **Monolithic Deployment:** Whole application deployed together.

5. **Tight Coupling:** Layers depend on each other.

Comparison with Other Patterns

Pattern	Purpose	Difference
Layered	Separation of concerns	Vertical layers
Hexagonal	Decoupling from external	Ports and adapters
Clean	Dependency inversion	Domain-centric
Onion	Core domain focus	Inner layers independent

Common Variations

1. **Closed Layers:** Each layer only talks to layer directly below.

2. **Open Layers:** Layers can bypass intermediate layers.

3. **Three-Tier:** Presentation, Business, Data (most common).

4. **N-Tier:** More than three layers (adds integration, service layers).

Best Practices

1. **Define Clear Responsibilities:** Each layer has one purpose.

2. **Avoid Leakage:** Presentation shouldn't know database details.

3. **Use Dependency Injection:** Manage dependencies between layers.

4. **Keep Layers Thin:** Each layer should do one thing well.

5. **Test in Isolation:** Mock lower layers when testing upper layers.

[Page 7]

Chapter 5: Hexagonal Architecture (Ports & Adapters)

Detailed Explanation

Hexagonal Architecture, also known as Ports & Adapters, isolates the core domain from external concerns like databases, user interfaces, and external services. The domain communicates with the outside world through ports and adapters.

Understanding the Problem

In traditional layered architectures, the domain (business logic) often becomes coupled to infrastructure:

- Database logic mixed with business rules
- UI changes require domain changes
- Hard to test domain logic without infrastructure
- Difficulty swapping technologies

The Solution

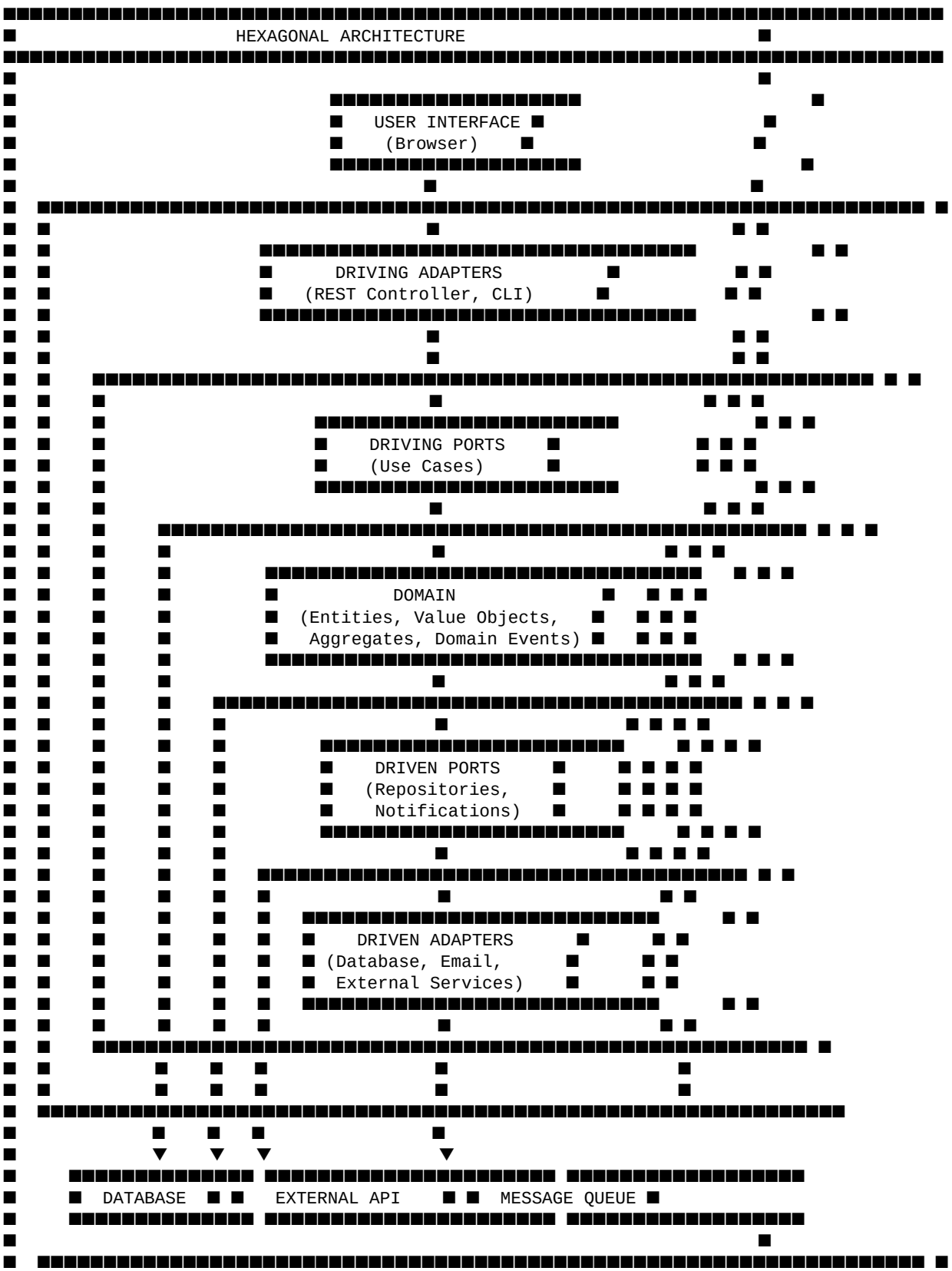
Hexagonal Architecture places the domain at the center. External systems connect through ports (interfaces) and adapters (implementations). The domain knows nothing about how data is persisted or presented.

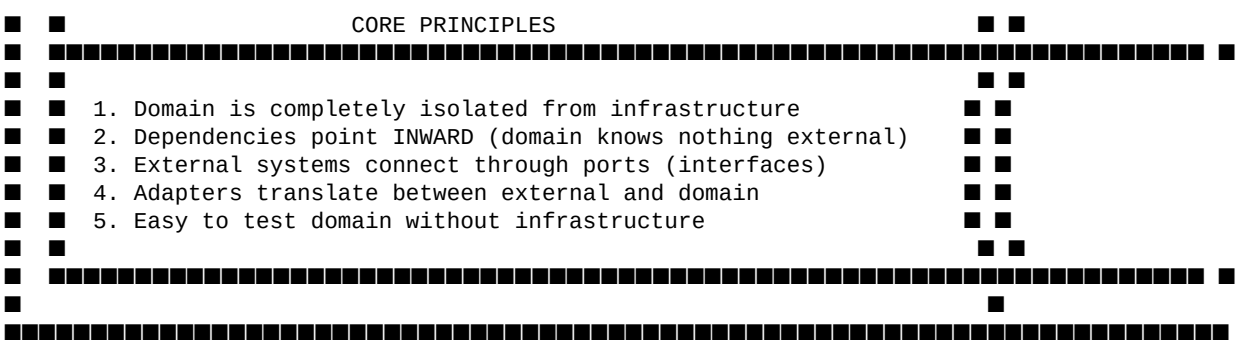
Key Concepts

- **Domain:** Core business logic, completely isolated
- **Port:** Interface defining how to interact with the domain
- **Adapter:** Implementation connecting external systems to ports
- **Driving Side (Left):** Initiates actions (UI, tests, APIs)

- **Driven Side (Right):** Responds to actions (database, external services)

Diagram: Hexagonal Architecture





Implementation Example

```
// ========== DOMAIN LAYER (CORE) ==========

// Entity
public class User {
    private final UserId id;
    private String name;
    private Email email;
    private boolean active;

    public User(UserId id, String name, Email email) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.active = true;
    }

    public void deactivate() {
        this.active = false;
    }

    public void changeEmail(Email newEmail) {
        this.email = newEmail;
    }

    // Getters
    public UserId getId() { return id; }
    public String getName() { return name; }
    public Email getEmail() { return email; }
    public boolean isActive() { return active; }
}

// Value Object
public class Email {
    private final String value;

    public Email(String value) {
        if (value == null || !value.contains("@")) {
            throw new IllegalArgumentException("Invalid email");
        }
        this.value = value;
    }

    public String getValue() { return value; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Email)) return false;
```

```
        Email email = (Email) o;
        return Objects.equals(value, email.value);
    }

    @Override
    public int hashCode() { return Objects.hash(value); }
}

// Value Object
public class UserId {
    private final UUID value;

    public UserId() {
        this.value = UUID.randomUUID();
    }

    public UserId(String value) {
        this.value = UUID.fromString(value);
    }

    public String getValue() { return value.toString(); }
}

// ========== DRIVING PORTS (INPUT) ==========

public interface UserUseCase {
    UserResponse createUser(CreateUserCommand command);
    UserResponse getUser(String userId);
    void deactivateUser(String userId);
}

// Command Objects
public class CreateUserCommand {
    private final String name;
    private final String email;

    public CreateUserCommand(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
}

// Response Objects
public class UserResponse {
    private final String id;
    private final String name;
    private final String email;
    private final boolean active;

    public UserResponse(String id, String name, String email, boolean active) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.active = active;
    }

    // Getters...
}

// ========== DRIVEN PORTS (OUTPUT) ==========

public interface UserRepository {
```

```
    User save(User user);
    Optional<User> findById(UserId id);
    Optional<User> findByEmail(Email email);
    void delete(User user);
}

public interface NotificationPort {
    void sendWelcomeEmail(Email email, String name);
    void sendAccountDeactivatedEmail(Email email, String name);
}

// ========== APPLICATION SERVICE (USE CASE IMPLEMENTATION) ==========

@Service
public class UserService implements UserUseCase {
    private final UserRepository userRepository;
    private final NotificationPort notificationPort;

    public UserService(UserRepository userRepository, NotificationPort notificationPort) {
        this.userRepository = userRepository;
        this.notificationPort = notificationPort;
    }

    @Override
    public UserResponse createUser(CreateUserCommand command) {
        // Validate
        Email email = new Email(command.getEmail());

        // Check if user exists
        if (userRepository.findByEmail(email).isPresent()) {
            throw new UserAlreadyExistsException(email.getValue());
        }

        // Create domain entity
        User user = new User(new UserId(), command.getName(), email);

        // Persist
        userRepository.save(user);

        // Send notification
        notificationPort.sendWelcomeEmail(email, command.getName());

        // Return response
        return new UserResponse(
            user.getId().getValue(),
            user.getName(),
            user.getEmail().getValue(),
            user.isActive()
        );
    }

    @Override
    public UserResponse getUser(String userId) {
        User user = userRepository.findById(new UserId(userId))
            .orElseThrow(() -> new UserNotFoundException(userId));

        return new UserResponse(
            user.getId().getValue(),
            user.getName(),
            user.getEmail().getValue(),
            user.isActive()
        );
    }

    @Override
```

```
    public void deactivateUser(String userId) {
        User user = userRepository.findById(new UserId(userId))
            .orElseThrow(() -> new UserNotFoundException(userId));

        user.deactivate();
        userRepository.save(user);

        notificationPort.sendAccountDeactivatedEmail(user.getEmail(), user.getName());
    }
}

// ========== DRIVING ADAPTERS (REST API) ==========

@RestController
@RequestMapping("/api/users")
public class UserController {
    private final UserUseCase userUseCase;

    public UserController(UserUseCase userUseCase) {
        this.userUseCase = userUseCase;
    }

    @PostMapping
    public ResponseEntity<UserResponse> createUser(@RequestBody CreateUserRequest request) {
        CreateUserCommand command = new CreateUserCommand(request.getName(), request.getEmail()
        UserResponse response = userUseCase.createUser(command);
        return ResponseEntity.created(URI.create("/api/users/" + response.getId()))
            .body(response);
    }

    @GetMapping("/{id}")
    public ResponseEntity<UserResponse> getUser(@PathVariable String id) {
        UserResponse response = userUseCase.getUser(id);
        return ResponseEntity.ok(response);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deactivateUser(@PathVariable String id) {
        userUseCase.deactivateUser(id);
        return ResponseEntity.noContent().build();
    }
}

// Request DTO
public class CreateUserRequest {
    private String name;
    private String email;

    // Getters and setters...
}

// ========== DRIVEN ADAPTERS ==========

@Repository
public class JpaUserRepository implements UserRepository {
    private final SpringDataJpaRepository jpaRepository;

    public JpaUserRepository(SpringDataJpaRepository jpaRepository) {
        this.jpaRepository = jpaRepository;
    }

    @Override
    public User save(User user) {
        UserEntity entity = toEntity(user);
        entity = jpaRepository.save(entity);
```

```
        return toDomain(entity);
    }

    @Override
    public Optional<User> findById(UserId id) {
        return jpaRepository.findById(id.getValue())
            .map(this::toDomain);
    }

    @Override
    public Optional<User> findByEmail(Email email) {
        return jpaRepository.findByEmail(email.getValue())
            .map(this::toDomain);
    }

    @Override
    public void delete(User user) {
        jpaRepository.deleteById(user.getId().getValue());
    }

    private UserEntity toEntity(User user) {
        return new UserEntity(
            user.getId().getValue(),
            user.getName(),
            user.getEmail().getValue(),
            user.isActive()
        );
    }

    private User toDomain(UserEntity entity) {
        User user = new User(
            new UserId(entity.getId()),
            entity.getName(),
            new Email(entity.getEmail())
        );
        if (!entity.isActive()) {
            user.deactivate();
        }
        return user;
    }
}

// JPA Entity (infrastructure)
@Entity
@Table(name = "users")
public class UserEntity {
    @Id
    private String id;
    private String name;
    private String email;
    private boolean active;

    protected UserEntity() {}

    public UserEntity(String id, String name, String email, boolean active) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.active = active;
    }

    // Getters and setters...
}

// Email Notification Adapter
```

```
@Component
public class EmailNotificationAdapter implements NotificationPort {
    private final JavaMailSender mailSender;

    public EmailNotificationAdapter(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    @Override
    public void sendWelcomeEmail(Email email, String name) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setTo(email.getValue());
        message.setSubject("Welcome to our platform!");
        message.setText("Hello " + name + ", welcome!");
        mailSender.send(message);
    }

    @Override
    public void sendAccountDeactivatedEmail(Email email, String name) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setTo(email.getValue());
        message.setSubject("Account Deactivated");
        message.setText("Hello " + name + ", your account has been deactivated.");
        mailSender.send(message);
    }
}
```

When to Use Hexagonal Architecture

Suitable Situations:

1. When you have complex business logic that should be independent of infrastructure.

2. When you need to support multiple external interfaces (web, mobile, API).

3. When you need to test the domain thoroughly without external dependencies.

4. When you may need to swap out infrastructure technologies.

5. When following Domain-Driven Design.

Real-World Use Cases:

- **Financial Systems:** Complex rules that must be tested
- **E-commerce Platforms:** Multiple interfaces (web, mobile, third-party)
- **Healthcare Systems:** Critical business logic
- **Insurance Systems:** Complex calculations and rules
- **Any application with rich domain logic**

Advantages of Hexagonal Architecture

1. **Domain Independence:** Core logic doesn't depend on infrastructure.

2. **Testability:** Domain can be tested in isolation.

3. **Technology Flexibility:** Can swap databases, frameworks easily.

4. **Multiple Interfaces:** Easily support different clients.

5. **Clean Separation:** Clear boundaries between responsibilities.

Disadvantages of Hexagonal Architecture

1. **Complexity:** More indirection than layered architecture.

2. **Overhead:** More code for interfaces and adapters.

3. **Learning Curve:** Requires understanding of the pattern.

4. **Not for Simple CRUD:** Overkill for simple data entry.

5. **Indirection:** May be harder to navigate codebase.

Comparison with Other Patterns

Pattern	Purpose	Difference
---	---	---
Hexagonal	Decouple domain from infrastructure	Ports and adapters
Layered	Separation of concerns	Vertical layers
Clean	Dependency inversion	Domain-centric
Onion	Domain core	Similar concept

[Page 8]

Chapter 6: Onion Architecture

Detailed Explanation

Onion Architecture organizes code into concentric layers centered around the domain model. Each layer depends only on the layers inside it, with the domain model at the core.

Understanding the Problem

Traditional layered architectures have dependencies flowing outward (UI depends on business, which depends on data). This makes the domain dependent on infrastructure. Onion Architecture inverts this: the domain knows nothing about the outer layers.

The Solution

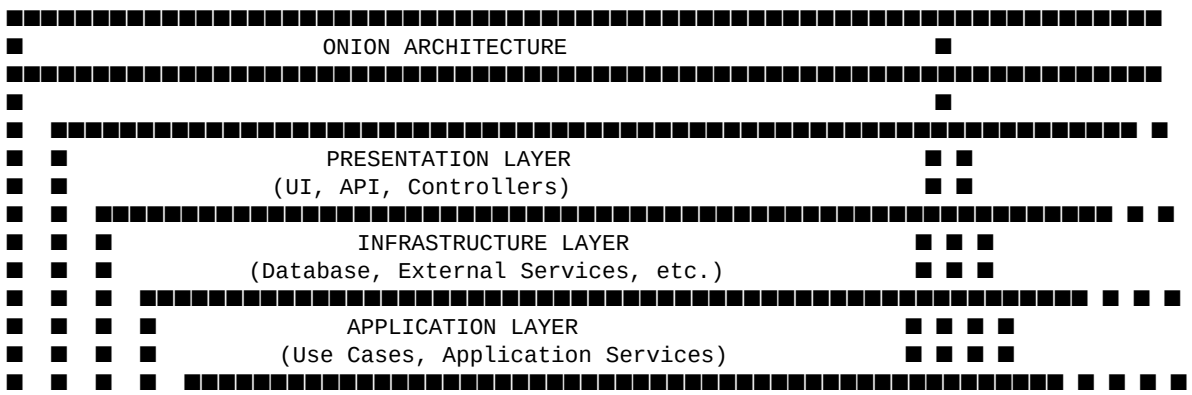
Onion Architecture places the domain at the center, with layers wrapping around it:

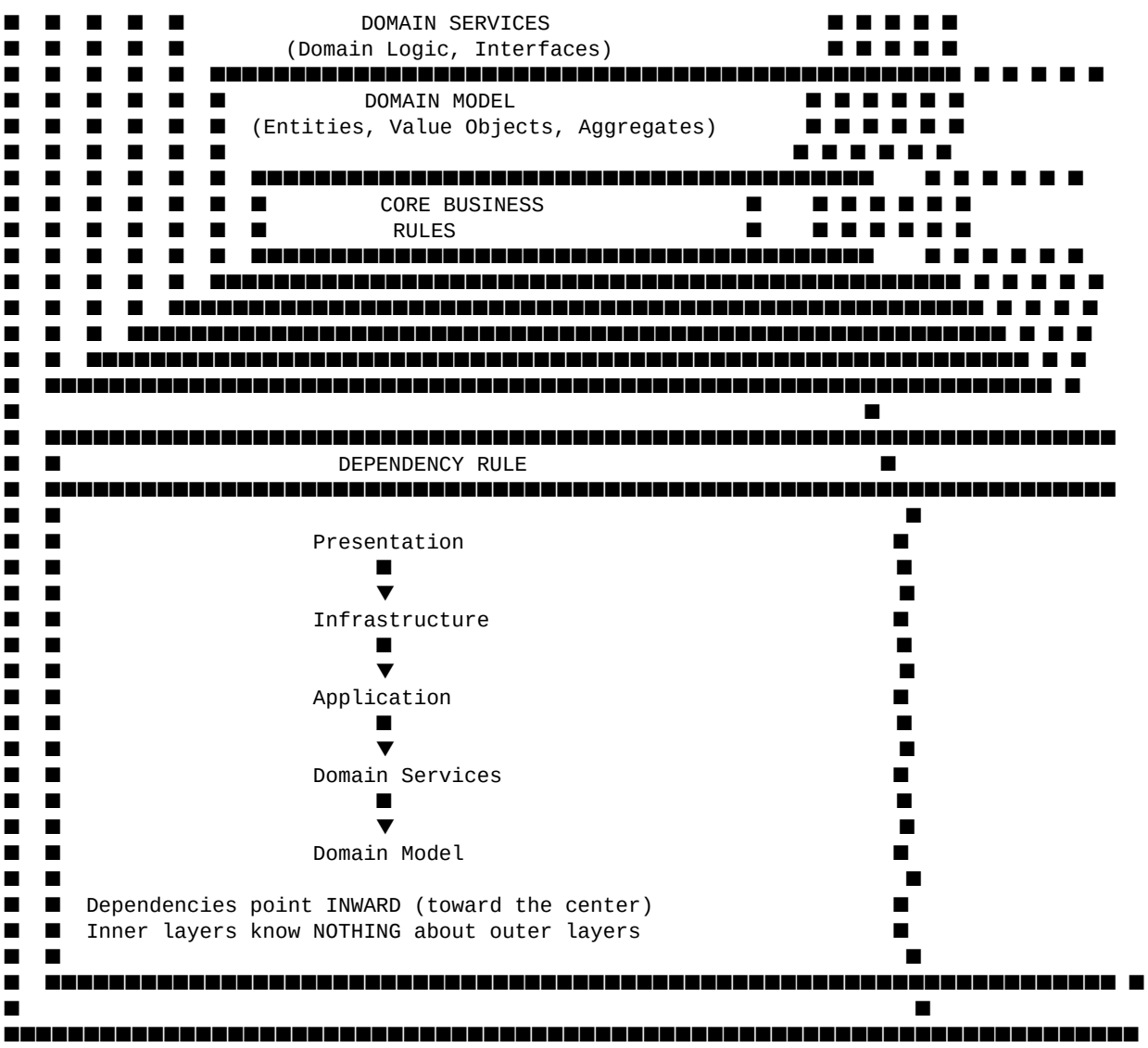
1. **Domain Model (Core):** Entities, value objects, aggregates
2. **Domain Services:** Business logic that doesn't fit in entities
3. **Application Services:** Use cases, orchestration
4. **Infrastructure:** Databases, external services
5. **Presentation:** UI, APIs

Key Principles

- **Dependencies Point Inward:** Inner layers know nothing about outer layers
- **Domain is Independent:** No dependencies on infrastructure
- **Interfaces at Boundaries:** Outer layers implement interfaces defined in inner layers
- **Testability:** Core can be tested without infrastructure

Diagram: Onion Architecture





Implementation Example

```
// ========== DOMAIN MODEL (CORE) ==========
public class Order {
    private final OrderId id;
    private final CustomerId customerId;
    private final List<OrderItem> items;
    private OrderStatus status;
    private Money total;
    private final Instant createdAt;

    public Order(OrderId id, CustomerId customerId, List<OrderItem> items) {
        this.id = id;
        this.customerId = customerId;
        this.items = Collections.unmodifiableList(items);
        this.status = OrderStatus.DRAFT;
        this.total = calculateTotal();
        this.createdAt = Instant.now();
    }

    public Money calculateTotal() {
        return items.stream()
```

```
                .map(OrderItem::getSubtotal)
                .reduce(Money.ZERO, Money::add);
    }

    public void submit() {
        if (status != OrderStatus.DRAFT) {
            throw new InvalidOrderStateException("Order already submitted");
        }
        if (items.isEmpty()) {
            throw new EmptyOrderException("Cannot submit empty order");
        }
        this.status = OrderStatus.SUBMITTED;
    }

    public void approve() {
        if (status != OrderStatus.SUBMITTED) {
            throw new InvalidOrderStateException("Order not submitted");
        }
        this.status = OrderStatus.APPROVED;
    }

    public void ship() {
        if (status != OrderStatus.APPROVED) {
            throw new InvalidOrderStateException("Order not approved");
        }
        this.status = OrderStatus.SHIPPED;
    }

    public void cancel() {
        if (status == OrderStatus.SHIPPED) {
            throw new InvalidOrderStateException("Cannot cancel shipped order");
        }
        this.status = OrderStatus.CANCELLED;
    }

    // Getters...
}

public class OrderItem {
    private final ProductId productId;
    private final int quantity;
    private final Money unitPrice;

    public OrderItem(ProductId productId, int quantity, Money unitPrice) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("Quantity must be positive");
        }
        this.productId = productId;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
    }

    public Money getSubtotal() {
        return unitPrice.multiply(quantity);
    }

    // Getters...
}

public enum OrderStatus {
    DRAFT, SUBMITTED, APPROVED, SHIPPED, CANCELLED
}

// Value Objects
public class OrderId {
```

```
    private final UUID value;
    public OrderId() { this.value = UUID.randomUUID(); }
    public OrderId(String value) { this.value = UUID.fromString(value); }
    public String getValue() { return value.toString(); }
}

public class Money {
    private final BigDecimal amount;
    private final Currency currency;
    public static final Money ZERO = new Money(BigDecimal.ZERO, Currency.USD);

    public Money(BigDecimal amount, Currency currency) {
        this.amount = amount;
        this.currency = currency;
    }

    public Money add(Money other) {
        if (!currency.equals(other.currency)) {
            throw new CurrencyMismatchException();
        }
        return new Money(amount.add(other.amount), currency);
    }

    public Money multiply(int factor) {
        return new Money(amount.multiply(BigDecimal.valueOf(factor)), currency);
    }
}

// ========== DOMAIN SERVICES ==========
public interface OrderRepository {
    Order save(Order order);
    Optional<Order> findById(OrderId id);
}

public interface InventoryService {
    boolean reserveItems(OrderId orderId, List<OrderItem> items);
    void releaseItems(OrderId orderId);
}

public interface PaymentService {
    PaymentResult processPayment(OrderId orderId, Money amount);
    void refund(OrderId orderId);
}

// ========== APPLICATION LAYER (USE CASES) ==========
public class SubmitOrderUseCase {
    private final OrderRepository orderRepository;
    private final InventoryService inventoryService;
    private final PaymentService paymentService;

    public SubmitOrderUseCase(OrderRepository orderRepository,
                              InventoryService inventoryService,
                              PaymentService paymentService) {
        this.orderRepository = orderRepository;
        this.inventoryService = inventoryService;
        this.paymentService = paymentService;
    }

    public SubmitOrderResult execute(SubmitOrderCommand command) {
        OrderId orderId = new OrderId(command.getOrderId());
        Order order = orderRepository.findById(orderId)
            .orElseThrow(() -> new OrderNotFoundException(orderId));

        // Domain logic
        order.submit();
```

```
        // Infrastructure operations
        boolean inventoryReserved = inventoryService.reserveItems(orderId, order.getItems());
        if (!inventoryReserved) {
            order.cancel();
            orderRepository.save(order);
            return SubmitOrderResult.failure("Inventory not available");
        }

        PaymentResult payment = paymentService.processPayment(orderId, order.getTotal());
        if (!payment.isSuccessful()) {
            inventoryService.releaseItems(orderId);
            order.cancel();
            orderRepository.save(order);
            return SubmitOrderResult.failure("Payment failed: " + payment.getError());
        }

        orderRepository.save(order);
        return SubmitOrderResult.success(orderId);
    }
}

// ========== APPLICATION DTOs ==========
public class SubmitOrderCommand {
    private final String orderId;

    public SubmitOrderCommand(String orderId) {
        this.orderId = orderId;
    }

    public String getOrderId() { return orderId; }
}

public class SubmitOrderResult {
    private final boolean success;
    private final String message;
    private final String orderId;

    private SubmitOrderResult(boolean success, String message, String orderId) {
        this.success = success;
        this.message = message;
        this.orderId = orderId;
    }

    public static SubmitOrderResult success(OrderId orderId) {
        return new SubmitOrderResult(true, null, orderId.getValue());
    }

    public static SubmitOrderResult failure(String message) {
        return new SubmitOrderResult(false, message, null);
    }
}

// ========== INFRASTRUCTURE LAYER ==========
@Repository
public class JpaOrderRepository implements OrderRepository {
    private final SpringDataJpaRepository jpaRepository;
    private final OrderMapper mapper;

    public JpaOrderRepository(SpringDataJpaRepository jpaRepository, OrderMapper mapper) {
        this.jpaRepository = jpaRepository;
        this.mapper = mapper;
    }

    @Override
    public Order save(Order order) {
```

```
            OrderEntity entity = mapper.toEntity(order);
            entity = jpaRepository.save(entity);
            return mapper.toDomain(entity);
        }

        @Override
        public Optional<Order> findById(OrderId id) {
            return jpaRepository.findById(id.getValue())
                .map(mapper::toDomain);
        }
    }

    @Service
    public class InventoryServiceAdapter implements InventoryService {
        private final ExternalInventoryApi api;

        public InventoryServiceAdapter(ExternalInventoryApi api) {
            this.api = api;
        }

        @Override
        public boolean reserveItems(OrderId orderId, List<OrderItem> items) {
            // Map domain to external API
            ReserveRequest request = mapToRequest(orderId, items);
            ReserveResponse response = api.reserve(request);
            return response.isSuccessful();
        }

        @Override
        public void releaseItems(OrderId orderId) {
            api.release(orderId.getValue());
        }
    }

    // ========== PRESENTATION LAYER ==========
    @RestController
    @RequestMapping("/api/orders")
    public class OrderController {
        private final SubmitOrderUseCase submitOrderUseCase;

        public OrderController(SubmitOrderUseCase submitOrderUseCase) {
            this.submitOrderUseCase = submitOrderUseCase;
        }

        @PostMapping("/{orderId}/submit")
        public ResponseEntity<SubmitOrderResponse> submitOrder(@PathVariable String orderId) {
            SubmitOrderCommand command = new SubmitOrderCommand(orderId);
            SubmitOrderResult result = submitOrderUseCase.execute(command);

            if (result.isSuccess()) {
                return ResponseEntity.ok(new SubmitOrderResponse("success", result.getOrderId()));
            } else {
                return ResponseEntity.badRequest()
                    .body(new SubmitOrderResponse("failure", result.getMessage()));
            }
        }
    }
```

When to Use Onion Architecture

Suitable Situations:

1. When you have complex domain logic that must be protected from infrastructure.

2. When you need to maintain the system for many years.

3. When different teams own different layers.

4. When you want to delay infrastructure decisions.

5. When following Domain-Driven Design.

Real-World Use Cases:

- **Large Enterprise Systems:** Complex business rules
- **Financial Systems:** Critical domain logic
- **Legacy Modernization:** Protecting new domain from legacy systems
- **Product Development:** Where requirements evolve over time

Advantages of Onion Architecture

1. **Domain Centric:** Business logic is the main focus.

2. **Testable:** Core can be tested without infrastructure.

3. **Flexible:** Easy to swap infrastructure.

4. **Clear Boundaries:** Well-defined separation of concerns.

5. **Maintainable:** Changes isolated to layers.

Disadvantages of Onion Architecture

1. **Complexity:** More layers than layered architecture.

2. **Learning Curve:** Requires understanding the pattern.

3. **Overhead:** More code for interfaces and mappings.

4. **Not for Simple CRUD:** Overkill for simple applications.

5. **Navigation:** Harder to follow code flow.

Comparison with Other Patterns

Pattern	Core Focus	Dependency Direction
---	---	---
Layered	Separation	Outward
Hexagonal	Ports/Adapters	Inward to domain
Onion	Domain	Inward to domain
Clean	Use Cases	Inward to entities

[Page 9]

Chapter 7: Clean Architecture

Detailed Explanation

Clean Architecture, introduced by Robert C. Martin (Uncle Bob), organizes code into concentric circles with the most stable concepts at the center. It emphasizes dependency inversion and independence from frameworks.

Understanding the Problem

Applications become dependent on frameworks, databases, and external libraries, making them hard to test and change. When requirements change, changing the framework becomes nearly impossible.

The Solution

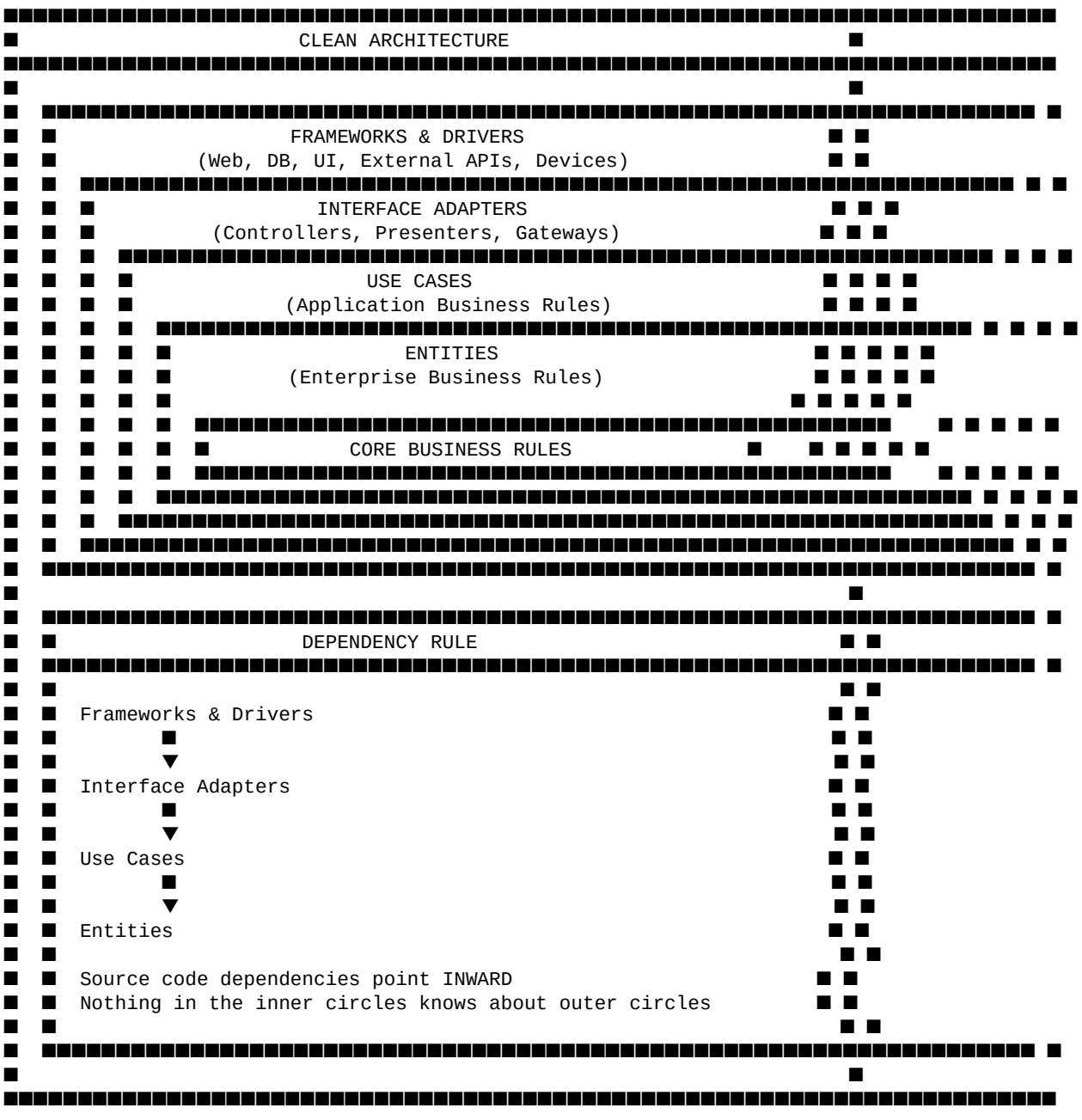
Clean Architecture defines four layers (circles) with strict dependency rules:

1. **Entities:** Enterprise-wide business rules (most stable)
2. **Use Cases:** Application-specific business rules
3. **Interface Adapters:** Convert data between use cases and external world
4. **Frameworks & Drivers:** Databases, web frameworks, UI (most volatile)

Key Principles

- **Dependency Rule:** Dependencies point inward (outer layers depend on inner)
- **Independence from Frameworks:** Core doesn't know about frameworks
- **Testability:** Core can be tested without UI, database, etc.
- **Independence from UI:** UI can change without affecting core
- **Independence from Database:** Database can be swapped

Diagram: Clean Architecture



Implementation Example

```
// ========== ENTITIES LAYER ==========
public class Product {
    private final ProductId id;
    private String name;
    private Money price;
    private int stockQuantity;
    private boolean active;

    public Product(ProductId id, String name, Money price, int stockQuantity) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.stockQuantity = stockQuantity;
        this.active = true;
    }

    public void updatePrice(Money newPrice) {
        if (newPrice.isLessThanOrEqualTo(Money.ZERO)) {
            throw new BusinessRuleException("Price must be positive");
        }
        this.price = newPrice;
    }

    public void reduceStock(int quantity) {
        if (quantity <= 0) {
            throw new BusinessRuleException("Quantity must be positive");
        }
        if (stockQuantity < quantity) {
            throw new BusinessRuleException("Insufficient stock");
        }
        this.stockQuantity -= quantity;
    }

    public void increaseStock(int quantity) {
        if (quantity <= 0) {
            throw new BusinessRuleException("Quantity must be positive");
        }
        this.stockQuantity += quantity;
    }

    public void deactivate() {
        this.active = false;
    }

    // Getters...
}

// ========== USE CASES LAYER ==========
public interface ProductRepository {
    Product save(Product product);
    Optional<Product> findById(ProductId id);
    List<Product> findAllActive();
}

public interface StockNotificationService {
    void notifyLowStock(ProductId productId, int currentStock);
}

public class UpdateProductPriceUseCase {
    private final ProductRepository productRepository;
    private final StockNotificationService stockNotificationService;
    public UpdateProductPriceUseCase(ProductRepository productRepository,
```

```
                                       StockNotificationService stockNotificationService) {
        this.productRepository = productRepository;
        this.stockNotificationService = stockNotificationService;
    }

    public UpdatePriceResponse execute(UpdatePriceCommand command) {
        Product product = productRepository.findById(new ProductId(command.getProductId()))
            .orElseThrow(() -> new ProductNotFoundException(command.getProductId()));

        Money newPrice = new Money(command.getNewPrice(), Currency.USD);
        product.updatePrice(newPrice);

        productRepository.save(product);

        // Notify if stock is low
        if (product.getStockQuantity() < 10) {
            stockNotificationService.notifyLowStock(product.getId(), product.getStockQuantity()
        }

        return new UpdatePriceResponse(product.getId().getValue(), product.getPrice().getAmount
    }
}

// Input Port (Interface for use case)
public interface UpdateProductPriceInputPort {
    UpdatePriceResponse execute(UpdatePriceCommand command);
}

// Command and Response
public class UpdatePriceCommand {
    private final String productId;
    private final BigDecimal newPrice;

    public UpdatePriceCommand(String productId, BigDecimal newPrice) {
        this.productId = productId;
        this.newPrice = newPrice;
    }

    public String getProductId() { return productId; }
    public BigDecimal getNewPrice() { return newPrice; }
}

public class UpdatePriceResponse {
    private final String productId;
    private final BigDecimal newPrice;

    public UpdatePriceResponse(String productId, BigDecimal newPrice) {
        this.productId = productId;
        this.newPrice = newPrice;
    }

    // Getters...
}

// ========== INTERFACE ADAPTERS LAYER ==========
@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final UpdateProductPriceInputPort updatePriceUseCase;

    public ProductController(UpdateProductPriceInputPort updatePriceUseCase) {
        this.updatePriceUseCase = updatePriceUseCase;
    }

    @PatchMapping("/{id}/price")
```

```
    public ResponseEntity<UpdatePriceResponseDto> updatePrice(
            @PathVariable String id,
            @RequestBody UpdatePriceRequestDto request) {

        UpdatePriceCommand command = new UpdatePriceCommand(id, request.getNewPrice());
        UpdatePriceResponse response = updatePriceUseCase.execute(command);

        return ResponseEntity.ok(new UpdatePriceResponseDto(
            response.getProductId(),
            response.getNewPrice()
        ));
    }
}

// DTOs
public class UpdatePriceRequestDto {
    private BigDecimal newPrice;
    // getters and setters...
}

public class UpdatePriceResponseDto {
    private final String productId;
    private final BigDecimal newPrice;

    public UpdatePriceResponseDto(String productId, BigDecimal newPrice) {
        this.productId = productId;
        this.newPrice = newPrice;
    }
    // getters...
}

// Presenter (transforms domain response to UI format)
public class UpdatePricePresenter {
    public UpdatePriceViewModel present(UpdatePriceResponse response) {
        return new UpdatePriceViewModel(
            response.getProductId(),
            response.getNewPrice(),
            "Price updated successfully"
        );
    }
}

public class UpdatePriceViewModel {
    private final String productId;
    private final BigDecimal newPrice;
    private final String message;

    public UpdatePriceViewModel(String productId, BigDecimal newPrice, String message) {
        this.productId = productId;
        this.newPrice = newPrice;
        this.message = message;
    }
    // getters...
}

// ========== FRAMEWORKS & DRIVERS LAYER ==========
@Repository
public class JpaProductRepository implements ProductRepository {
    private final SpringDataJpaRepository jpaRepository;
    private final ProductMapper mapper;

    public JpaProductRepository(SpringDataJpaRepository jpaRepository, ProductMapper mapper) {
        this.jpaRepository = jpaRepository;
        this.mapper = mapper;
    }
```

```
    @Override
    public Product save(Product product) {
        ProductEntity entity = mapper.toEntity(product);
        entity = jpaRepository.save(entity);
        return mapper.toDomain(entity);
    }

    @Override
    public Optional<Product> findById(ProductId id) {
        return jpaRepository.findById(id.getValue())
            .map(mapper::toDomain);
    }

    @Override
    public List<Product> findAllActive() {
        return jpaRepository.findByActiveTrue()
            .stream()
            .map(mapper::toDomain)
            .collect(Collectors.toList());
    }
}

@Service
public class EmailStockNotificationService implements StockNotificationService {
    private final JavaMailSender mailSender;

    public EmailStockNotificationService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    @Override
    public void notifyLowStock(ProductId productId, int currentStock) {
        // Implementation
    }
}

// Configuration
@Configuration
public class ArchitectureConfiguration {

    @Bean
    public UpdateProductPriceInputPort updateProductPriceUseCase(
            ProductRepository productRepository,
            StockNotificationService stockNotificationService) {
        return new UpdateProductPriceUseCase(productRepository, stockNotificationService);
    }
}
```

When to Use Clean Architecture

Suitable Situations:

1. For complex applications with significant business logic.

2. When you need to maintain the system for many years.

3. When you want to delay decisions about frameworks and databases.

4. When you need high testability.

5. When you have multiple delivery mechanisms (web, mobile, API).

Real-World Use Cases:

- **Enterprise Applications:** Long-lived business systems
- **Financial Trading Systems:** Complex rules, must be testable
- **E-commerce Platforms:** Evolving business requirements
- **SaaS Products:** Multi-tenant, evolving features

Advantages of Clean Architecture

1. **Framework Independence:** Core doesn't depend on frameworks.

2. **Testability:** Each circle can be tested in isolation.

3. **UI Independence:** Can change UI without affecting core.

4. **Database Independence:** Can swap databases easily.

5. **Use Case Centric:** Business requirements drive design.

Disadvantages of Clean Architecture

1. **Complexity:** Overkill for simple applications.

2. **Learning Curve:** Requires understanding of concepts.

3. **Boilerplate:** More code for interfaces and mappings.

4. **Indirection:** May be harder to navigate.

5. **Team Training:** Entire team must understand the pattern.

Comparison with Other Patterns

Pattern	Core Focus	Key Difference

Pattern	Core Focus	Key Difference
Clean	Entities/Use Cases	Dependency inversion
Hexagonal	Ports/Adapters	Less prescriptive
Onion	Domain	Very similar
Layered	Separation	Dependencies outward

Clean vs Onion

Clean Architecture and Onion Architecture are very similar. Clean Architecture:

- More explicit about use cases

- Clearer definition of the inner circles

- Emphasizes "screaming architecture" (use cases are visible)

[Page 10]

PART THREE: MICROSERVICES ARCHITECTURE PATTERNS

Chapter 8: Monolithic Architecture

Detailed Explanation

Monolithic Architecture is a traditional approach where the entire application is built as a single, indivisible unit. All components—UI, business logic, data access—are packaged and deployed together.

Understanding the Problem

Before microservices, monoliths were the standard. They work well for many applications, especially in the early stages. The challenge comes as applications grow and teams expand.

The Solution

Monoliths can be structured in different ways:

1. **Single Process:** All code in one executable

2. **Modular Monolith:** Well-defined modules within a single deployment

3. **Distributed Monolith:** Anti-pattern (services but tightly coupled)

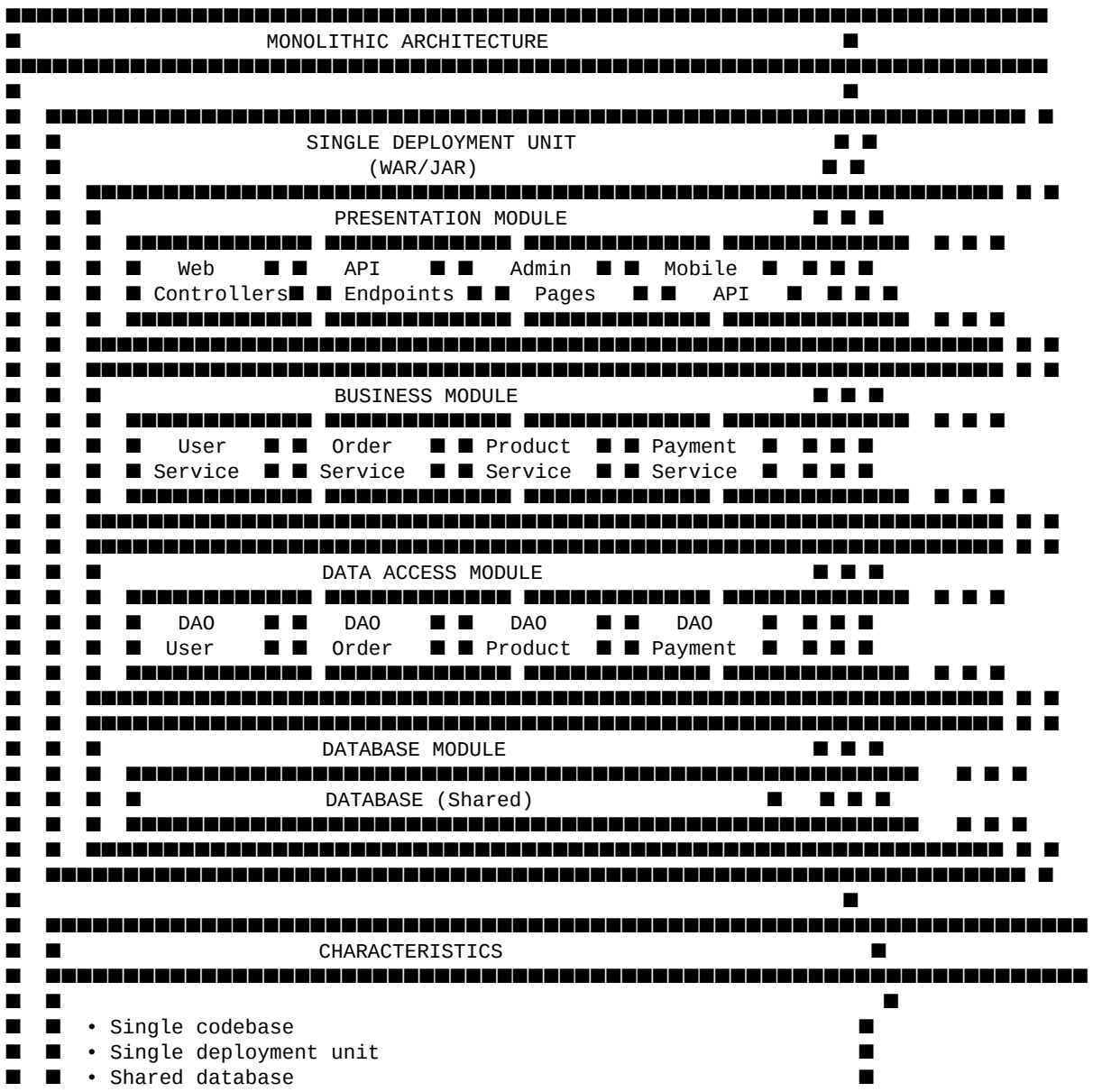
Types of Monoliths

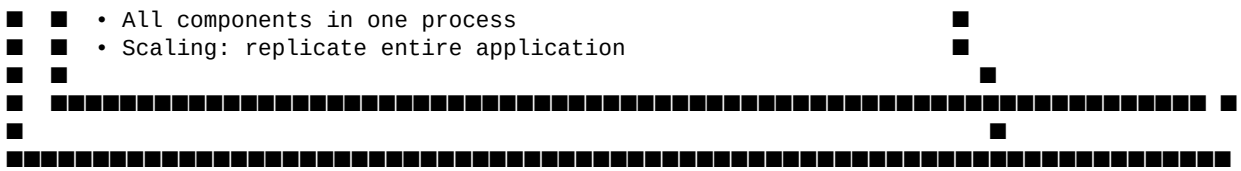
- **Single Process Monolith:** Everything in one deployment unit

- **Modular Monolith:** Logical modules with clear boundaries, but single deployment

- **Distributed Monolith:** Deployed as services but tightly coupled (worst of both)

Diagram: Monolithic Architecture





When to Use Monolithic Architecture

Suitable Situations:

1. For small to medium applications.

2. When the team is small (2-5 developers).

3. When you're building a startup (speed over scale).

4. When you don't have operational expertise for distributed systems.

5. When your business logic is simple.

Real-World Use Cases:

- **Startup MVPs:** Fast development
- **Internal Tools:** Limited scope and users
- **Simple Web Applications:** Basic CRUD operations
- **Prototypes:** Proving concepts
- **Legacy Systems:** Existing codebases

Advantages of Monolithic Architecture

1. **Simple Development:** Single codebase, easy to understand.

2. **Easy Deployment:** Deploy one artifact.

3. **Simple Testing:** End-to-end tests are straightforward.

4. **Performance:** No network calls between modules.

5. **Simple Transactions:** ACID across all modules.

6. **Easy Debugging:** Single process to debug.

Disadvantages of Monolithic Architecture

1. **Scalability:** Scale everything, not just what's needed.

2. **Deployment Coupling:** All changes require full deployment.

3. **Team Coordination:** Teams step on each other's toes.

4. **Technology Lock-in:** Single tech stack for everything.

5. **Slow Onboarding:** Large codebase hard to understand.

6. **Long Build Times:** Compiling whole application takes time.

Comparison with Microservices

Aspect	**Monolith**	**Microservices**
--------	----------	---------------
Deployment	One unit	Independent services
Scaling	All together	Individual services
Team Structure	Single team	Multiple teams
Technology	Single stack	Polyglot
Complexity	Low	High
Development Speed	Fast initially	Fast after split

Modular Monolith as Middle Ground

A modular monolith offers many benefits of microservices without the operational complexity:

```
// Well-defined module boundaries within a monolith
module user {
    exports com.example.user.api;
    provides UserService with UserServiceImpl;
}

module order {
    requires user;
    exports com.example.order.api;
}
```

[Page 11]

Chapter 9: Microservices Architecture

Detailed Explanation

Microservices Architecture structures an application as a collection of small, independent services that communicate over a network. Each service is focused on a specific business capability, can be deployed independently, and has its own database.

Understanding the Problem

Monoliths become unwieldy as they grow:

- Too many developers on one codebase
- Deployments require coordinating everyone
- Scaling inefficient (everything scales together)
- Technology lock-in

The Solution

Break the application into small, autonomous services:

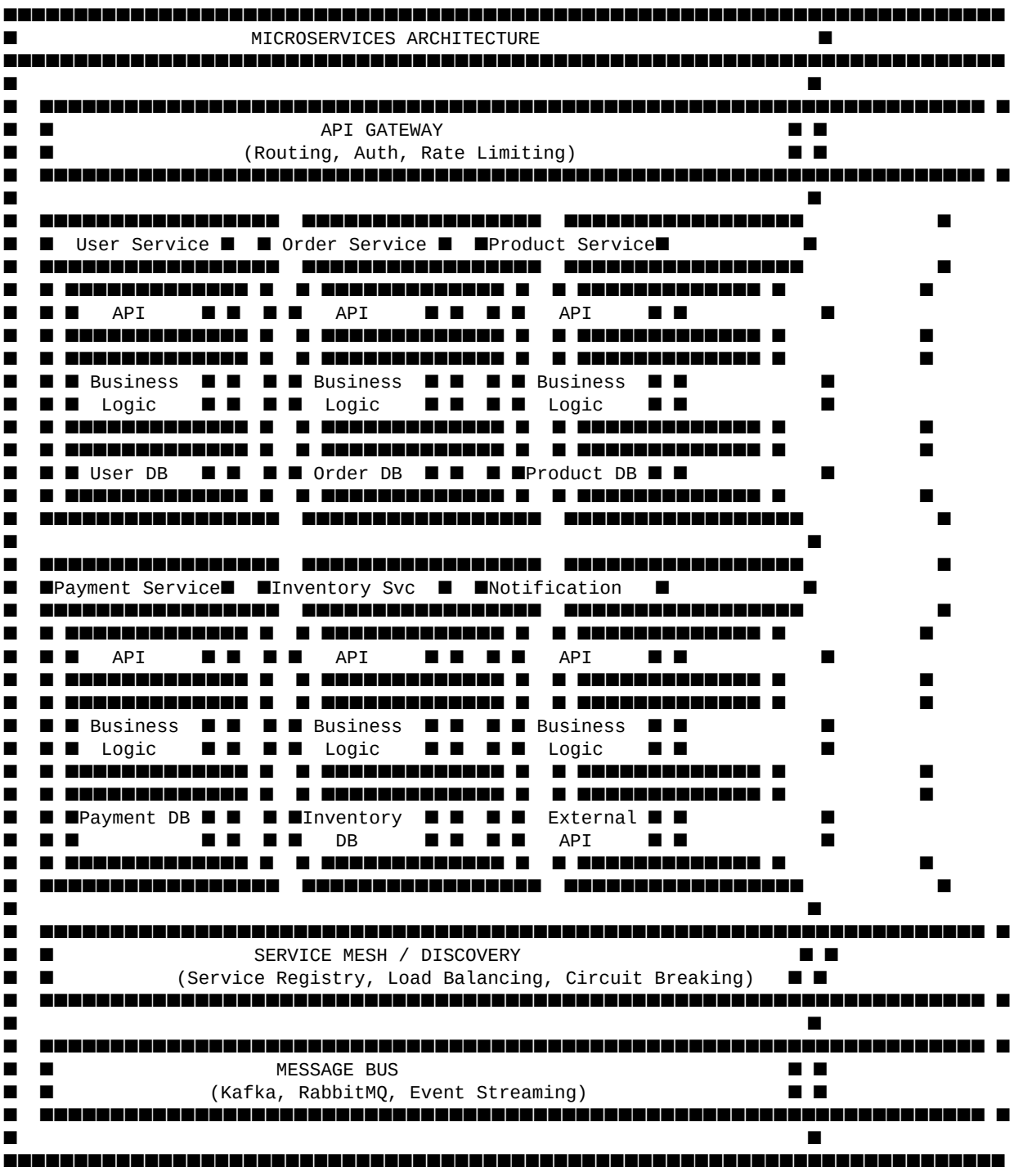
1. **Single Responsibility:** Each service does one thing well
2. **Independent Deployment:** Services deploy independently
3. **Decentralized Data:** Each service owns its database
4. **Polyglot:** Use the best technology for each service
5. **Organized Around Capabilities:** Teams own services

Key Principles

- **Decentralized Governance:** Teams choose their own technologies
- **Decentralized Data Management:** Database per service
- **Infrastructure Automation:** CI/CD, containers
- **Design for Failure:** Services may fail; handle gracefully

- **Evolutionary Design:** Can rewrite services individually

Diagram: Microservices Architecture



When to Use Microservices Architecture

Suitable Situations:

1. When the application is large and complex.

2. When you have multiple teams (5+ developers per service).

3. When different parts have different scalability needs.

4. When you need independent deployment cycles.

5. When you want to use polyglot technologies.

Real-World Use Cases:

- **Large E-commerce:** Amazon, eBay
- **Streaming Services:** Netflix, Spotify
- **Social Media:** Facebook, Twitter
- **SaaS Platforms:** Salesforce, Shopify
- **Enterprise Systems:** Complex business domains

Advantages of Microservices Architecture

1. **Independent Deployment:** Deploy services independently.

2. **Technology Diversity:** Choose best tech per service.

3. **Team Autonomy:** Teams own services end-to-end.

4. **Scalability:** Scale only what's needed.

5. **Resilience:** Failures isolated to single service.

6. **Re-writable:** Can rewrite one service at a time.

Disadvantages of Microservices Architecture

1. **Complexity:** Distributed systems are complex.

2. **Data Consistency:** Harder than ACID transactions.

3. **Network Latency:** Calls between services.

4. **Operational Overhead:** Need container orchestration.

5. **Debugging:** Harder to trace requests.

6. **Testing:** More complex to test interactions.

Comparison with Monolith

Aspect	Monolith	Microservices
--------	----------	---------------
Development	Simple start	Complex start
Deployment	One unit	Many units
Scaling	Everything	Per service
Team structure	One team	Multiple teams
Technology	One stack	Polyglot
Operations	Simple	Complex

[Page 12]

Chapter 10: Service-Oriented Architecture (SOA)

Detailed Explanation

Service-Oriented Architecture (SOA) is a design approach where application components provide services to other components via a network. It emphasizes loose coupling, reusability, and interoperability.

Understanding the Problem

Before SOA, systems were monolithic and hard to integrate. Different departments built systems that couldn't talk to each other, leading to data silos and duplication.

The Solution

SOA introduces:

- **Services:** Self-contained business functions

- **Enterprise Service Bus (ESB):** Central communication hub

- **Contracts:** Well-defined service interfaces

- **Orchestration:** Combining services into business processes

Key Concepts

- **Service:** Coarse-grained business capability

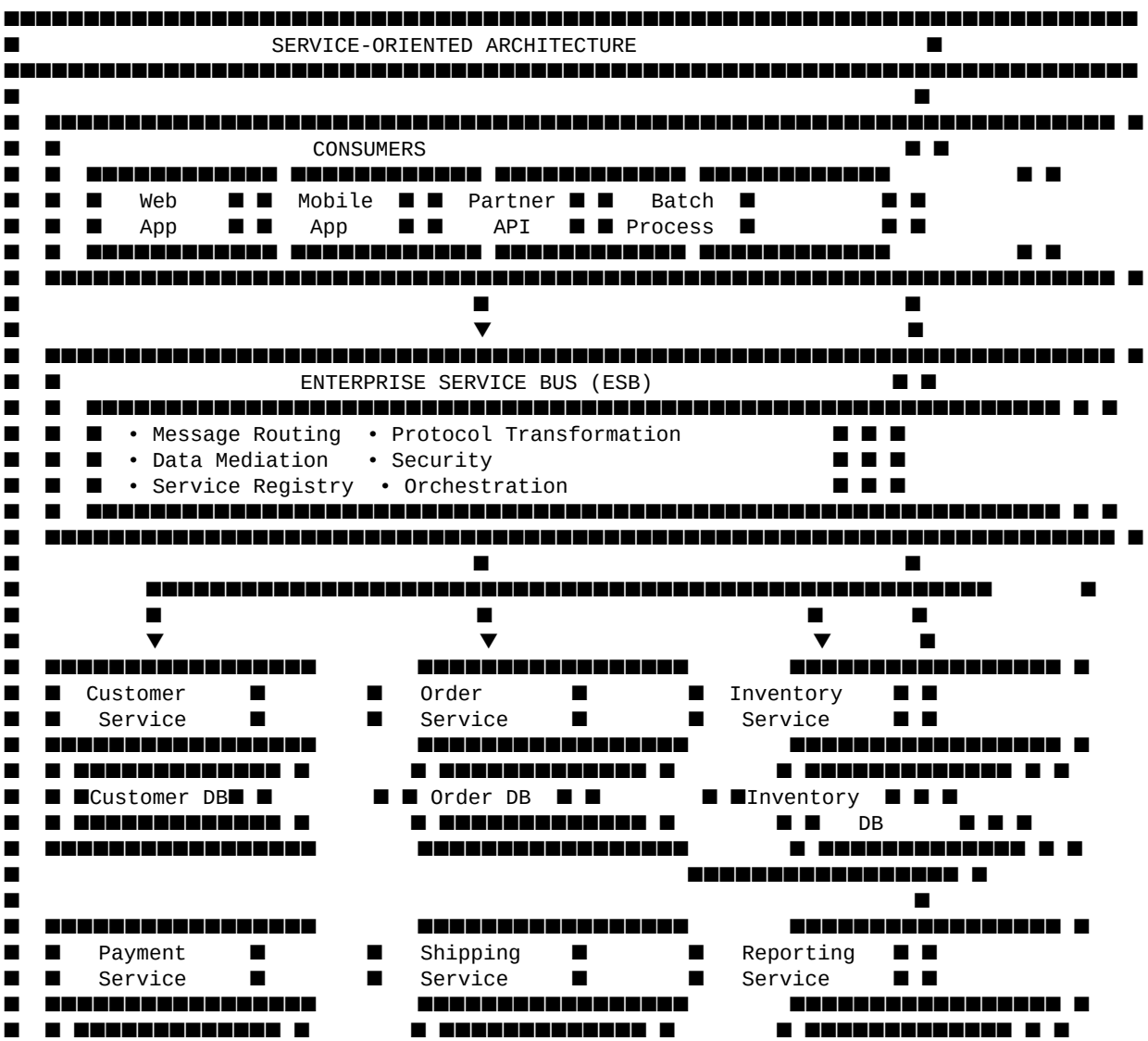
- **Contract:** Formal definition of service interface

- **ESB:** Middleware for service integration

- **Orchestration:** Coordinating multiple services

- **Governance:** Managing service lifecycle

Diagram: SOA Architecture





When to Use SOA

Suitable Situations:

1. When integrating multiple existing systems.

2. When you need to share services across the enterprise.

3. When you have a centralized IT governance team.

4. When you need protocol mediation (SOAP to JMS, etc.).

5. When you have complex orchestration requirements.

Real-World Use Cases:

- **Enterprise Application Integration:** Connecting legacy systems
- **Banking:** Integrating core banking with customer portals
- **Healthcare:** HL7 integration between systems
- **Government:** Citizen service portals
- **Telecom:** Billing and provisioning systems

Advantages of SOA

1. **Reusability:** Services can be reused across applications.

2. **Interoperability:** Works across different technologies.

3. **Governance:** Centralized management of services.

4. **Orchestration:** Compose services into business processes.

5. **Protocol Independence:** Can bridge different protocols.

Disadvantages of SOA

1. **ESB Complexity:** Central bus becomes bottleneck.

2. **Heavyweight:** XML, SOAP, WS-* standards.

3. **Slow:** Transformation overhead.

4. **Centralized Governance:** Can become bottleneck.

5. **Legacy Integration:** Often used to connect old systems.

SOA vs Microservices

Aspect	SOA	Microservices
--------	-----	----------
Communication	ESB, heavy protocols	Lightweight (REST, gRPC)
Granularity	Coarse-grained	Fine-grained
Governance	Centralized	Decentralized
Data	Shared databases	Database per service
Deployment	Often monolithic per service	Independent containers
Technology	Standardized	Polyglot

[Page 13]

Chapter 11: Self-Contained Systems (SCS)

Detailed Explanation

Self-Contained Systems (SCS) is an architecture pattern where each system is a complete, independent web application with its own UI, backend, and data storage. Integration happens through the UI (links, iframes) and backend APIs.

Understanding the Problem

Microservices solve backend complexity but don't address UI. When you split the backend, the UI often becomes a monolith that needs to know about all services.

The Solution

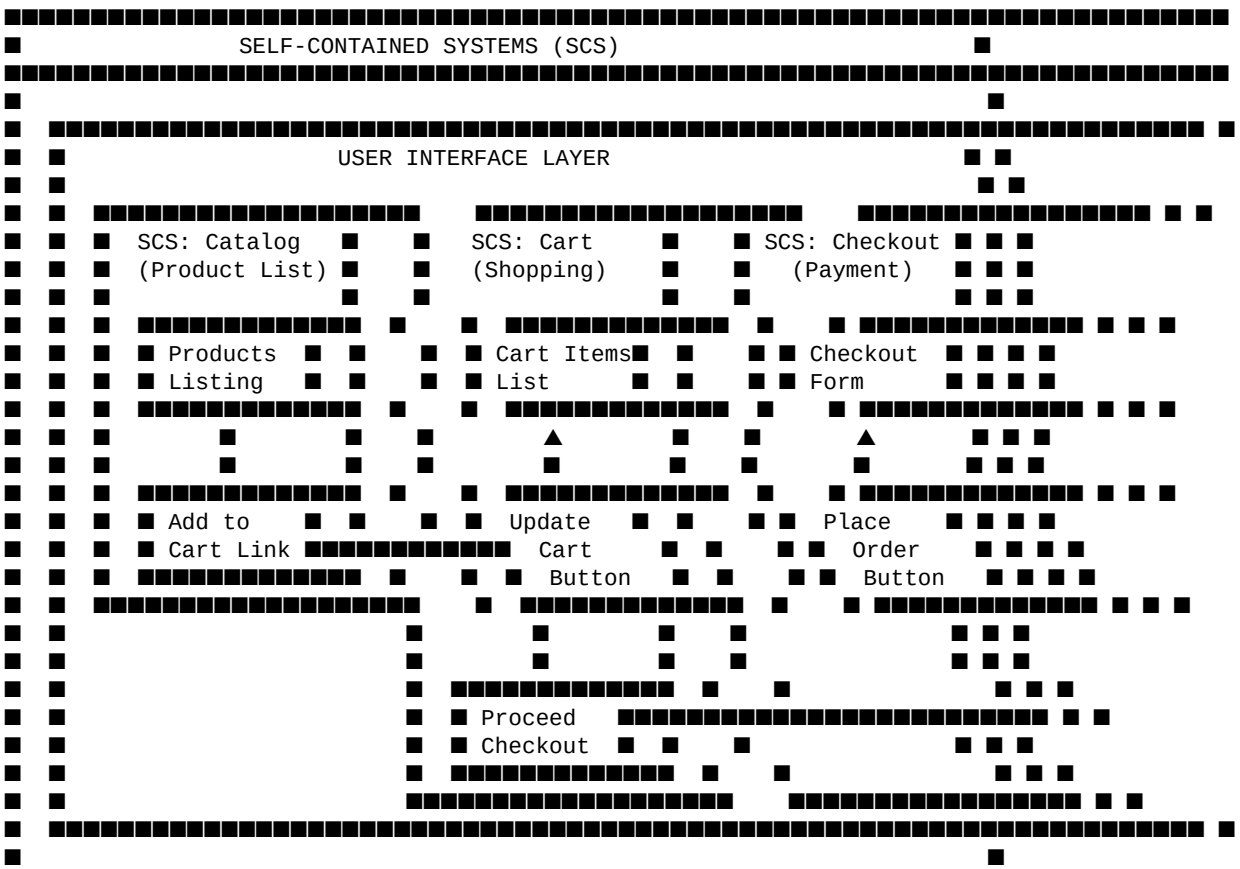
SCS creates complete systems that are:

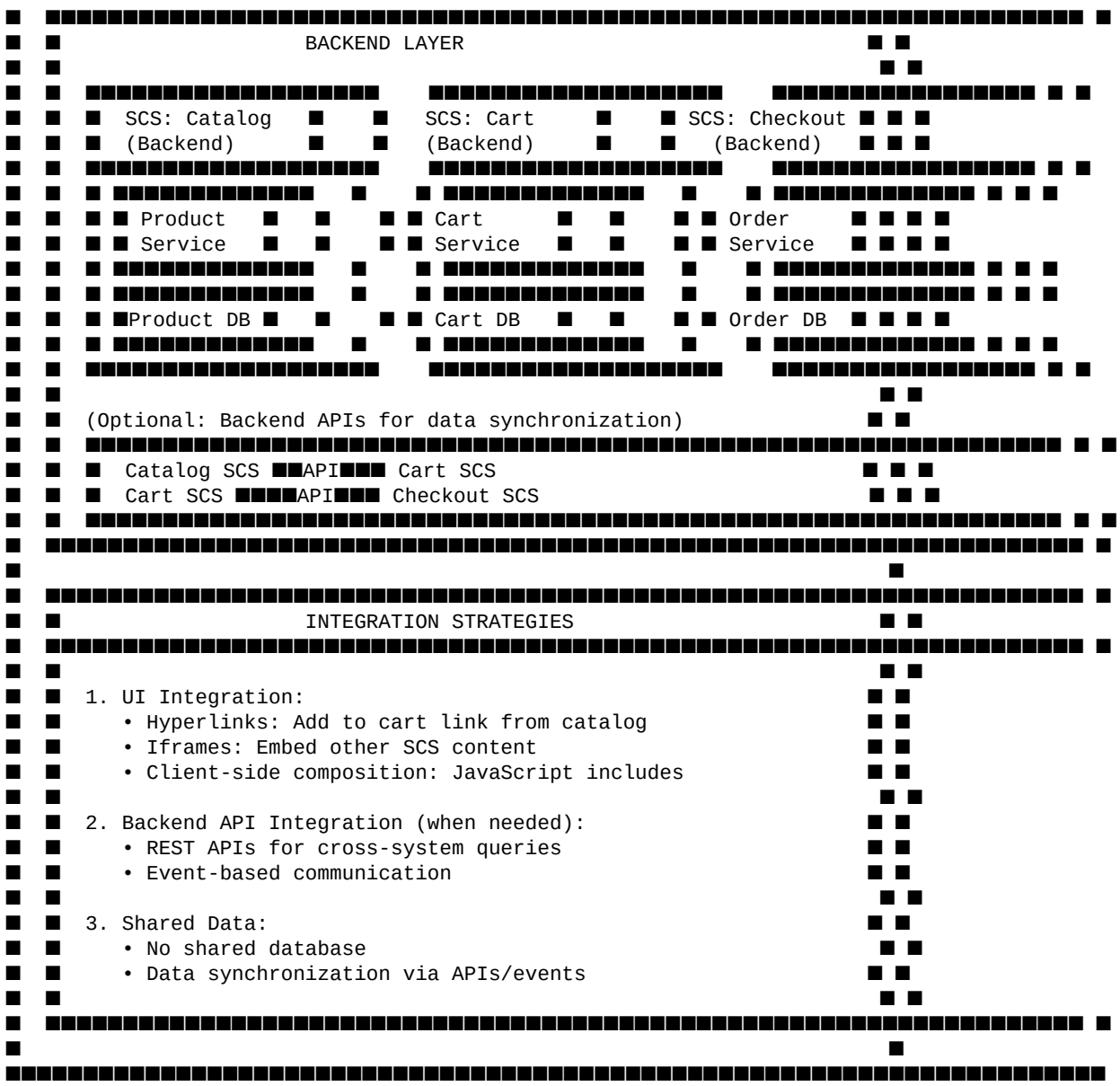
- **Self-Contained:** Each has UI, backend, and database
- **Independent:** Can be developed and deployed separately
- **Integrated via UI:** Systems link to each other
- **Shared Context:** Consistent look and feel across systems

Key Principles

- **Complete System:** Each SCS has its own UI
- **No Shared Code:** Avoid direct dependencies
- **UI Integration:** Use links, iframes, or client-side composition
- **Backend APIs:** Optional for cross-system data exchange
- **Team Autonomy:** Each team owns an SCS

Diagram: Self-Contained Systems





When to Use Self-Contained Systems

Suitable Situations:

1. When you have multiple teams that need to work independently.

2. When you want to avoid a monolithic UI.

3. When different parts of the application have different release cycles.

4. When you want to gradually replace parts of a legacy system.

5. When you need geographic distribution of teams.

Real-World Use Cases:

- **E-commerce Platforms:** Separate SCS for catalog, cart, checkout
- **Travel Booking:** Flight search, hotel search, booking
- **Banking:** Account management, payments, loans
- **Marketplaces:** Seller dashboard, buyer experience, admin

Advantages of SCS

1. **Team Autonomy:** Teams own complete systems.

2. **Independent Deployment:** Deploy SCS separately.

3. **Technology Freedom:** Each SCS can use different tech.

4. **Scalability:** Scale individual SCS.

5. **No Integration Overhead:** No ESB or complex orchestration.

Disadvantages of SCS

1. **UI Consistency:** Harder to maintain consistent look.

2. **Data Duplication:** Some data may be duplicated.

3. **Cross-System Queries:** Hard to query across SCS.

4. **Session Management:** Need to share user session across SCS.

5. **Integration Complexity:** UI integration has challenges.

SCS vs Microservices

Aspect	**SCS**	**Microservices**
--------	-----	----------
UI	Each SCS has UI	Single UI calling services
Granularity	Complete system	Backend only

Aspect	SCS	Microservices
Database	Per SCS	Per service
Deployment	Per SCS	Per service
Integration	UI links/iframes	API calls

[Page 14]

Chapter 12: Space-Based Architecture

Detailed Explanation

Space-Based Architecture (also known as Cloud Architecture) is designed to handle high scalability by using in-memory data grids and processing units that can scale horizontally. It eliminates the database as a bottleneck.

Understanding the Problem

Traditional architectures have the database as a central bottleneck. As load increases:

- Database connection limits

- Read/write contention

- Hard to scale beyond certain point

The Solution

Space-Based Architecture removes the database from the processing path:

- **Processing Units:** Contain application logic, in-memory data

- **Virtual Middleware:** Handles messaging, grid management

- **Data Grid:** Distributed, in-memory data storage

- **No Central Database:** Data partitioned across grid

Key Concepts

- **Processing Unit:** Deployable unit with app logic and data

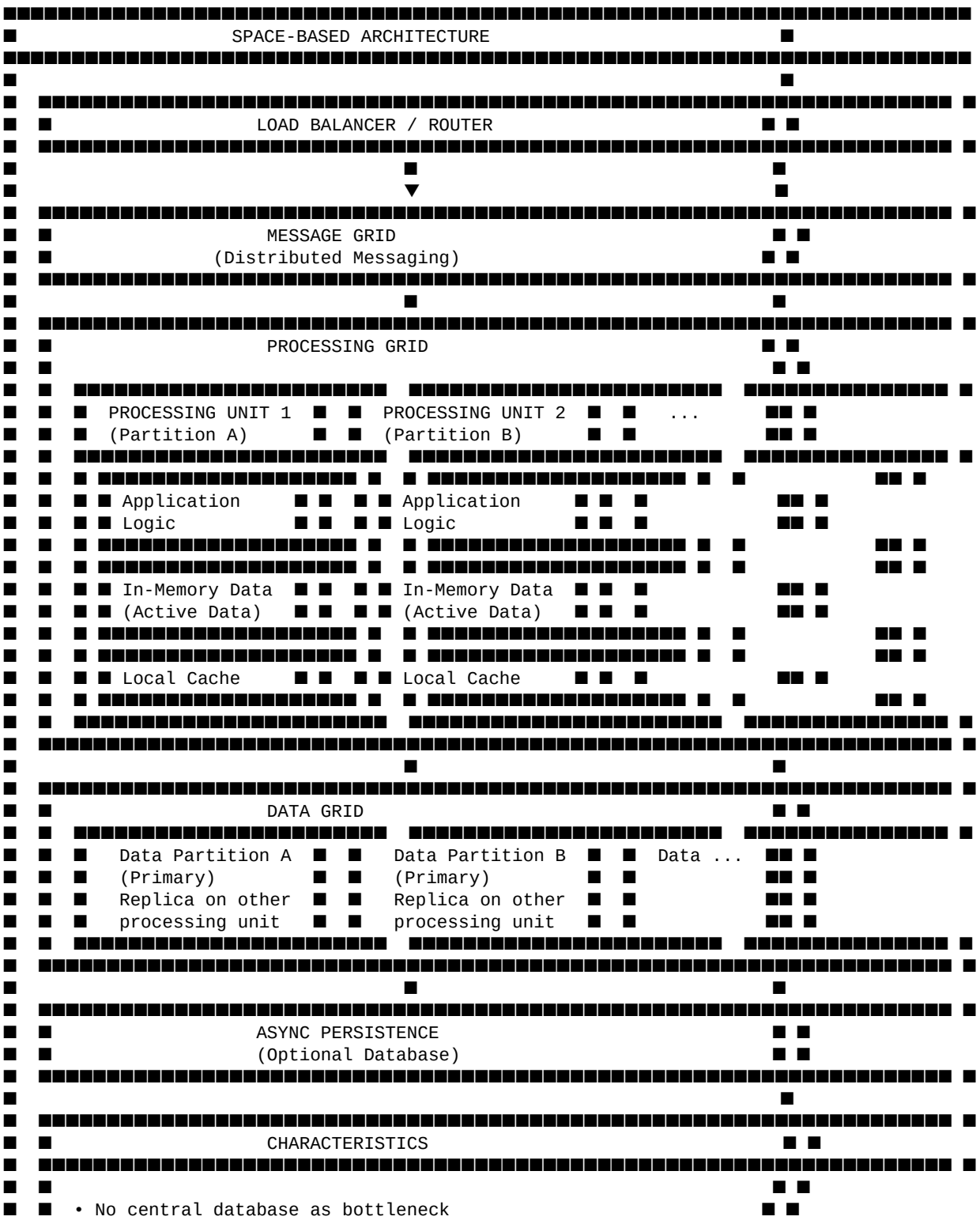
- **Virtual Middleware:** Provides messaging, grid, and data services

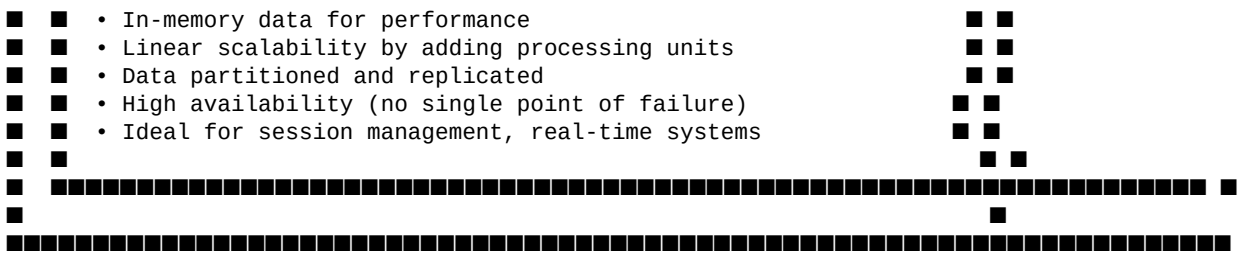
- **Data Grid:** Distributed in-memory cache

- **Partitioning:** Data divided across processing units

- **Replication:** Each data chunk replicated for availability

Diagram: Space-Based Architecture





When to Use Space-Based Architecture

Suitable Situations:

1. For high-volume transaction processing (e.g., financial trading).

2. For session management in large-scale web applications.

3. For real-time data processing.

4. When you need linear scalability.

5. For systems with high write loads that would overwhelm databases.

Real-World Use Cases:

- **Financial Trading:** Real-time trade processing
- **Gaming Platforms:** Leaderboards, player state
- **E-commerce:** Shopping cart management during sales
- **Telecom:** Call processing
- **Large-Scale Web Apps:** Session management

Advantages of Space-Based Architecture

1. **Linear Scalability:** Add processing units to scale.

2. **No Database Bottleneck:** Data in memory.

3. **High Performance:** In-memory data access.

4. **High Availability:** Replication prevents data loss.

5. **Elastic:** Scale up and down with load.

Disadvantages of Space-Based Architecture

1. **Complexity:** Much more complex than traditional.

2. **Data Consistency:** Eventual consistency model.

3. **Memory Costs:** Keeping data in memory is expensive.

4. **Learning Curve:** Requires understanding of distributed grids.

5. **Persistence:** Need async persistence for durability.

Space-Based vs Traditional

Aspect	Traditional	Space-Based
---	---	---
Data storage	Central database	In-memory grid
Scaling	Hard (DB bottleneck)	Linear (add nodes)
Performance	Database I/O	Memory access
Consistency	ACID	Eventual
Complexity	Lower	Higher

[Page 15]

PART FOUR: EVENT-DRIVEN PATTERNS

Chapter 13: Event-Driven Architecture (EDA)

Detailed Explanation

Event-Driven Architecture is a design pattern where system components communicate by producing and consuming events. Events represent significant state changes, and services react to events asynchronously.

Understanding the Problem

Traditional request-response architectures create tight coupling:

- Caller must know about callee
- Synchronous calls block
- Changes require coordination
- Hard to add new consumers

The Solution

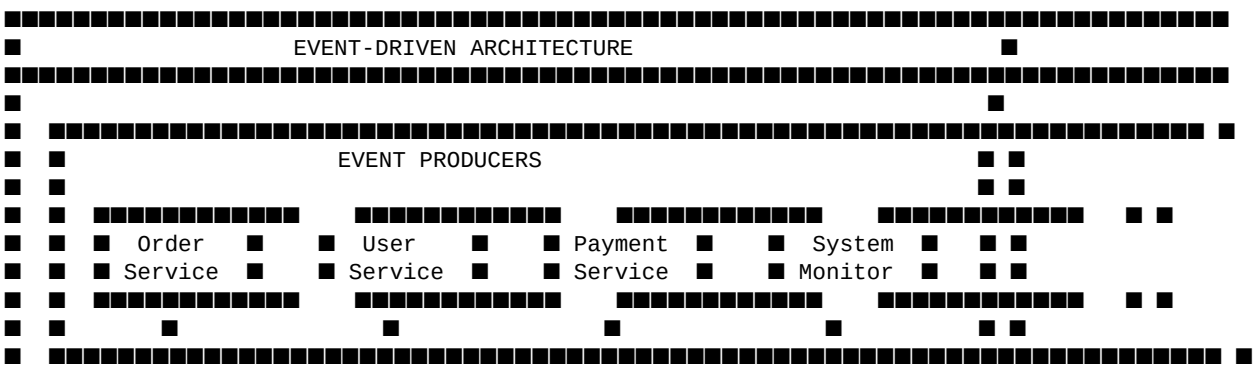
Event-Driven Architecture uses events as the primary communication mechanism:

- **Events:** Immutable records of something that happened
- **Producers:** Publish events
- **Consumers:** Subscribe to events
- **Event Bus:** Routes events to subscribers
- **Decoupled:** Producers don't know consumers

Key Concepts

- **Event:** Fact that something occurred
- **Event Bus:** Channel for event distribution
- **Publisher:** Source of events
- **Subscriber:** Reacts to events
- **Event Processing:** Synchronous or asynchronous
- **Event Store:** Persistent log of events

Diagram: Event-Driven Architecture





When to Use Event-Driven Architecture

Suitable Situations:

1. When you need loose coupling between components.

2. When you have multiple consumers interested in the same data.

3. When you need to scale components independently.

4. When you can tolerate eventual consistency.

5. For real-time or near-real-time processing.

Real-World Use Cases:

- **Order Processing:** Orchestrating multiple services
- **Notification Systems:** Email, SMS, push notifications
- **Analytics:** Collecting user behavior data
- **Audit Logging:** Recording all significant events
- **IoT Systems:** Processing sensor data streams

Advantages of Event-Driven Architecture

1. **Loose Coupling:** Publishers don't know consumers.

2. **Scalability:** Components scale independently.

3. **Flexibility:** Add new consumers without changing producers.

4. **Resilience:** Failures don't cascade.

5. **Auditability:** Event log provides complete history.

Disadvantages of Event-Driven Architecture

1. **Complexity:** Harder to understand system flow.

2. **Debugging:** Harder to trace request paths.

3. **Consistency:** Eventual consistency.

4. **Delivery Guarantees:** Need to handle at-least-once, exactly-once.

5. **Tooling:** Requires event streaming infrastructure.

EDA vs Request-Response

Aspect	Request-Response	Event-Driven
--------	------------------	----------
Coupling	Tight	Loose
Flow	Synchronous	Asynchronous
Scalability	Hard	Easy
Debugging	Easy	Hard
Consistency	Strong	Eventual

[Page 16]

Chapter 14: Event Sourcing

Detailed Explanation

Event Sourcing stores the state of an application as a sequence of events. Instead of storing current state, you store every change as an event, and the current state is derived by replaying events.

Understanding the Problem

Traditional CRUD stores only the current state. This has limitations:

- No history of how state evolved

- Audit trails require manual implementation

- Hard to debug or replay

- Temporal queries difficult

The Solution

Event Sourcing stores every state change as an event:

- **Event Log:** Append-only store of all changes

- **State Derivation:** Current state = replay all events

- **Immutability:** Events never change

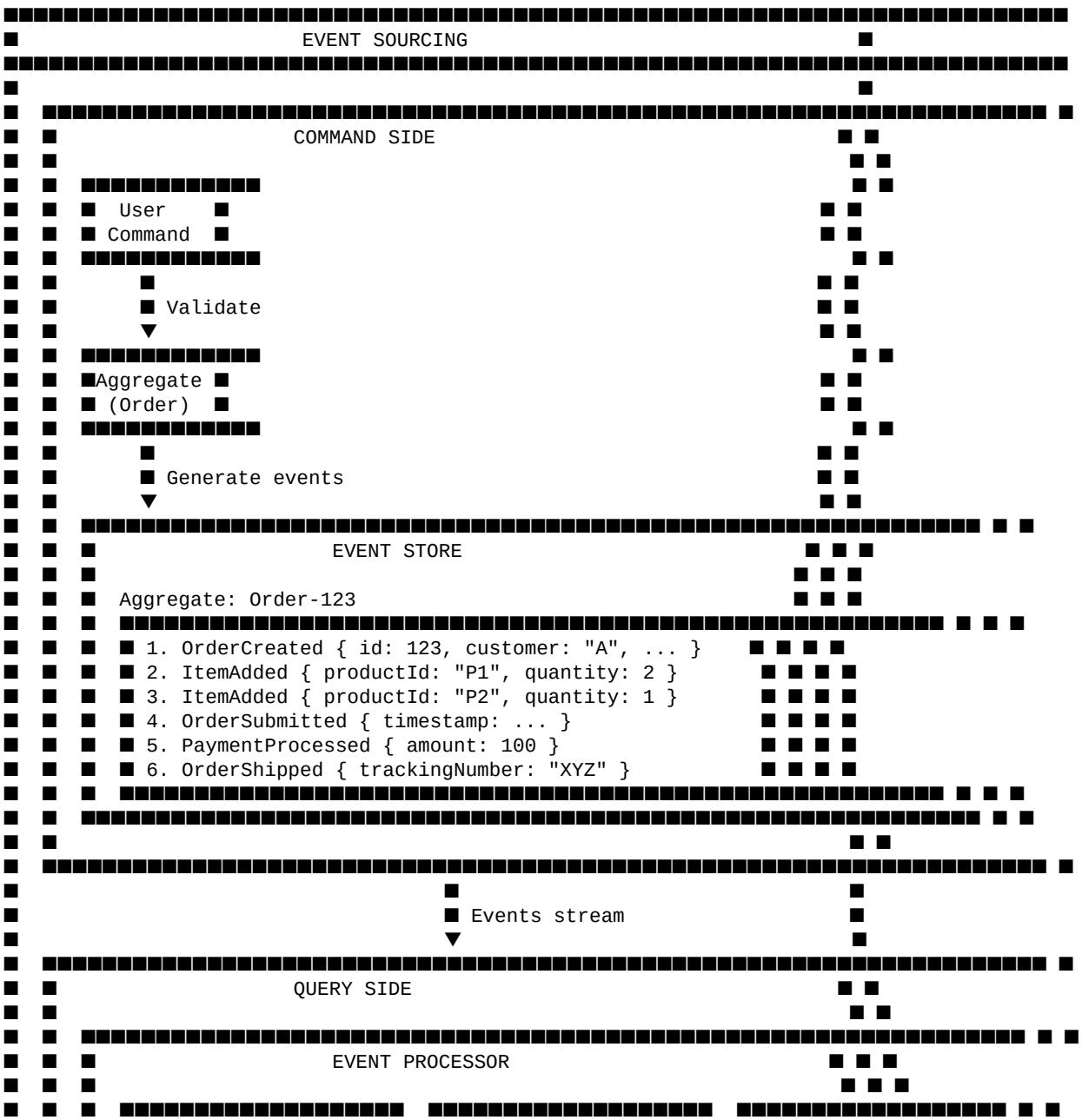
- **Audit Trail:** Complete history by default

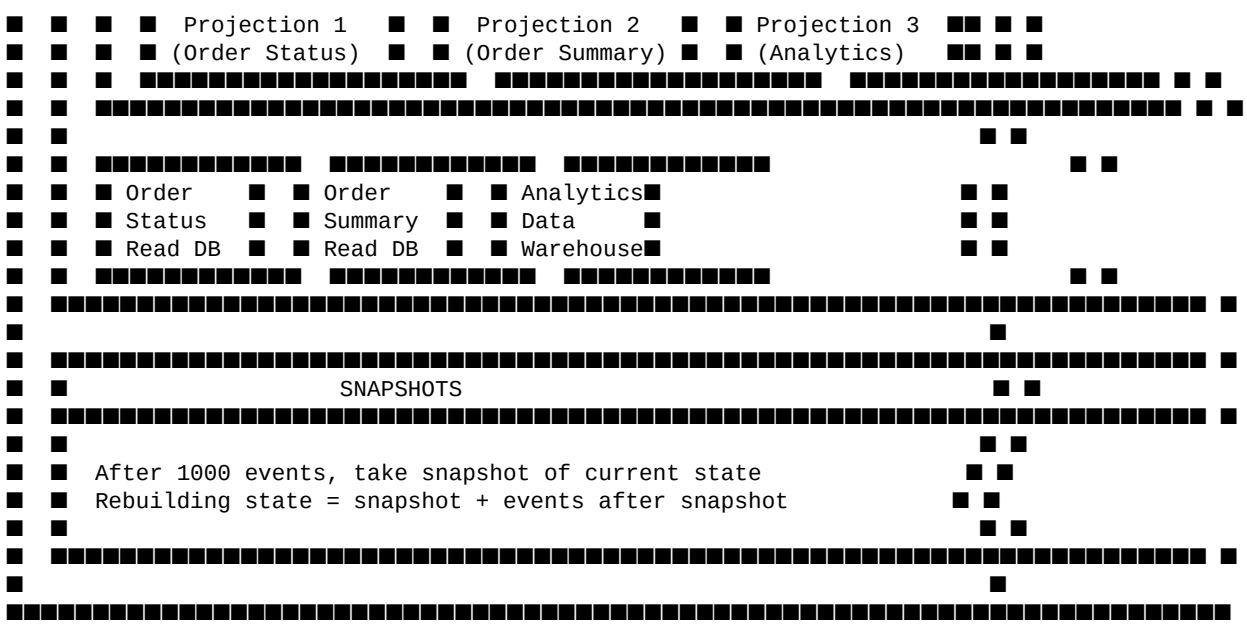
- **Replay:** Can recreate state at any point

Key Concepts

- **Event:** Immutable fact that something happened
- **Event Store:** Persistent, append-only log
- **Aggregate:** Domain entity that generates events
- **Snapshot:** Saved state to avoid replaying all events
- **Projection:** Read model built from events

Diagram: Event Sourcing





When to Use Event Sourcing

Suitable Situations:

1. When you need a complete audit trail.

2. When you need to reconstruct state at any point in time.

3. When you need to debug and replay failures.

4. When you have complex business processes with many steps.

5. When you want to support temporal queries.

Real-World Use Cases:

- **Financial Systems:** Complete transaction history
- **Order Management:** Tracking order lifecycle
- **Banking:** Account statement history
- **Collaboration Tools:** Document change history
- **Compliance Systems:** Regulatory audit requirements

Advantages of Event Sourcing

1. **Complete Audit Trail:** Every change recorded.

2. **Temporal Queries:** Can ask "what was state at time X?"

3. **Debugging:** Replay events to reproduce bugs.

4. **Event Integration:** Events naturally feed other systems.

5. **Flexible Projections:** Different read models for different needs.

Disadvantages of Event Sourcing

1. **Complexity:** More complex than CRUD.

2. **Eventual Consistency:** Projections eventually consistent.

3. **Storage:** Event store grows continuously.

4. **Event Versioning:** Changing event structures is hard.

5. **Learning Curve:** Team must understand the pattern.

Event Sourcing vs CRUD

Aspect	CRUD	Event Sourcing
--------	------	----------
Storage	Current state	Event log
History	Lost	Complete
Audit	Manual	Automatic
Temporal queries	Hard	Easy
Complexity	Simple	Complex

[Page 17]

Chapter 15: Command Query Responsibility Segregation (CQRS)

Detailed Explanation

CQRS separates the operations that read data (queries) from the operations that write data (commands). Different models optimize for each operation.

Understanding the Problem

Traditional CRUD uses the same model for reads and writes, causing:

- Complex domain logic mixed with query concerns
- Performance conflicts (reads vs writes)
- Scalability challenges (can't scale reads independently)
- Security issues (same permissions for read/write)

The Solution

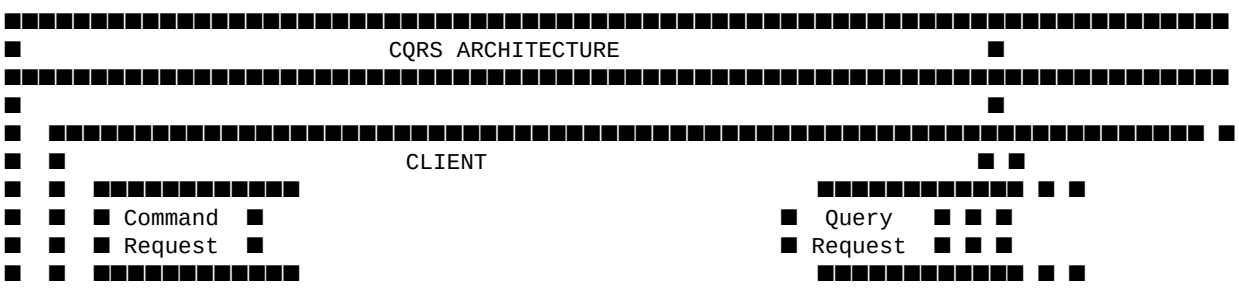
CQRS separates:

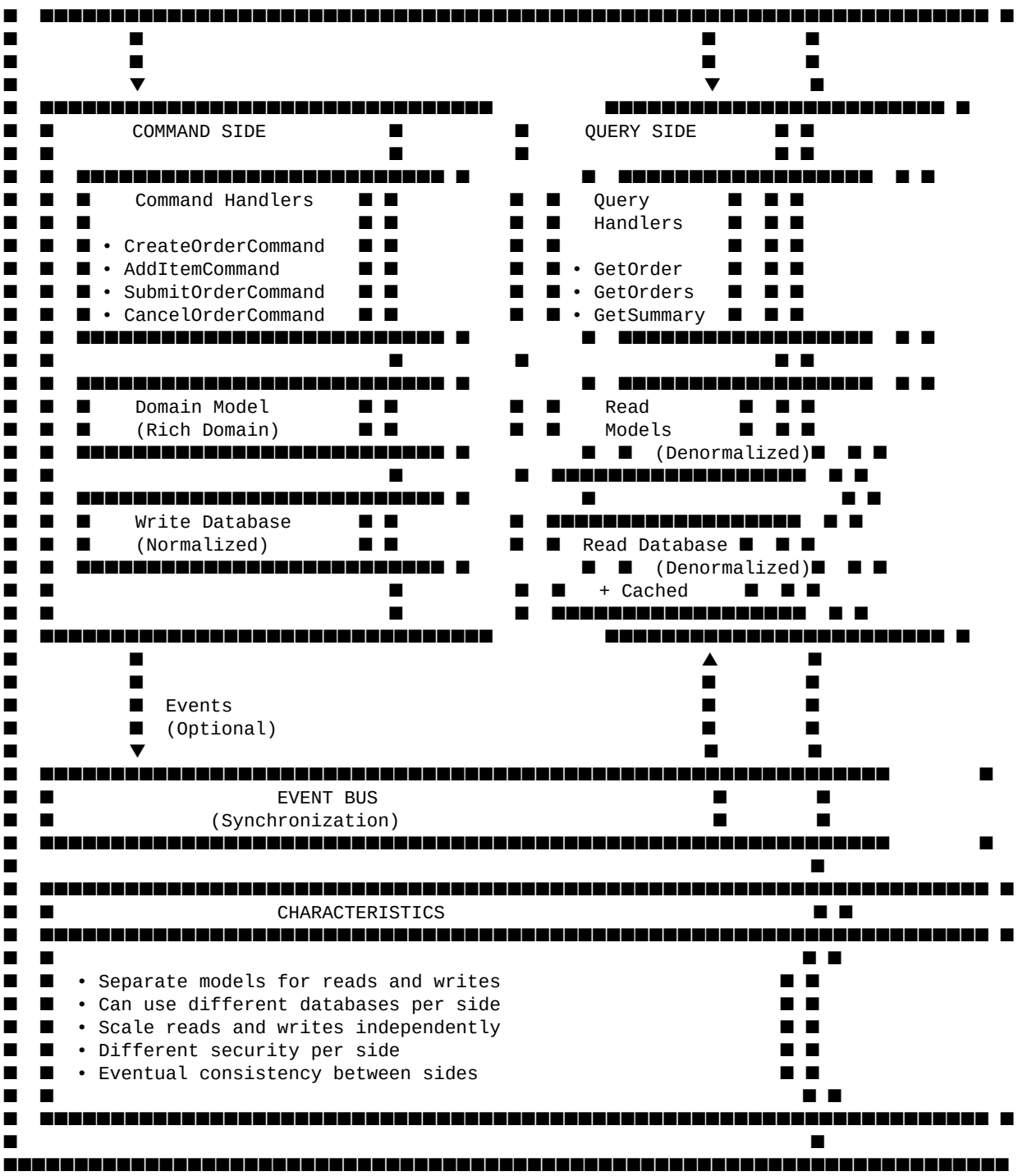
- **Command Side:** Handles updates (create, update, delete)
- **Query Side:** Handles reads (queries)
- **Models:** Different models for each side
- **Data Stores:** Can use separate databases
- **Eventual Consistency:** Reads may lag behind writes

Key Concepts

- **Command:** Intention to change state (imperative)
- **Query:** Request for data (no side effects)
- **Command Model:** Optimized for writes, business logic
- **Query Model:** Optimized for reads, denormalized
- **Eventual Consistency:** Query side eventually consistent

Diagram: CQRS Architecture





When to Use CQRS

Suitable Situations:

1. When read and write workloads have different performance requirements.

2. When you have complex domain logic on the write side.

3. When you need different security for reads and writes.

4. When you want to scale reads and writes independently.

5. When you need multiple read models for different contexts.

Real-World Use Cases:

- **E-commerce:** Product catalog (many reads, few writes)
- **Banking:** Account transactions (complex write logic)
- **Collaboration Tools:** Document editing (collaborative writes)
- **Analytics:** Read-only reports from event streams
- **High-Traffic APIs:** Read-heavy with caching

Advantages of CQRS

1. **Independent Scaling:** Scale reads and writes separately.

2. **Optimized Models:** Best model for each operation.

3. **Performance:** Read queries can be highly optimized.

4. **Security:** Different permissions per side.

5. **Flexibility:** Can use different technologies per side.

Disadvantages of CQRS

1. **Complexity:** More complex than CRUD.

2. **Consistency:** Eventual consistency between sides.

3. **Development Overhead:** Need to maintain two models.

4. **Synchronization:** Need to keep sides in sync.

5. **Learning Curve:** Team must understand the pattern.

CQRS with Event Sourcing

CQRS and Event Sourcing pair naturally:

- Command side: Uses event sourcing
- Query side: Builds projections from events
- Events: Natural synchronization mechanism

[Page 18]

Chapter 16: Publish-Subscribe Architecture

Detailed Explanation

Publish-Subscribe is a messaging pattern where publishers send messages to topics, and subscribers receive messages they've subscribed to. It decouples senders from receivers.

Understanding the Problem

Direct communication creates coupling:

- Sender must know receiver
- Adding new receivers requires code changes
- Hard to broadcast to multiple receivers

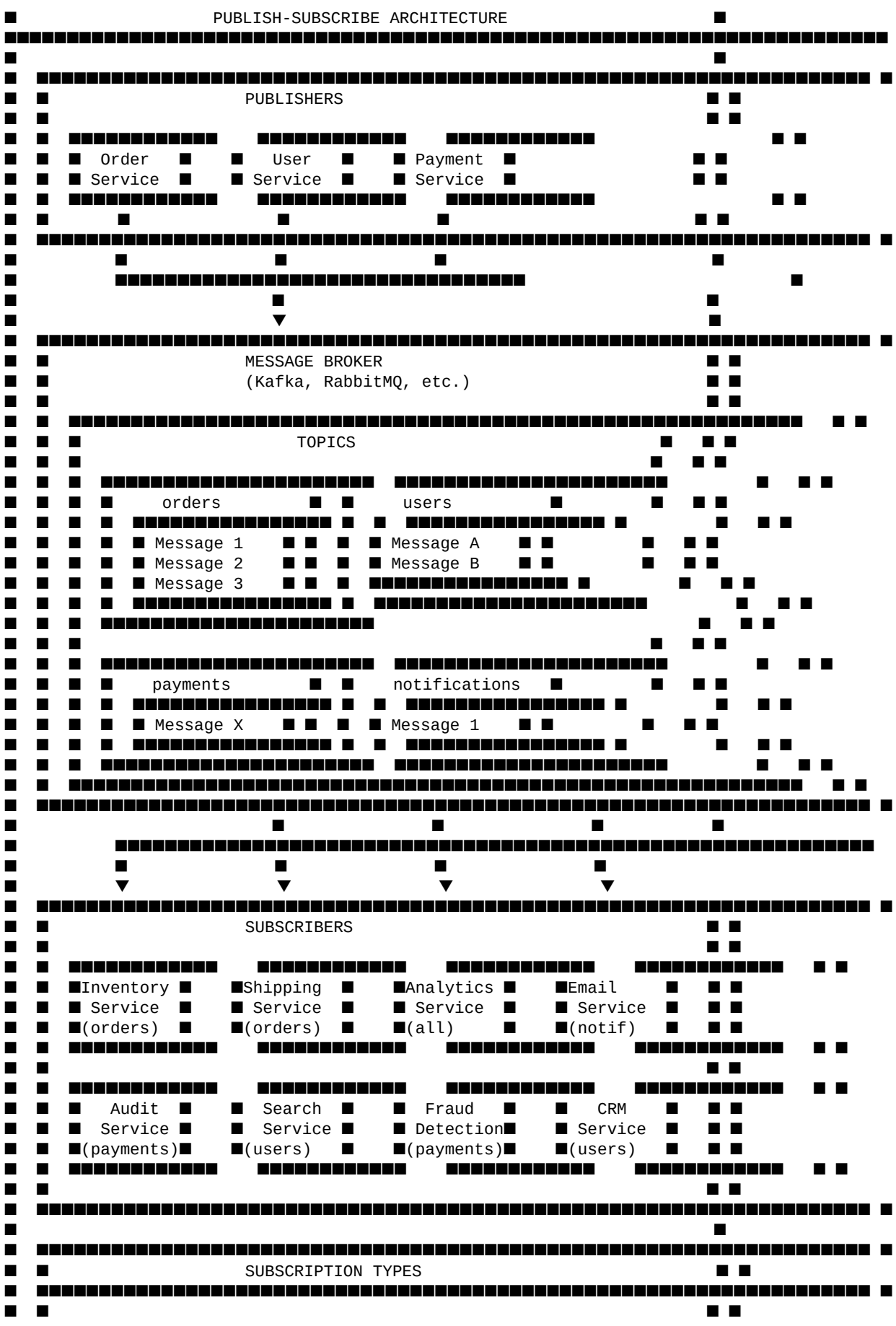
The Solution

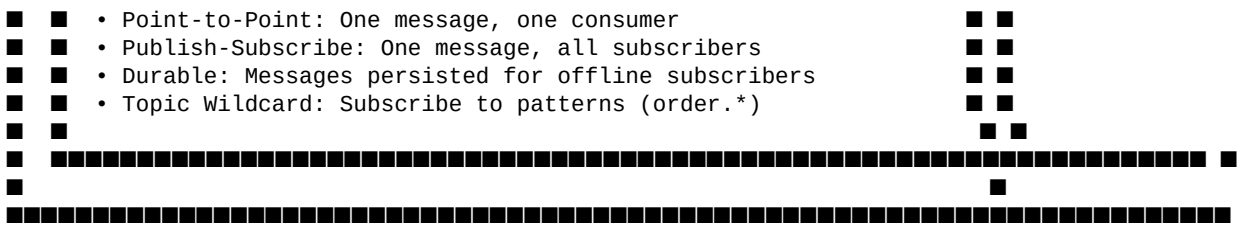
Publish-Subscribe uses:

- **Topics:** Named channels for messages
- **Publishers:** Send messages to topics
- **Subscribers:** Express interest in topics
- **Broker:** Routes messages to all subscribers
- **Decoupling:** Publishers don't know subscribers

Diagram: Publish-Subscribe Architecture




PUBLISH-SUBSCRIBE ARCHITECTURE
PUBLISHERS
Order Service
User Service
Payment Service
MESSAGE BROKER
(Kafka, RabbitMQ, etc.)
TOPICS
orders
Message 1
Message 2
Message 3
users
Message A
Message B
payments
Message X
notifications
Message 1
SUBSCRIBERS
Inventory Service (orders)
Shipping Service (orders)
Analytics Service (all)
Email Service (notif)
Audit Service (payments)
Search Service (users)
Fraud Detection (payments)
CRM Service (users)
SUBSCRIPTION TYPES



When to Use Publish-Subscribe

Suitable Situations:

1. When multiple consumers need the same message.

2. When you want loose coupling between components.

3. When you need to broadcast notifications.

4. When you want to add consumers without changing publishers.

5. For event-driven architectures.

Real-World Use Cases:

- **Microservices:** Asynchronous communication
- **Notification Systems:** Broadcast to multiple channels
- **Real-time Updates:** Live scores, stock tickers
- **Data Integration:** Sync across systems
- **IoT:** Sensor data distribution

Advantages of Publish-Subscribe

1. **Loose Coupling:** Publishers don't know subscribers.

2. **Scalability:** Add subscribers without affecting publishers.

3. **Flexibility:** Easy to add new consumers.

4. **Asynchronous:** No blocking.

5. **Broadcast:** One message, many consumers.

Disadvantages of Publish-Subscribe

1. **Complexity:** Extra infrastructure.

2. **Delivery Guarantees:** Harder to ensure exactly-once.

3. **Ordering:** Messages may arrive out of order.

4. **Debugging:** Hard to trace message flow.

5. **Topic Explosion:** Too many topics can be hard to manage.

Pub-Sub vs Message Queue

Aspect	Publish-Subscribe	Message Queue
--------	-------------------	---------------
Pattern	One-to-many	One-to-one
Consumers	All subscribers	One consumer
Use case	Broadcast	Work distribution
Decoupling	High	Medium

[Page 19]

PART FIVE: MESSAGE-BASED PATTERNS

Chapter 17: Message Bus Architecture

Detailed Explanation

Message Bus Architecture uses a central message bus as the communication backbone for integrating multiple systems. It enables decoupled communication between services through a common infrastructure.

Understanding the Problem

As systems grow, point-to-point connections become unmanageable:

- N services require N² connections
- Changes require updating all connected systems
- Hard to add new systems
- No central monitoring or control

The Solution

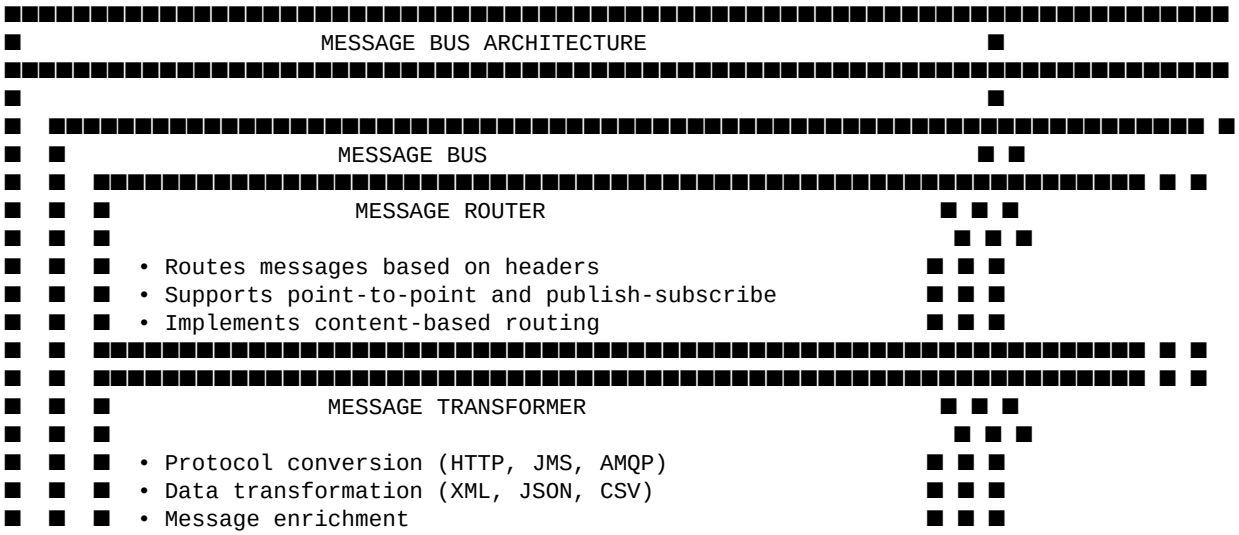
Message Bus provides:

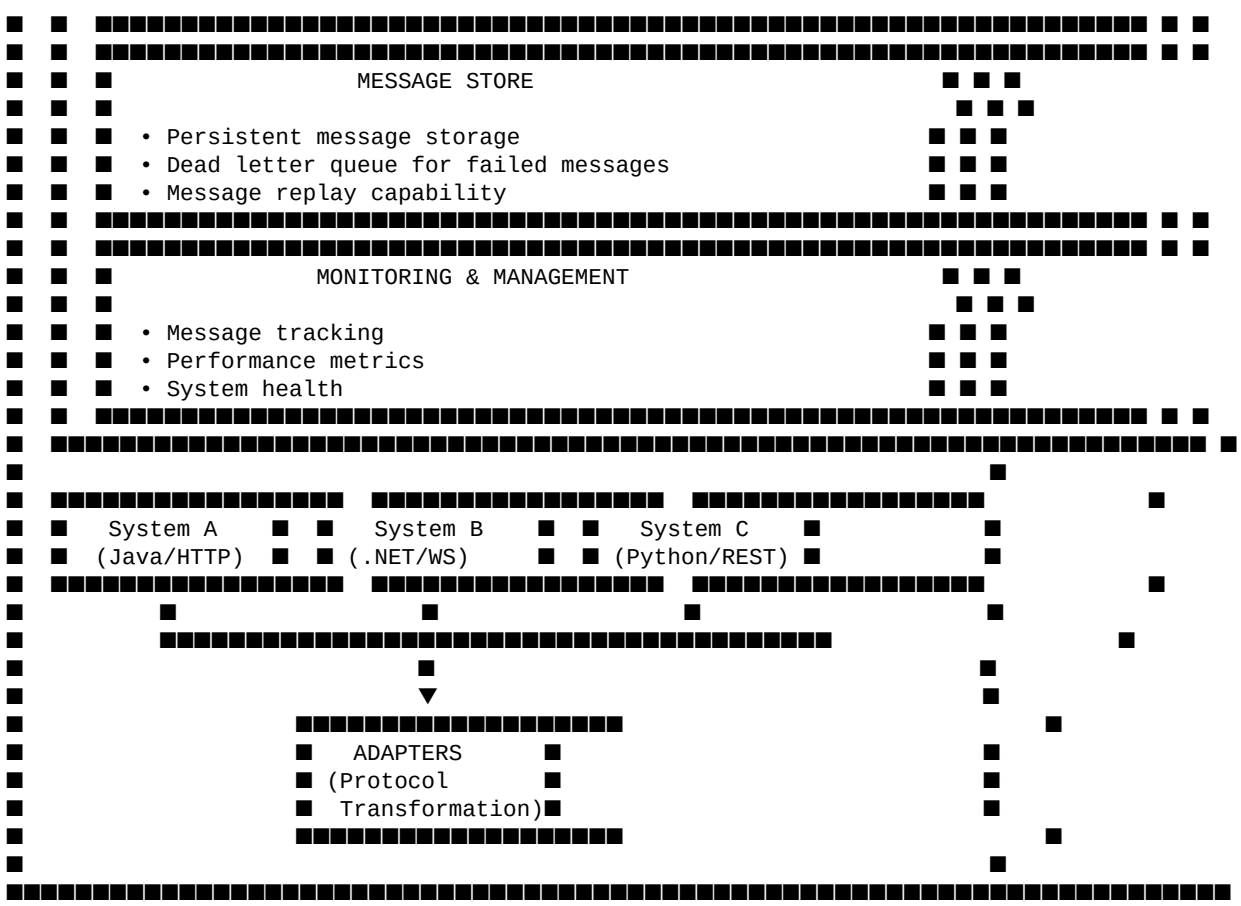
- **Central Hub:** All communication through the bus
- **Protocol Translation:** Convert between formats
- **Routing:** Direct messages to destinations
- **Mediation:** Transform and enrich messages
- **Monitoring:** Central observability

Key Concepts

- **Message Bus:** Central communication infrastructure
- **Channel:** Logical path for messages
- **Message:** Data packet with headers and payload
- **Endpoint:** System connected to the bus
- **Mediator:** Transforms and routes messages

Diagram: Message Bus Architecture





When to Use Message Bus Architecture

Suitable Situations:

1. When integrating multiple heterogeneous systems.

2. When you need protocol translation.

3. When you have complex routing requirements.

4. When you need central monitoring and management.

5. When you have legacy systems to integrate.

Real-World Use Cases:

- **Enterprise Application Integration:** Connecting ERP, CRM, SCM
- **Financial Services:** Trading systems integration

- **Healthcare:** HL7 message routing
- **Telecom:** Billing and provisioning systems
- **Retail:** Point-of-sale to backend integration

Advantages of Message Bus Architecture

1. **Centralized Control:** One place to manage integration.

2. **Protocol Mediation:** Connect different technologies.

3. **Routing Flexibility:** Content-based, header-based routing.

4. **Monitoring:** Central observability.

5. **Reusability:** Shared infrastructure for all integrations.

Disadvantages of Message Bus Architecture

1. **Single Point of Failure:** Bus becomes critical.

2. **Performance Bottleneck:** All messages through bus.

3. **Complexity:** Bus itself is complex to manage.

4. **Cost:** Requires dedicated infrastructure.

5. **Coupling:** Systems still coupled to bus.

Message Bus vs ESB

Message bus is often part of an ESB, but:
- **Message Bus:** Focuses on message transport
- **ESB:** Adds orchestration, business rules, service management

---

[Page 20]

Chapter 18: Message Queue Architecture

Detailed Explanation

Message Queue Architecture uses queues to decouple producers and consumers, providing reliable, asynchronous communication between components.

Understanding the Problem

Direct synchronous communication has issues:

- Producer waits for consumer
- If consumer is down, request fails
- No buffering for load spikes
- Tight coupling in time

The Solution

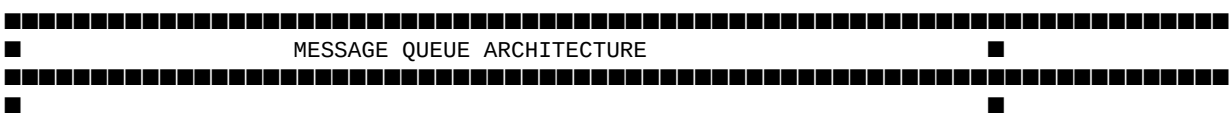
Message Queues provide:

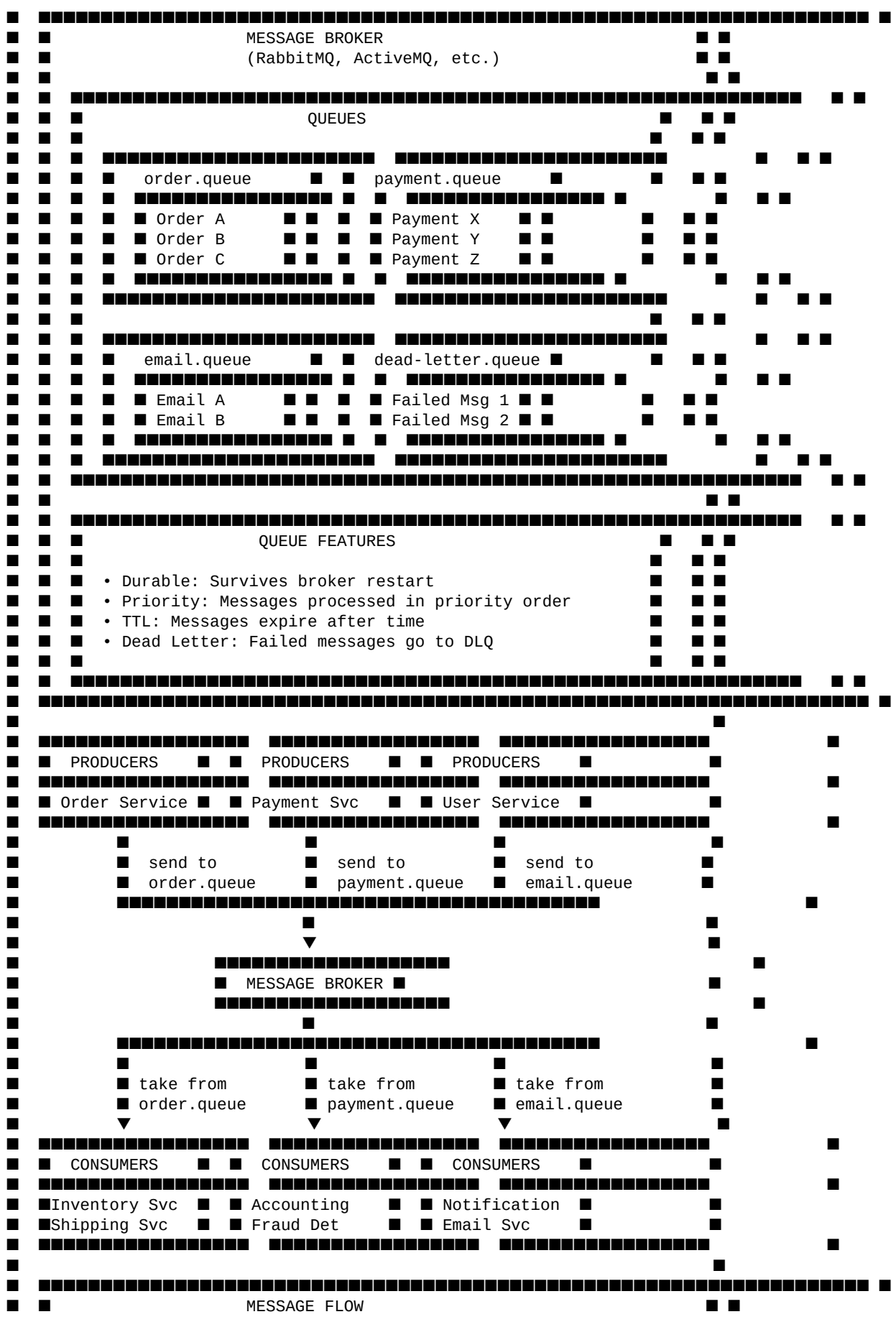
- **Queue:** Buffer between producer and consumer
- **Producer:** Sends messages to queue
- **Consumer:** Takes messages from queue
- **Persistence:** Messages stored until consumed
- **Acknowledgment:** Consumer confirms processing

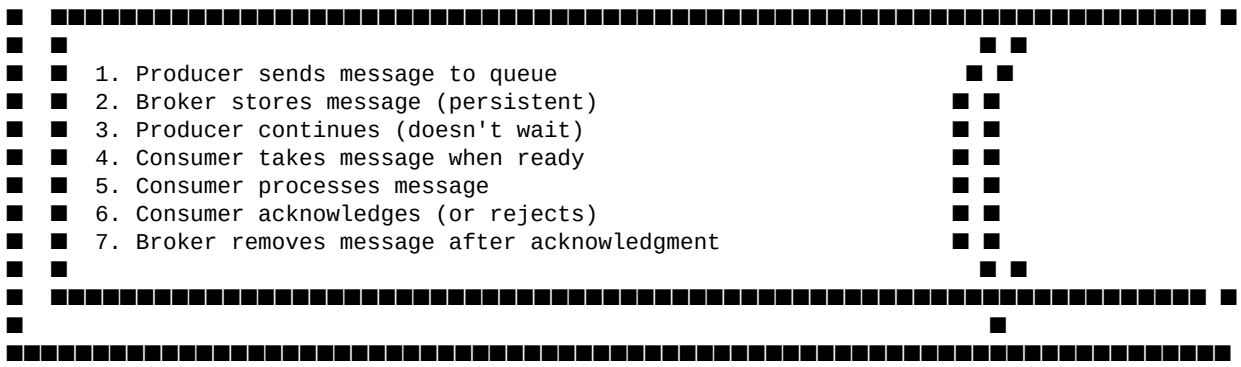
Key Concepts

- **Queue:** FIFO buffer for messages
- **Producer:** Sends messages to queue
- **Consumer:** Receives messages from queue
- **Broker:** Manages queues and delivery
- **Acknowledgment:** Consumer confirms receipt

Diagram: Message Queue Architecture




MESSAGE BROKER
(RabbitMQ, ActiveMQ, etc.)
QUEUES
order.queue
Order A
Order B
Order C
payment.queue
Payment X
Payment Y
Payment Z
email.queue
Email A
Email B
dead-letter.queue
Failed Msg 1
Failed Msg 2
QUEUE FEATURES
• Durable: Survives broker restart
• Priority: Messages processed in priority order
• TTL: Messages expire after time
• Dead Letter: Failed messages go to DLQ
PRODUCERS
Order Service
PRODUCERS
Payment Svc
PRODUCERS
User Service
send to order.queue
send to payment.queue
send to email.queue
MESSAGE BROKER
take from order.queue
take from payment.queue
take from email.queue
CONSUMERS
Inventory Svc
Shipping Svc
CONSUMERS
Accounting
Fraud Det
CONSUMERS
Notification
Email Svc
MESSAGE FLOW



When to Use Message Queue Architecture

Suitable Situations:

1. When you need to decouple producers and consumers.

2. When you have variable load (queue handles spikes).

3. When consumers can be offline (messages stored).

4. When you need reliable delivery.

5. For work distribution across multiple workers.

Real-World Use Cases:

- **Order Processing:** Queue orders for fulfillment
- **Email Delivery:** Queue emails for sending
- **Background Jobs:** Asynchronous task processing
- **Load Leveling:** Smooth traffic spikes
- **Microservices:** Async communication

Advantages of Message Queue Architecture

1. **Decoupling:** Producers and consumers independent.

2. **Buffering:** Handles load spikes.

3. **Reliability:** Messages persisted until processed.

4. **Load Balancing:** Multiple consumers share queue.

5. **Resilience:** Consumers can fail and restart.

Disadvantages of Message Queue Architecture

1. **Complexity:** Additional infrastructure.

2. **Latency:** Not suitable for real-time.

3. **Ordering:** FIFO only within single queue.

4. **Management:** Need to monitor queues.

5. **Once-Only Delivery:** Exactly-once is hard.

Queue vs Topic

Aspect	Queue	Topic
---	---	---
Pattern	Point-to-point	Publish-subscribe
Consumers	One per message	All subscribers
Use case	Work distribution	Broadcast

[Page 21]

Chapter 19: Enterprise Service Bus (ESB)

Detailed Explanation

Enterprise Service Bus (ESB) is a middleware solution that provides integration capabilities for connecting enterprise applications. It acts as a central nervous system for service-oriented architectures.

Understanding the Problem

Enterprises have many disparate systems:

- ERP, CRM, SCM, custom apps, legacy systems

- Different protocols, data formats, APIs

- Point-to-point integrations become unmanageable

The Solution

ESB provides:

- **Mediation:** Protocol translation, format conversion

- **Routing:** Intelligent message routing

- **Orchestration:** Process composition

- **Transformation:** Data mapping

- **Management:** Centralized governance

Key Concepts

- **Service:** Business function exposed on ESB

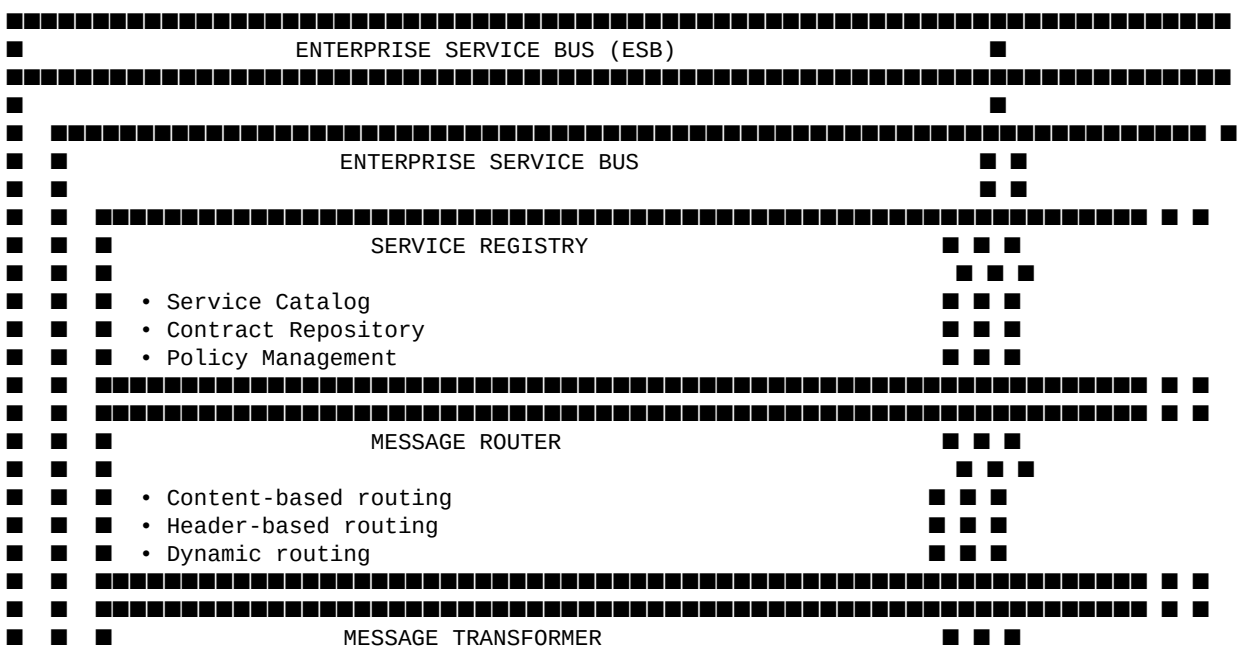
- **Endpoint:** Connection point for systems

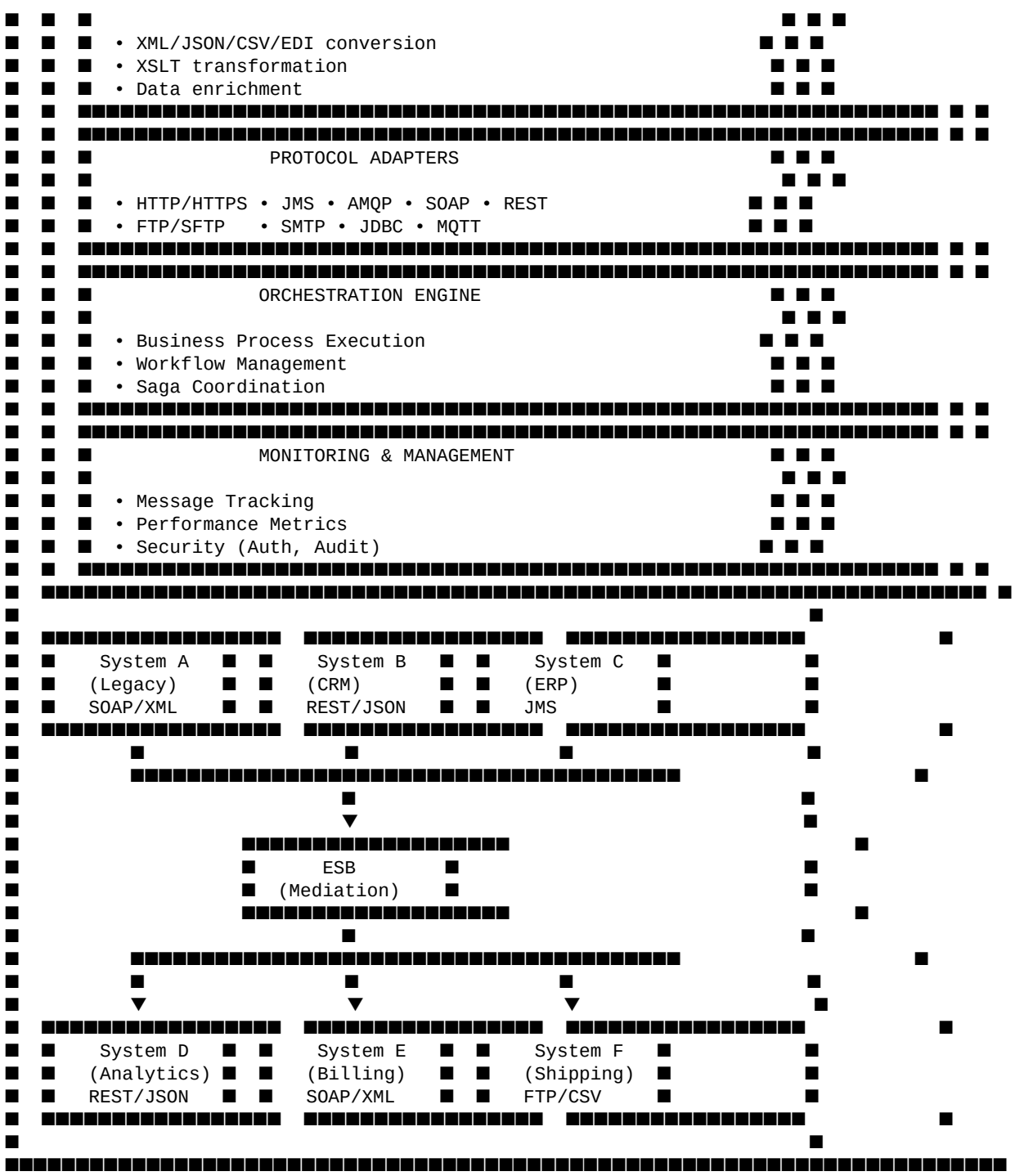
- **Message:** Data flowing through ESB

- **Mediation Flow:** Sequence of processing steps

- **Registry:** Catalog of services

Diagram: Enterprise Service Bus





When to Use ESB

Suitable Situations:

1. When you need to integrate many enterprise systems.

2. When you have diverse protocols and data formats.

3. When you need complex routing and transformation.

4. When you require centralized governance.

5. When you have legacy systems to integrate.

Real-World Use Cases:

- **Enterprise Integration:** Connecting SAP, Oracle, Salesforce
- **Banking:** Core banking system integration
- **Healthcare:** HL7 message routing
- **Government:** Citizen service portals
- **Telecom:** Billing, provisioning, CRM integration

Advantages of ESB

1. **Centralized Integration:** Single place for all integrations.

2. **Protocol Mediation:** Connect any system to any system.

3. **Governance:** Centralized control of services.

4. **Monitoring:** Complete visibility of message flow.

5. **Reusability:** Integration components reused.

Disadvantages of ESB

1. **Complexity:** ESB itself is complex.

2. **Bottleneck:** Central bus can become bottleneck.

3. **Single Point of Failure:** ESB down = everything down.

4. **Cost:** Expensive commercial products.

5. **Coupling:** Systems still coupled to ESB.

ESB vs Microservices

Aspect	ESB	Microservices
--------	-----	----------
Architecture	Centralized	Decentralized
Communication	Through bus	Direct/API gateway
Governance	Central	Decentralized
Deployment	One ESB	Many services
Scalability	Harder	Easy

[Page 22]

PART SIX: DISTRIBUTED SYSTEMS PATTERNS

Chapter 20: Peer-to-Peer Architecture

Detailed Explanation

Peer-to-Peer (P2P) architecture distributes workload equally among all nodes (peers). There is no central server; each node acts as both client and server.

Understanding the Problem

Client-server architecture has a central server that can become:

- Bottleneck
- Single point of failure
- Scaling limitation
- Costly to maintain

The Solution

P2P architecture:

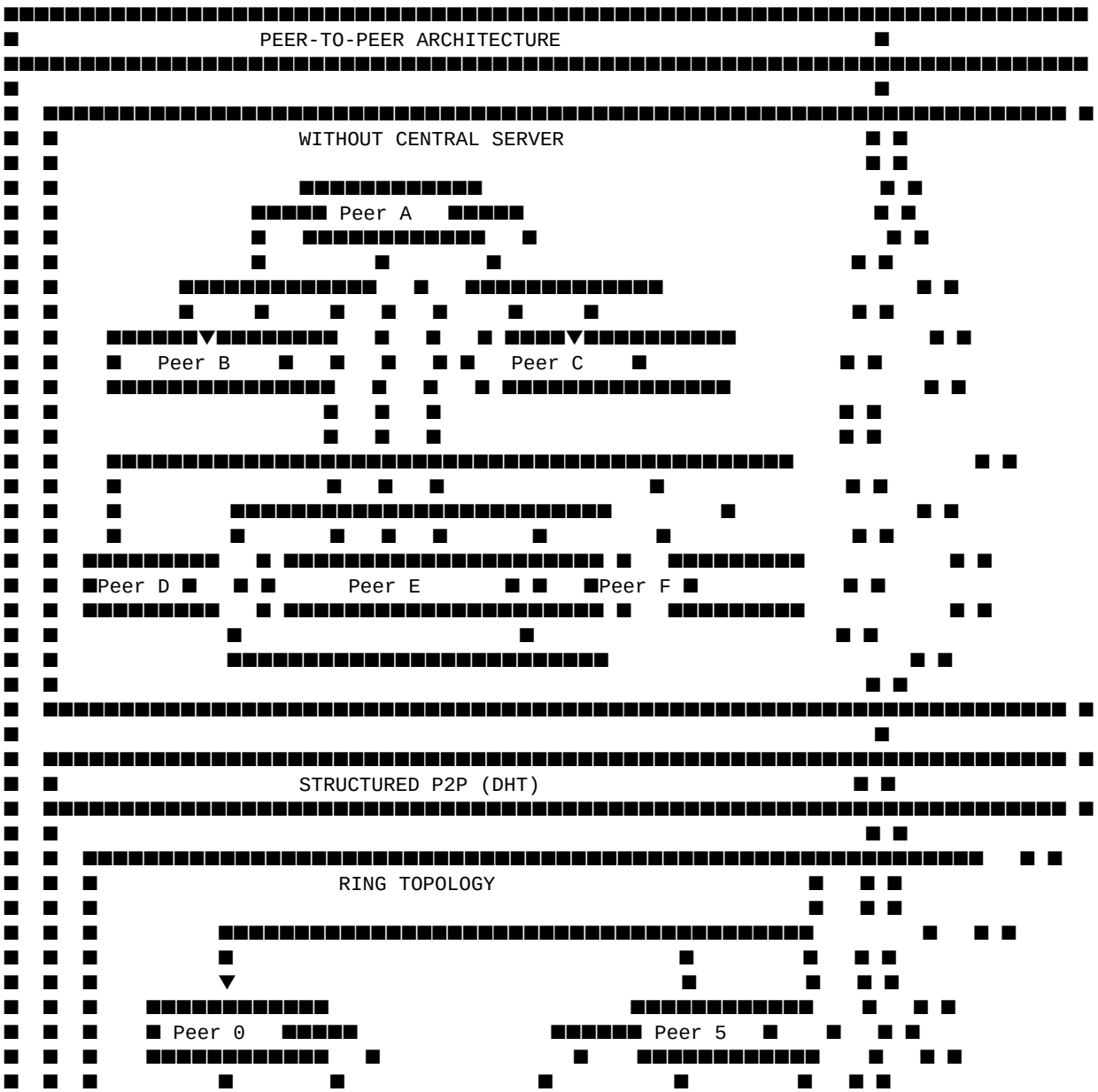
- **Decentralized:** No central server

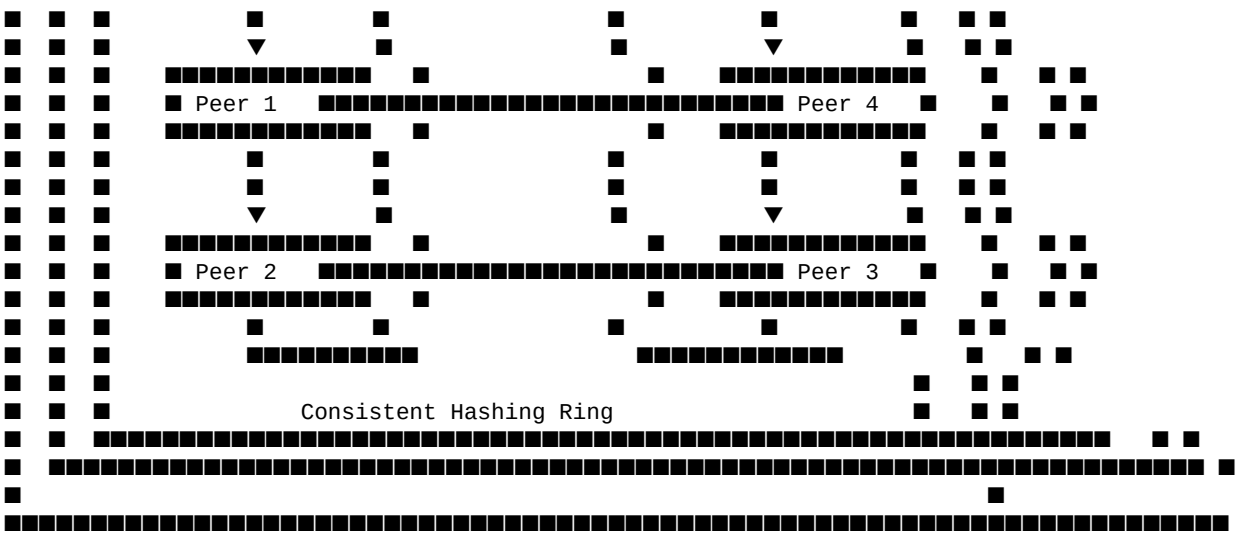
- **Equal Peers:** All nodes have same capabilities
- **Direct Communication:** Peers communicate directly
- **Self-Organizing:** Nodes join and leave dynamically

Types of P2P

1. **Unstructured:** Random network of peers
2. **Structured:** Organized network (DHT)
3. **Hybrid:** Central component for discovery

Diagram: Peer-to-Peer Architecture





When to Use Peer-to-Peer Architecture

Suitable Situations:

1. When you need massive scalability.

2. When you can tolerate eventual consistency.

3. When you want to avoid central infrastructure.

4. For file sharing, content distribution.

5. When nodes are numerous and dynamic.

Real-World Use Cases:

- **BitTorrent:** File sharing
- **Bitcoin/Cryptocurrency:** Distributed ledger
- **IPFS:** Distributed file system
- **Skype (early):** VoIP calling
- **Consensus Systems:** Blockchain

Advantages of P2P Architecture

1. **Scalability:** Scales with number of peers.

2. **No Central Point:** No single point of failure.

3. **Cost Effective:** No server infrastructure.

4. **Self-Organizing:** Nodes join/leave dynamically.

5. **Resilience:** Highly available.

Disadvantages of P2P Architecture

1. **Complexity:** Hard to implement correctly.

2. **Consistency:** Hard to maintain consistency.

3. **Security:** Harder to secure.

4. **NAT Traversal:** Communication behind firewalls.

5. **Churn:** Nodes joining/leaving cause instability.

P2P vs Client-Server

Aspect	P2P	Client-Server
--------	-----	---------------
Central point	No	Yes
Scalability	High	Limited by server
Complexity	High	Lower
Consistency	Hard	Easier
Management	Decentralized	Centralized

[Page 23]

Chapter 21: Master-Slave Architecture

Detailed Explanation

Master-Slave architecture divides nodes into a master that controls one or more slaves. The master assigns tasks, and slaves execute them.

Understanding the Problem

Distributed systems need coordination:

- Who assigns work?

- How to avoid conflicts?

- How to manage state?

The Solution

Master-Slave provides:

- **Master:** Coordinates work, assigns tasks

- **Slaves:** Execute tasks, report results

- **Single Leader:** Clear coordination point

- **Read/Write Split:** Master handles writes, slaves handle reads

Key Concepts

- **Master:** Single authoritative node

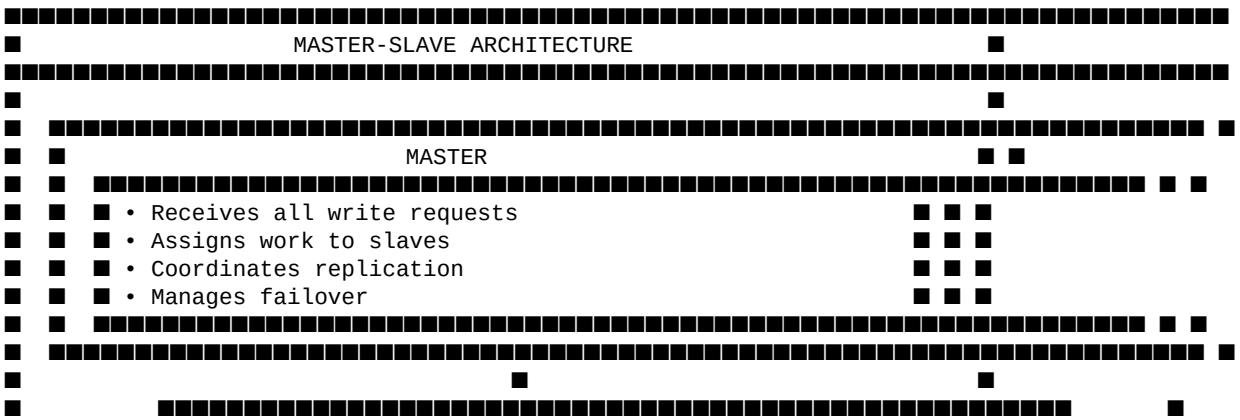
- **Slave:** Replica or worker node

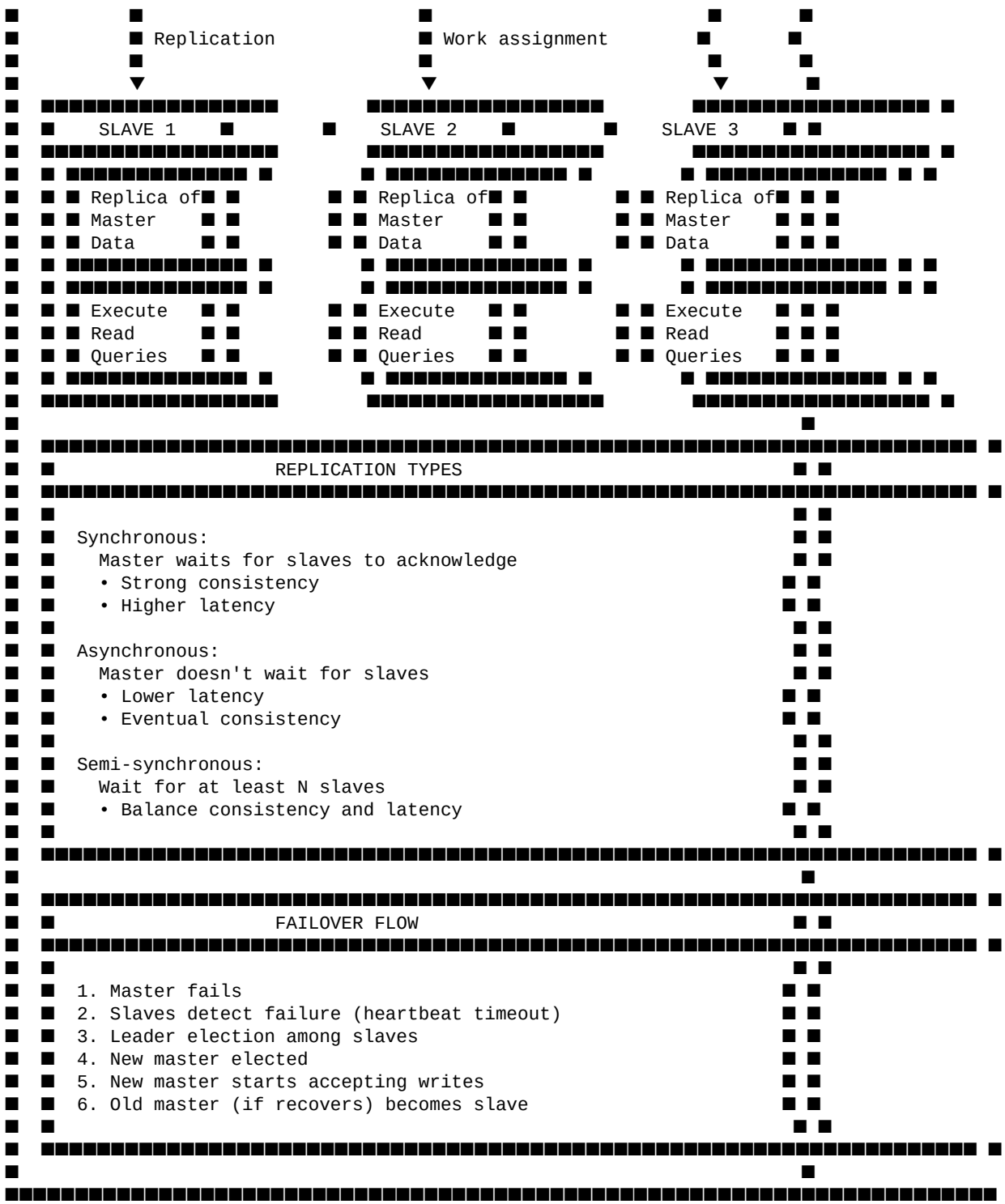
- **Leader Election:** Process to choose master

- **Failover:** Promoting slave when master fails

- **Replication:** Data copied to slaves

Diagram: Master-Slave Architecture





When to Use Master-Slave Architecture

Suitable Situations:

1. When you need a single source of truth.

2. When writes are less frequent than reads.

3. When you need read scalability.

4. When you can tolerate some read latency.

5. For database replication (read replicas).

Real-World Use Cases:

- **Database Replication:** MySQL master-slave, PostgreSQL streaming
- **MapReduce:** Job tracker (master) and task trackers (slaves)
- **Distributed File Systems:** NameNode (master) and DataNodes (slaves)
- **Queue Systems:** Leader nodes for partitions
- **Cluster Management:** Kubernetes control plane

Advantages of Master-Slave Architecture

1. **Read Scalability:** Add more slaves for reads.

2. **Simple Coordination:** Single authority.

3. **Fault Tolerance:** Slaves can take over.

4. **Backup:** Slaves serve as backup.

5. **Separation:** Writes and reads separated.

Disadvantages of Master-Slave Architecture

1. **Single Point of Failure:** Master is critical.

2. **Write Bottleneck:** All writes go to master.

3. **Replication Lag:** Slaves may be behind.

4. **Failover Complexity:** Promoting slave is complex.

5. **Wasted Resources:** Slaves may be underutilized.

Master-Slave vs Multi-Master

Aspect	Master-Slave	Multi-Master
--------	----------	----------
Writes	Single node	Multiple nodes
Consistency	Stronger	Conflict resolution needed
Write scalability	Limited	Better
Complexity	Lower	Higher

[Page 24]

Chapter 22: Shared-Nothing Architecture

Detailed Explanation

Shared-Nothing Architecture is a distributed computing architecture where each node is independent and self-sufficient. Nodes do not share memory or disk storage.

Understanding the Problem

Shared resources create contention:

- Shared memory requires synchronization
- Shared disk becomes bottleneck
- Adding nodes doesn't scale linearly

The Solution

Shared-Nothing ensures:

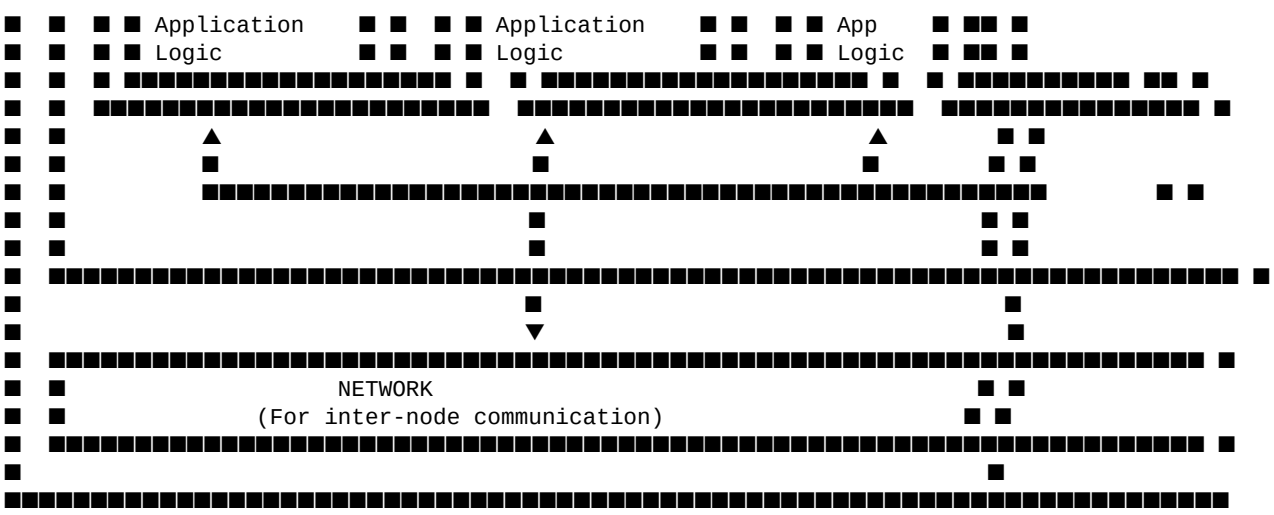
- **No Shared Memory:** Each node has own memory
- **No Shared Disk:** Each node has own storage
- **Independent:** Nodes can fail without affecting others
- **Linear Scalability:** Adding nodes adds capacity

Key Concepts

- **Partitioning:** Data divided across nodes
- **Independence:** Each node handles its own data
- **No Contention:** No locks across nodes
- **Failure Isolation:** Node failures isolated
- **Horizontal Scaling:** Add nodes to scale

Diagram: Shared-Nothing Architecture





When to Use Shared-Nothing Architecture

Suitable Situations:

1. When you need linear scalability.

2. When you can partition data.

3. When you need high availability.

4. For large-scale data processing.

5. When you want to avoid contention.

Real-World Use Cases:

- **Distributed Databases:** Cassandra, MongoDB, HBase
- **Data Warehouses:** Snowflake, Teradata
- **MapReduce:** Hadoop
- **Web Caches:** Memcached (shared-nothing design)
- **Microservices:** Each service has its own data

Advantages of Shared-Nothing Architecture

1. **Linear Scalability:** Adding nodes increases capacity linearly.

2. **No Contention:** No shared resources to compete for.

3. **Failure Isolation:** Node failures don't affect others.

4. **Simple Concurrency:** No complex locking across nodes.

5. **Cost Effective:** Use commodity hardware.

Disadvantages of Shared-Nothing Architecture

1. **Data Distribution:** Complex partitioning logic.

2. **Cross-Node Queries:** Hard to query across nodes.

3. **Data Skew:** Uneven distribution of data.

4. **Rebalancing:** Moving data when adding nodes is complex.

5. **Consistency:** Hard to maintain strong consistency.

Shared-Nothing vs Shared-Disk

Aspect	Shared-Nothing	Shared-Disk
--------	----------	----------
Scalability	Linear	Limited
Contention	None	High
Complexity	Higher	Lower
Consistency	Harder	Easier

[Page 25]

Chapter 23: Sharding Pattern

Detailed Explanation

Sharding is a database architecture pattern where data is horizontally partitioned across multiple databases. Each shard is an independent database containing a subset of the

data.

Understanding the Problem

A single database has limits:

- Storage capacity
- Write throughput
- Query performance
- Connection limits

The Solution

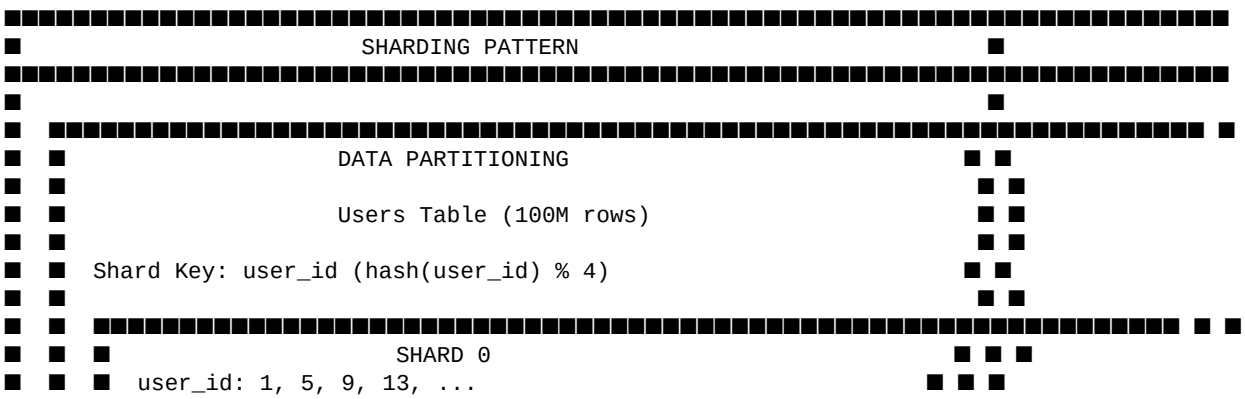
Sharding splits data across multiple databases:

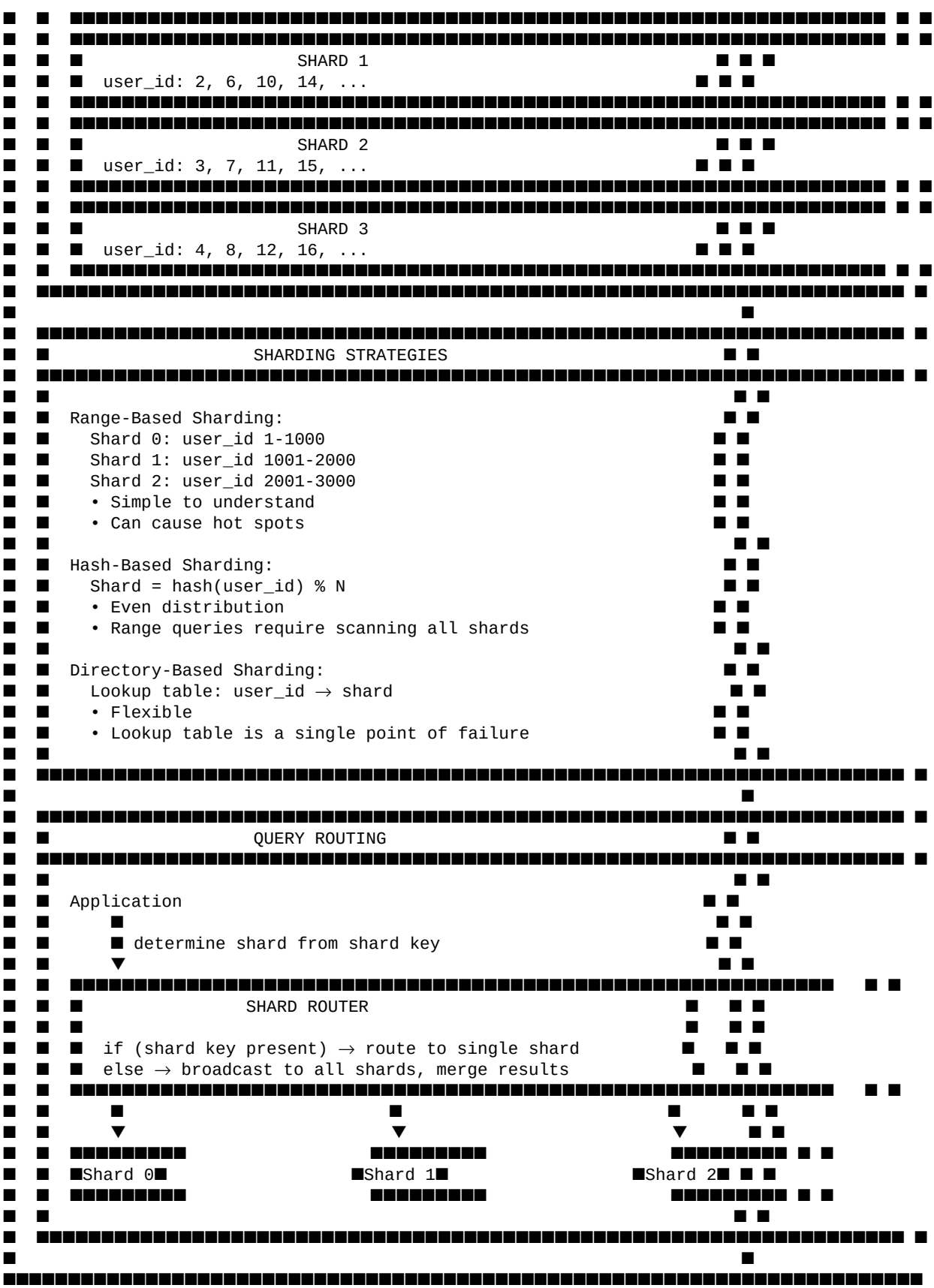
- **Shard Key:** Determines which shard data goes to
- **Horizontal Partitioning:** Same schema across shards
- **Independent:** Each shard can run on separate server
- **Scalable:** Add shards to increase capacity

Key Concepts

- **Shard Key:** Field used to distribute data
- **Shard:** Independent database instance
- **Range Partitioning:** Shard based on value ranges
- **Hash Partitioning:** Shard based on hash of key
- **Directory-Based:** Lookup table for shard location

Diagram: Sharding Pattern





When to Use Sharding Pattern

Suitable Situations:

1. When data is too large for a single database.

2. When write throughput exceeds single database capacity.

3. When you need linear scalability.

4. When data can be partitioned by a natural key.

5. For multi-tenant applications (shard per tenant).

Real-World Use Cases:

- **Large-Scale SaaS:** Shard per customer
- **Social Media:** User data partitioned by user ID
- **E-commerce:** Order data by region or time
- **Gaming:** Player data by region
- **Analytics:** Time-series data by date

Advantages of Sharding Pattern

1. **Scalability:** Add shards to scale.

2. **Performance:** Each shard handles less load.

3. **Availability:** Shard failures affect only subset.

4. **Parallelism:** Queries can run in parallel.

5. **Cost:** Use smaller, cheaper hardware.

Disadvantages of Sharding Pattern

1. **Complexity:** Application must handle sharding.

2. **Cross-Shard Queries:** Difficult and slow.

3. **Rebalancing:** Adding shards requires moving data.

4. **Shard Key Selection:** Hard to choose good key.

5. **Transactions:** No ACID across shards.

Sharding vs Partitioning

Aspect	Sharding	Partitioning
--------	----------	----------
Scope	Across servers	Within server
Goal	Scalability	Manageability
Complexity	High	Low
Query complexity	High	Low

[Page 26]

PART SEVEN: MODERN ARCHITECTURE PATTERNS

Chapter 24: Serverless Architecture

Detailed Explanation

Serverless Architecture (Function-as-a-Service) allows developers to run code without managing servers. The cloud provider handles infrastructure, scaling, and billing.

Understanding the Problem

Traditional servers require:

- Provisioning
- Configuration
- Scaling management
- Maintenance
- Paying for idle capacity

The Solution

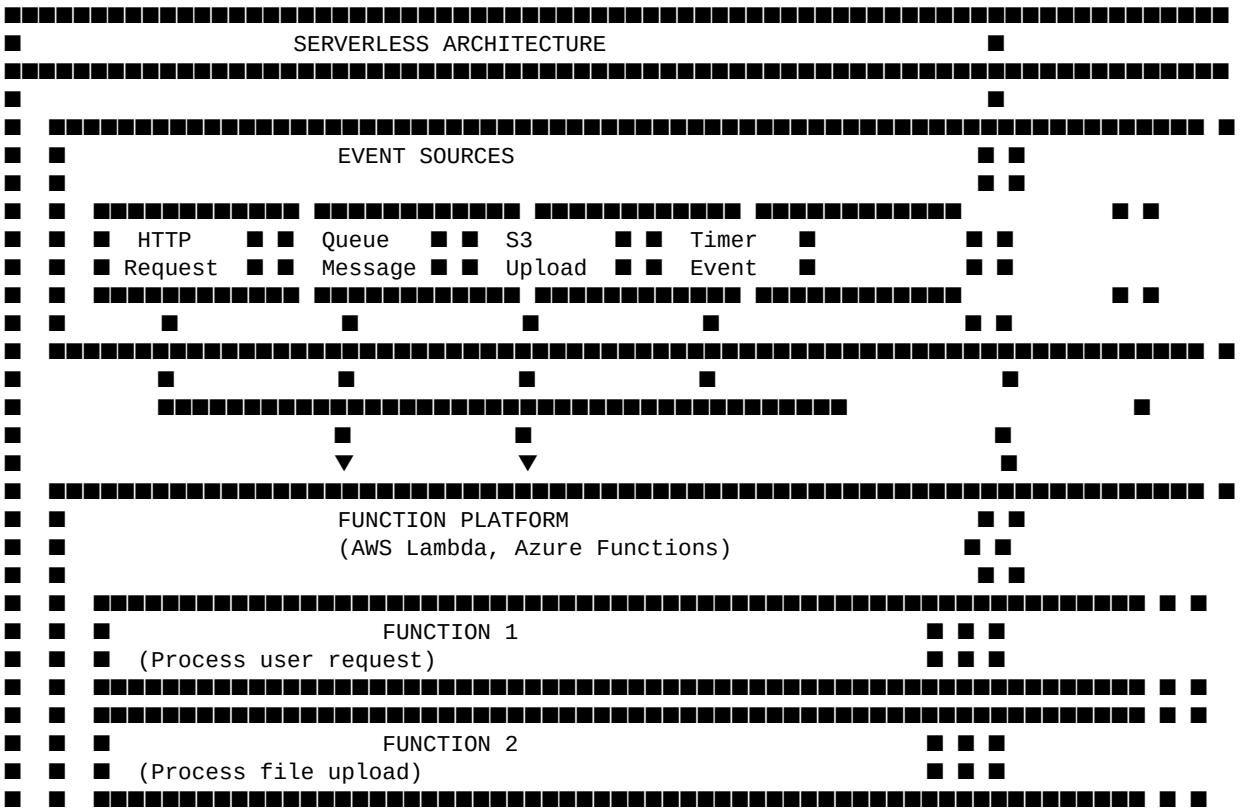
Serverless provides:

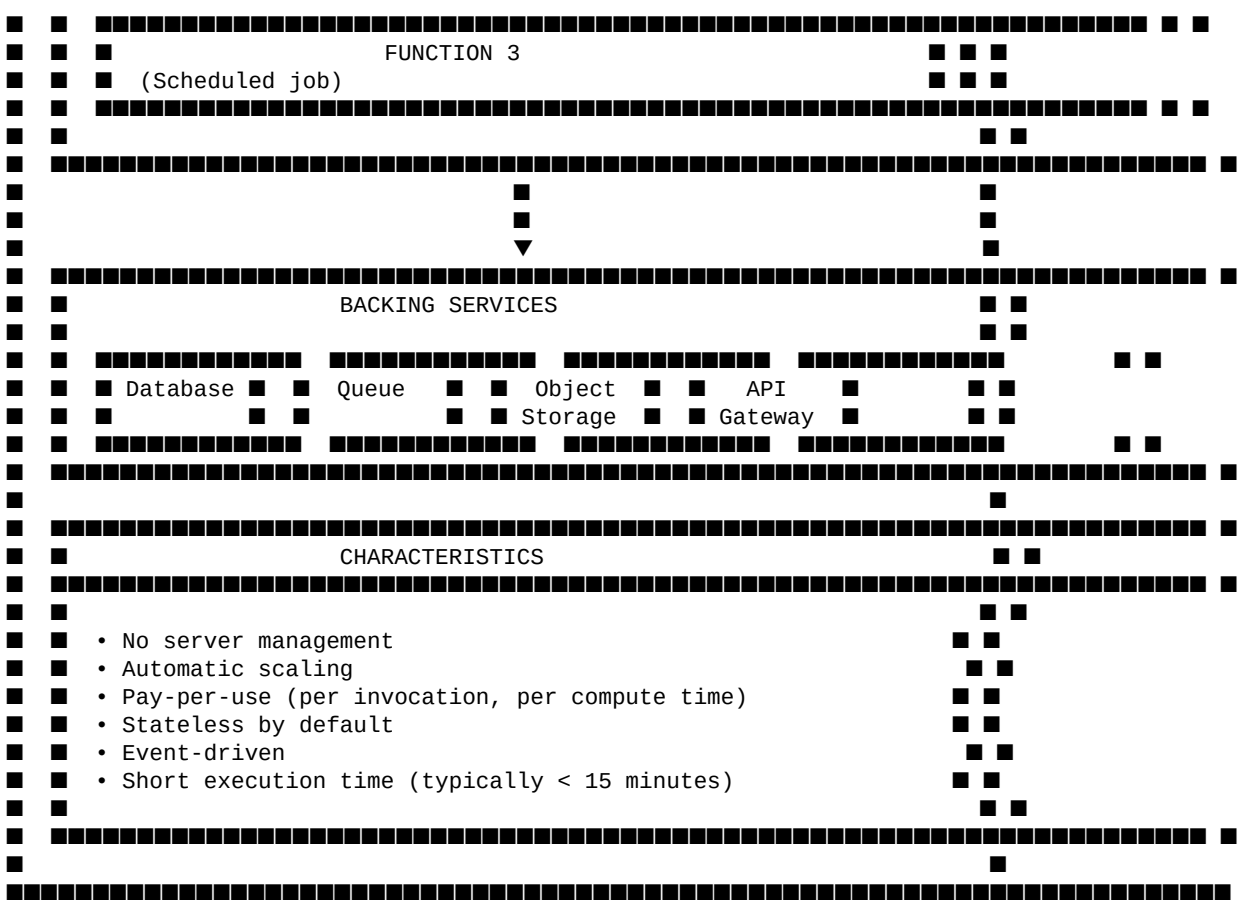
- **Functions:** Small units of code
- **Event Triggers:** Run on events
- **Auto-scaling:** Scale automatically
- **Pay-per-Use:** Pay only for execution time
- **No Server Management:** Provider handles infrastructure

Key Concepts

- **Function:** Small, stateless piece of code
- **Event:** Trigger for function execution
- **Cold Start:** First invocation after idle
- **Stateless:** No local state between invocations
- **FaaS:** Function-as-a-Service

Diagram: Serverless Architecture





When to Use Serverless Architecture

Suitable Situations:

1. For event-driven workloads (file processing, data transformation).

2. For unpredictable or bursty traffic.

3. For background tasks and scheduled jobs.

4. For API backends (with API Gateway).

5. For prototypes and MVPs.

Real-World Use Cases:

- **Image Processing:** Resize images on upload
- **Data Processing:** ETL pipelines

- **Chatbots:** Respond to messages
- **Mobile Backend:** API endpoints
- **Scheduled Jobs:** Daily reports

Advantages of Serverless Architecture

1. **No Server Management:** Focus on code.

2. **Auto-scaling:** Scales automatically.

3. **Cost Efficiency:** Pay only for usage.

4. **Fast Deployment:** Deploy in seconds.

5. **Event Integration:** Native event sources.

Disadvantages of Serverless Architecture

1. **Cold Start:** First invocation may be slow.

2. **Execution Limits:** Time, memory, disk limits.

3. **Vendor Lock-in:** Platform-specific APIs.

4. **Stateless:** No local state between invocations.

5. **Debugging:** Harder to debug distributed functions.

Serverless vs Containers

Aspect	Serverless	Containers
---	---	---
Management	Provider	You manage
Scaling	Automatic	Manual/HPA
Cost	Pay-per-use	Always running
Control	Limited	Full
Startup	Cold start	Faster

Chapter 25: Event-Driven Microservices

Detailed Explanation

Event-Driven Microservices combine microservices architecture with event-driven communication. Services communicate by publishing and consuming events, enabling loose coupling and scalability.

Understanding the Problem

Synchronous microservices communication creates:

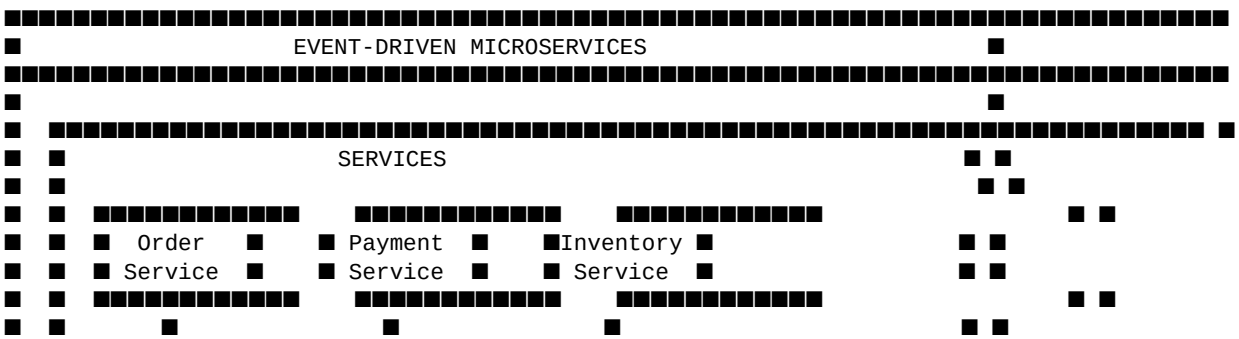
- Temporal coupling (services must be available)
- Chain failures
- Scalability challenges
- Hard to add new services

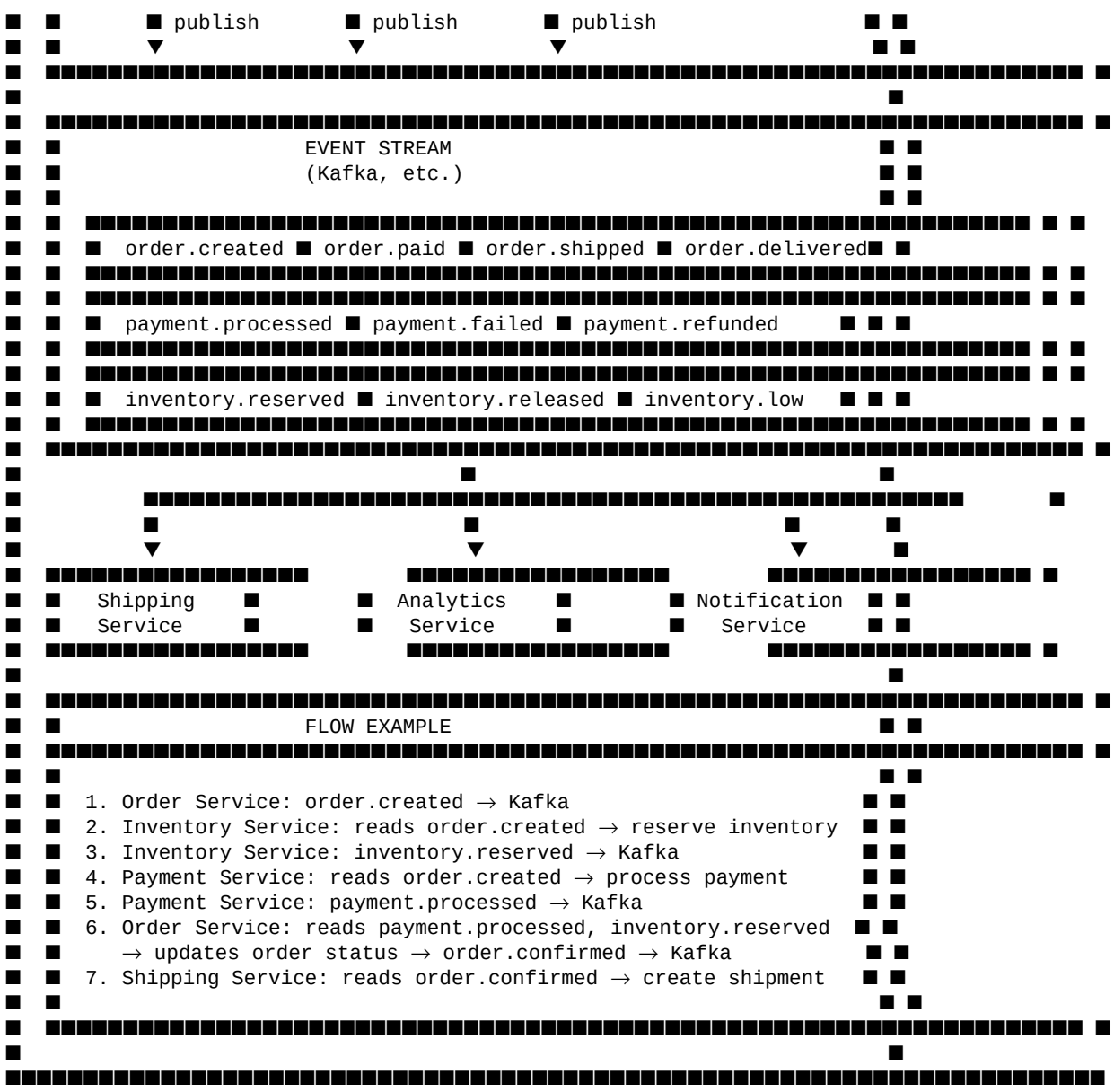
The Solution

Event-Driven Microservices use:

- **Events:** Primary communication mechanism
- **Event Store:** Persistent log of events
- **Async Communication:** No blocking
- **Decoupled:** Services don't know each other
- **Event Sourcing:** Optional for state management

Diagram: Event-Driven Microservices





When to Use Event-Driven Microservices

Suitable Situations:

1. When services need loose coupling.

2. When you need to handle long-running processes.

3. When multiple services need the same data.

4. When you want to replay past events.

5. For cross-service workflows.

Real-World Use Cases:

- **E-commerce Order Processing:** Multi-step orchestration
- **Financial Transactions:** Audit trails, reconciliation
- **IoT Systems:** Processing sensor events
- **Social Media:** Activity feeds
- **Logging and Monitoring:** Event collection

Advantages of Event-Driven Microservices

1. **Loose Coupling:** Services don't know each other.

2. **Scalability:** Services scale independently.

3. **Resilience:** Failures don't cascade.

4. **Auditability:** Complete event history.

5. **Flexibility:** Add new consumers anytime.

Disadvantages of Event-Driven Microservices

1. **Complexity:** Harder to understand flow.

2. **Consistency:** Eventual consistency.

3. **Debugging:** Harder to trace requests.

4. **Delivery Guarantees:** At-least-once, exactly-once.

5. **Tooling:** Requires event streaming platform.

Event-Driven vs Synchronous

Aspect	Event-Driven	Synchronous
Coupling	Loose	Tight

Aspect	Event-Driven	Synchronous
Latency	Higher	Lower
Complexity	Higher	Lower
Consistency	Eventual	Strong

[Page 28]

Chapter 26: Stream Processing Architecture

Detailed Explanation

Stream Processing Architecture processes continuous streams of data in real-time. Data is processed as it arrives, enabling low-latency insights and actions.

Understanding the Problem

Batch processing has limitations:

- High latency (hours/days)
- Not suitable for real-time insights
- Can't handle unbounded data streams
- Requires waiting for batches

The Solution

Stream Processing provides:

- **Continuous Processing:** Data processed as it arrives
- **Low Latency:** Sub-second to seconds
- **Unbounded Data:** Handles infinite streams
- **Stateful Operations:** Maintains state across events
- **Windowing:** Time-based aggregations

Key Concepts

- **Stream:** Continuous flow of events

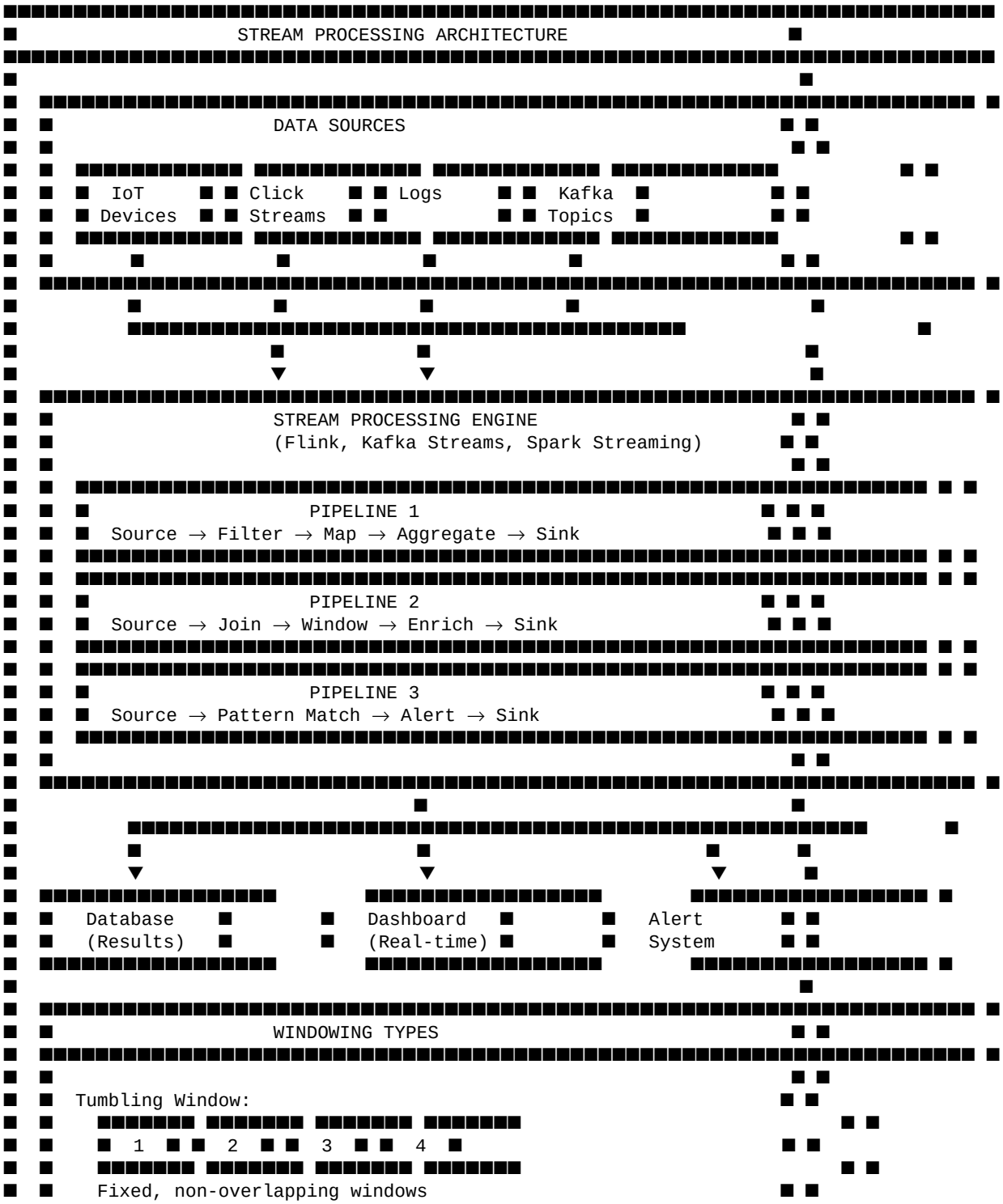
- **Processor:** Function applied to stream

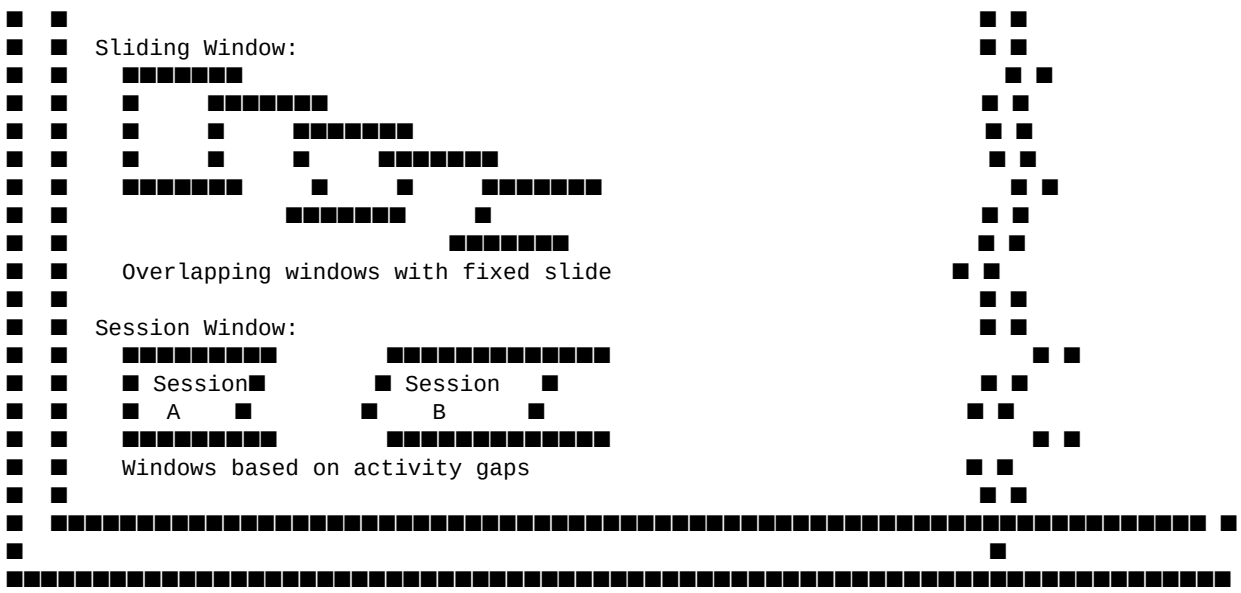
- **Window:** Time-based grouping of events

- **State:** Persistent information across events

- **Watermark:** Progress indicator for time-based processing

Diagram: Stream Processing Architecture





When to Use Stream Processing Architecture

Suitable Situations:

1. When you need real-time insights.

2. For event-driven applications.

3. When data is unbounded (infinite).

4. For monitoring and alerting systems.

5. For fraud detection and anomaly detection.

Real-World Use Cases:

- **Fraud Detection:** Real-time transaction monitoring
- **IoT Analytics:** Sensor data processing
- **Real-time Dashboards:** Live metrics
- **Recommendation Engines:** Personalization
- **Log Analysis:** Real-time log processing

Advantages of Stream Processing

1. **Low Latency:** Sub-second to seconds.

2. **Real-time:** Process as data arrives.

3. **Scalability:** Horizontal scaling.

4. **Fault Tolerance:** Exactly-once semantics.

5. **Stateful:** Maintain state across events.

Disadvantages of Stream Processing

1. **Complexity:** Harder than batch.

2. **State Management:** Complex state handling.

3. **Exactly-Once:** Hard to guarantee.

4. **Debugging:** Hard to debug streams.

5. **Resource Intensive:** More resources than batch.

Stream vs Batch

Aspect	Stream	Batch
--------	--------	-------
Latency	Low (ms to s)	High (minutes to hours)
Data	Unbounded	Bounded
Complexity	High	Lower
State	Stateful	Stateless

[Page 29]

Chapter 27: Data Mesh Architecture

Detailed Explanation

Data Mesh is a decentralized sociotechnical architecture for managing analytical data. It treats data as a product, with domain-oriented ownership and self-serve infrastructure.

Understanding the Problem

Traditional data architectures (data lakes, warehouses) have issues:

- Centralized teams become bottlenecks
- Data quality varies
- Hard to scale
- Discovery difficult

The Solution

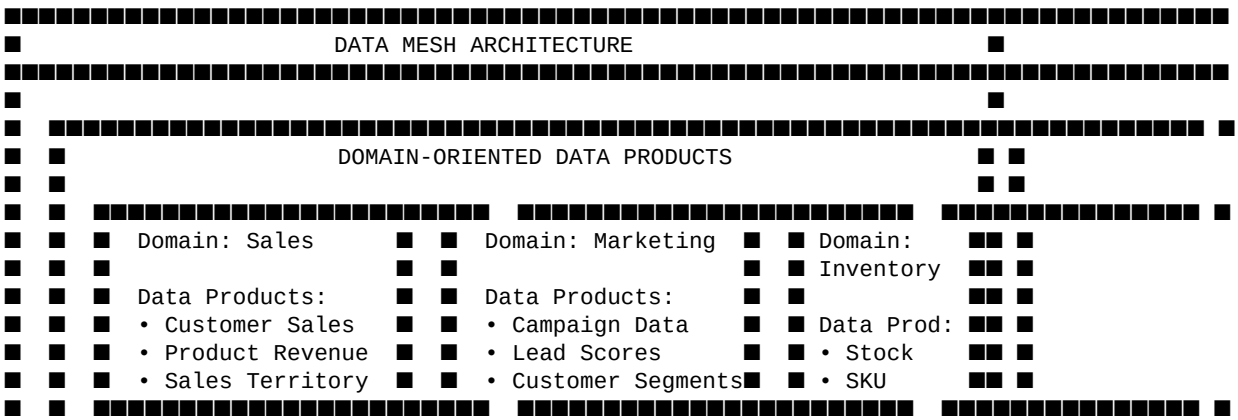
Data Mesh provides:

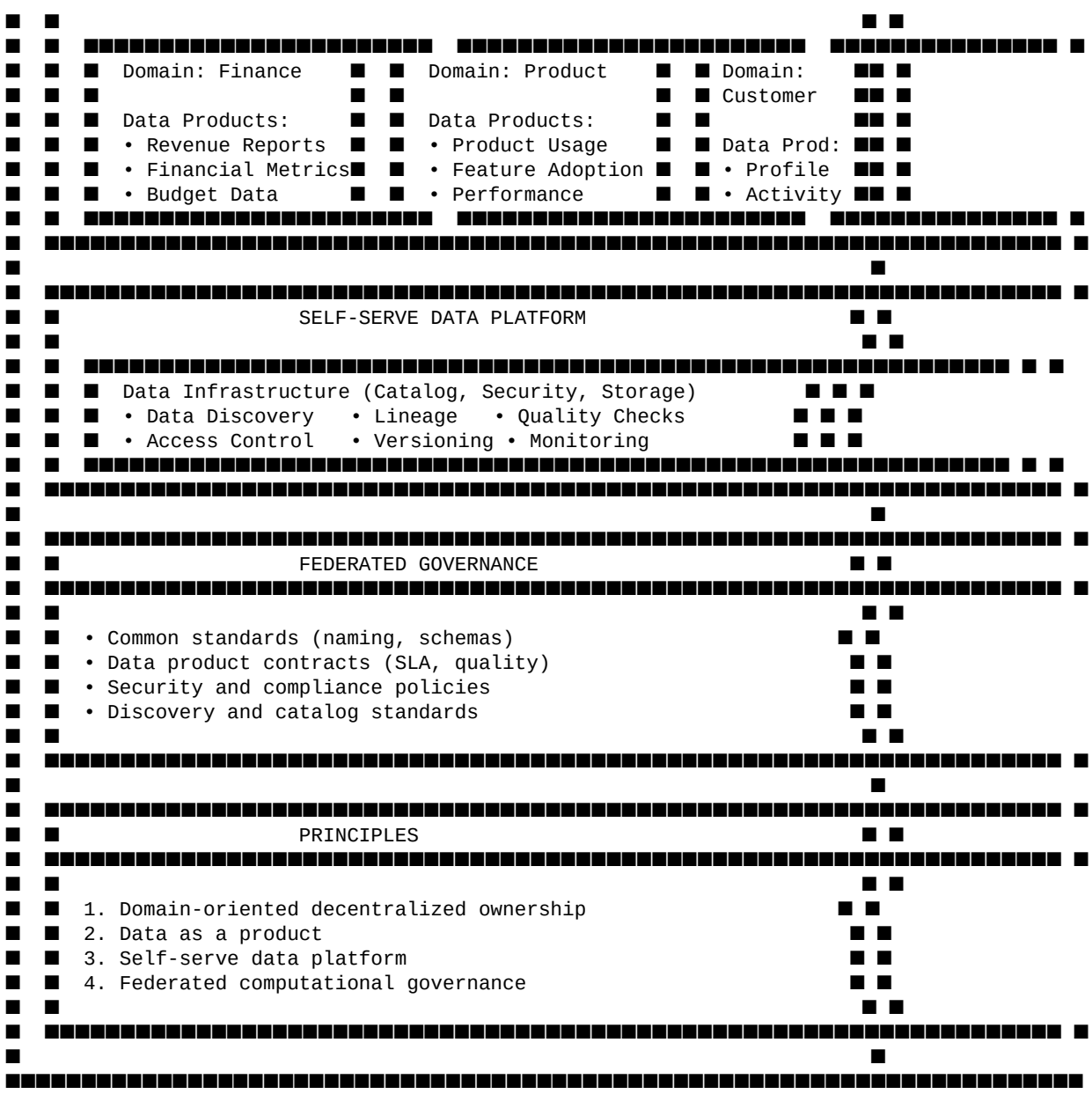
- **Domain-Oriented Ownership:** Data owned by domains
- **Data as a Product:** Data treated as product with owners, SLAs
- **Self-Serve Data Platform:** Infrastructure for data products
- **Federated Governance:** Common standards across domains

Key Concepts

- **Data Product:** Unit of data with owners, SLAs, schema
- **Domain:** Business capability that owns its data
- **Data Infrastructure:** Self-serve platform
- **Data Catalog:** Discovery and governance

Diagram: Data Mesh Architecture





When to Use Data Mesh Architecture

Suitable Situations:

1. When scaling analytical data across large organizations.

2. When centralized data teams become bottlenecks.

3. When data quality and discovery are problems.

4. When you have multiple business domains.

5. For complex data ecosystems.

Real-World Use Cases:

- **Large Enterprises:** Multiple business units sharing data
- **E-commerce:** Sales, marketing, inventory data products
- **Financial Services:** Risk, compliance, trading data
- **Healthcare:** Patient, provider, claims data

Advantages of Data Mesh

1. **Scalability:** Distributed ownership scales.

2. **Data Quality:** Domain ownership improves quality.

3. **Discovery:** Self-serve platform enables discovery.

4. **Agility:** Domains can move independently.

5. **Decentralization:** No central bottleneck.

Disadvantages of Data Mesh

1. **Complexity:** Requires organizational change.

2. **Investment:** Significant platform investment.

3. **Cultural Shift:** Teams must adopt new mindset.

4. **Governance:** Hard to maintain standards.

5. **Maturity:** Still emerging pattern.

Data Mesh vs Data Lake

Aspect	**Data Mesh**	**Data Lake**
--------	----------	----------
Ownership	Decentralized	Centralized
Scale	High	Limited by central team

Aspect	Data Mesh	Data Lake
Quality	Domain accountable	Central team accountable
Discovery	Self-serve	Central catalog

[Page 30]

Chapter 28: Cell-Based Architecture

Detailed Explanation

Cell-Based Architecture decomposes applications into independent, self-contained units called cells. Each cell is a complete, isolated instance of the application that can be deployed and scaled independently.

Understanding the Problem

Microservices address component-level independence but:

- Shared infrastructure (databases, message brokers) remain
- Failure domains still affect many services
- Complex deployment dependencies

The Solution

Cell-Based Architecture creates:

- **Cells:** Self-contained application units
- **Cell Boundaries:** Complete isolation
- **Cell Routing:** Traffic distribution across cells
- **Failure Containment:** Failures stay in one cell
- **Independent Scaling:** Scale cells independently

Key Concepts

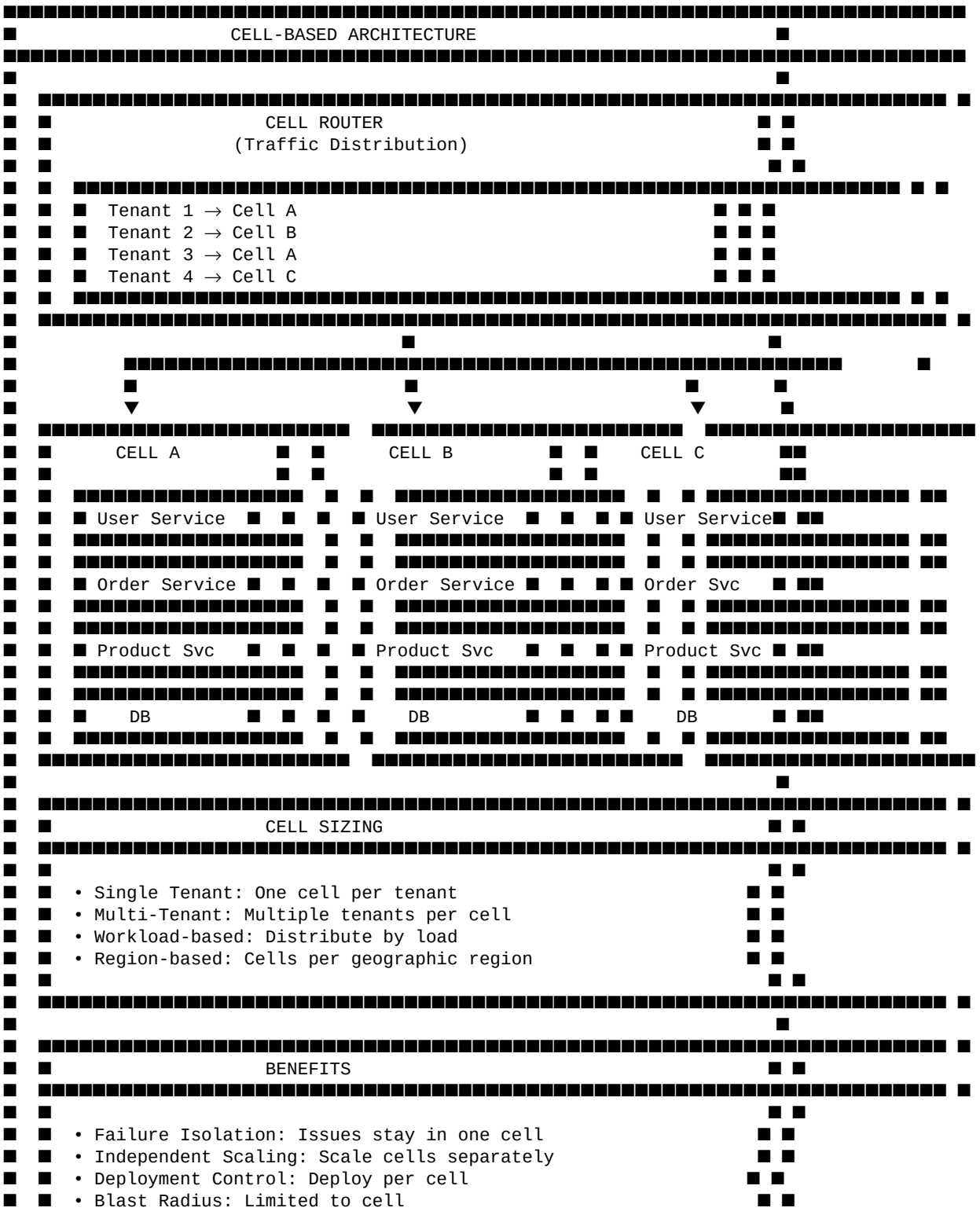
- **Cell:** Complete, isolated application instance
- **Cell Router:** Distributes traffic to cells

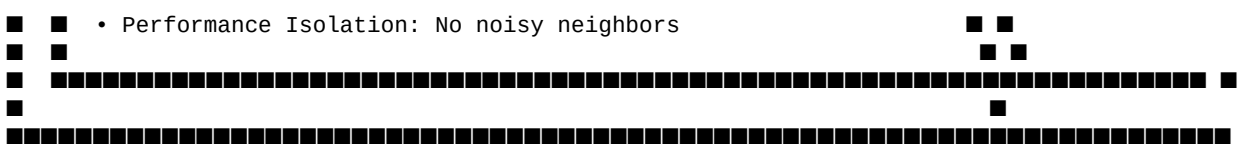
- **Cell API:** Entry point for each cell

- **Cell Data:** Each cell has its own data

- **Failure Domain:** Single cell failure boundary

Diagram: Cell-Based Architecture





When to Use Cell-Based Architecture

Suitable Situations:

1. When you need strong failure isolation.

2. When serving many tenants.

3. When you need independent scaling.

4. When you want to control deployment blast radius.

5. For multi-tenant SaaS applications.

Real-World Use Cases:

- **SaaS Platforms:** Multi-tenant applications
- **High-Scale Services:** Social media platforms
- **Financial Systems:** Trading platforms
- **Gaming:** Multiplayer game servers
- **IoT Platforms:** Device management

Advantages of Cell-Based Architecture

1. **Failure Isolation:** Failures contained to one cell.

2. **Independent Scaling:** Scale cells based on load.

3. **Deployment Control:** Deploy per cell (canary).

4. **Performance Isolation:** No noisy neighbors.

5. **Operational Simplicity:** Smaller failure domains.

Disadvantages of Cell-Based Architecture

1. **Data Duplication:** Data replicated across cells.

2. **Complex Routing:** Cell router must be reliable.

3. **Cross-Cell Operations:** Hard to query across cells.

4. **Resource Overhead:** Redundancy costs.

5. **Migration Complexity:** Moving tenants between cells.

Cell-Based vs Microservices

Aspect	Cell-Based	Microservices
---	---	---
Isolation	Cell level	Service level
Scale	Cell	Service
Data	Per cell	Per service
Complexity	Higher	Lower

[Page 31]

PART EIGHT: CROSS-CUTTING PATTERNS

Chapter 29: Circuit Breaker

Detailed Explanation

The Circuit Breaker pattern prevents cascading failures by stopping calls to a failing service. It wraps service calls and trips when failures reach a threshold, allowing the service time to recover.

Understanding the Problem

When a service fails, calls to it:

- Wait for timeout

- Consume resources

- Cause cascading failures

- Prevent recovery

The Solution

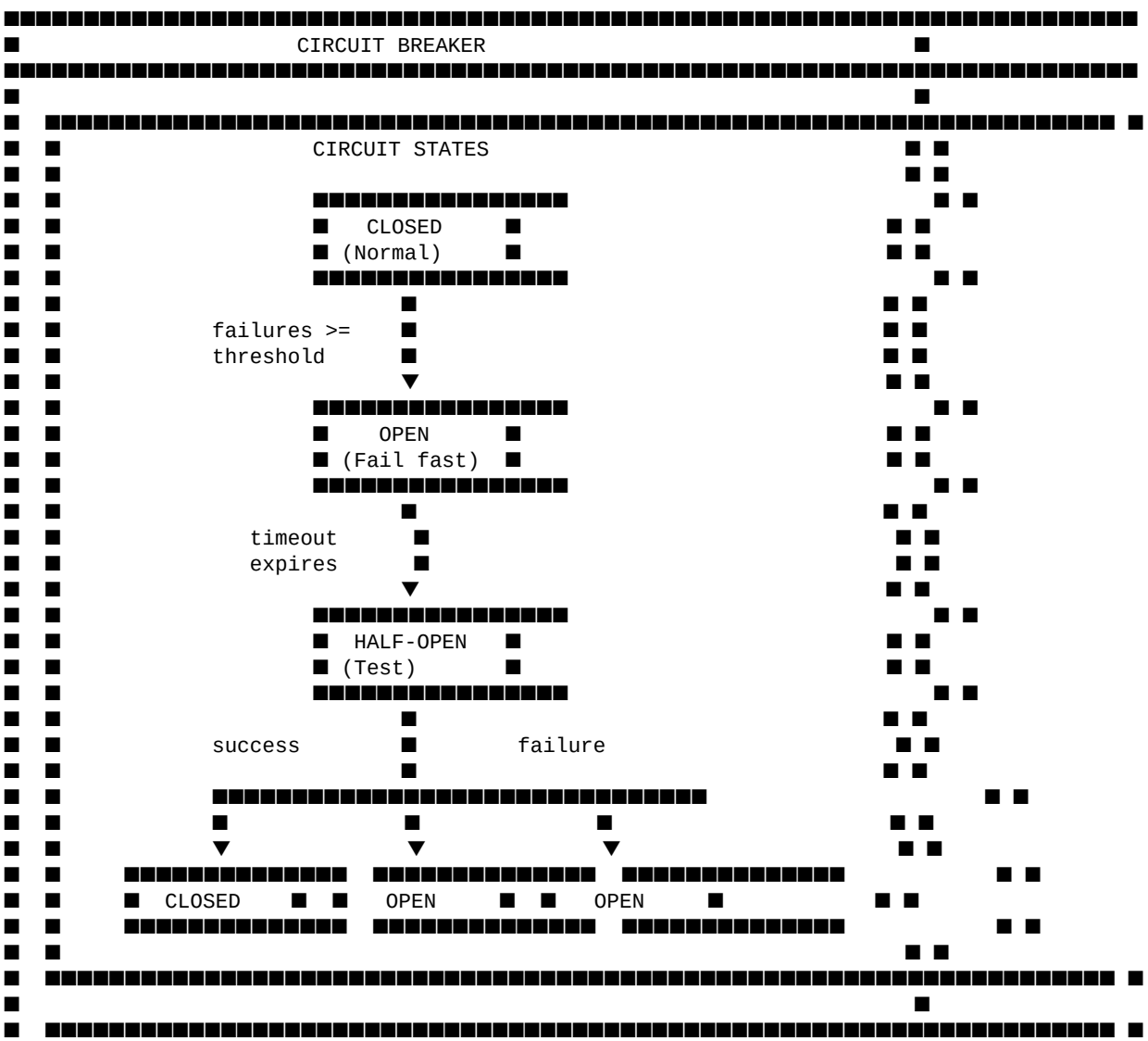
Circuit Breaker has three states:

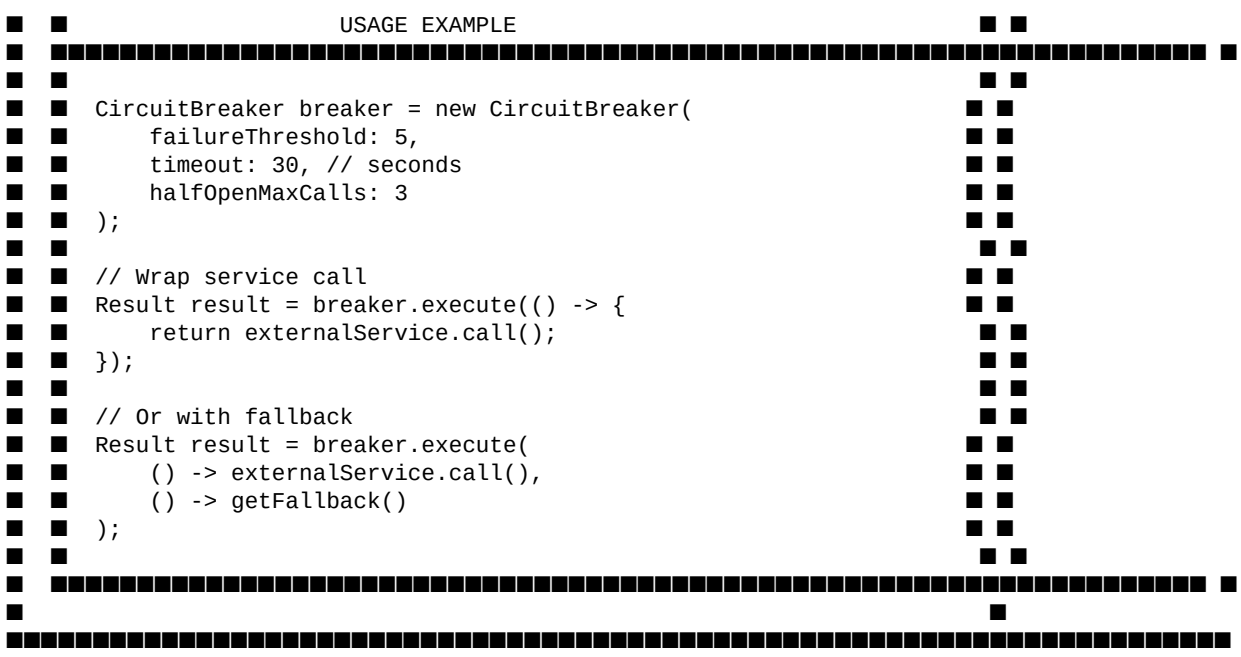
- **CLOSED:** Normal operation, failures counted

- **OPEN:** Fail fast, no calls

- **HALF-OPEN:** Test recovery with limited calls

Diagram: Circuit Breaker





When to Use Circuit Breaker

1. When calling remote services.
2. When failures are likely.
3. When you want fail-fast behavior.
4. When you need to give failing services time to recover.
5. In microservices architectures.

Real-World Use Cases:

- **Microservices:** Preventing cascading failures
- **API Gateways:** Protecting downstream services
- **Database Proxies:** Database failure handling
- **External APIs:** Third-party service calls

Advantages

- Prevents cascading failures
- Fast fail on open circuit
- Automatic recovery testing
- Reduces resource waste

- Improves overall resilience

Disadvantages

- Added complexity
- Tuning required
- State management needed
- May need to handle partial failures

[Page 32]

Chapter 30: API Gateway

Detailed Explanation

The API Gateway pattern provides a single entry point for all client requests to a microservices-based application, handling routing, authentication, rate limiting, and aggregation.

Understanding the Problem

Microservices have many endpoints, and clients need to:

- Know multiple service locations
- Handle authentication for each service
- Make multiple requests for one page
- Deal with different protocols

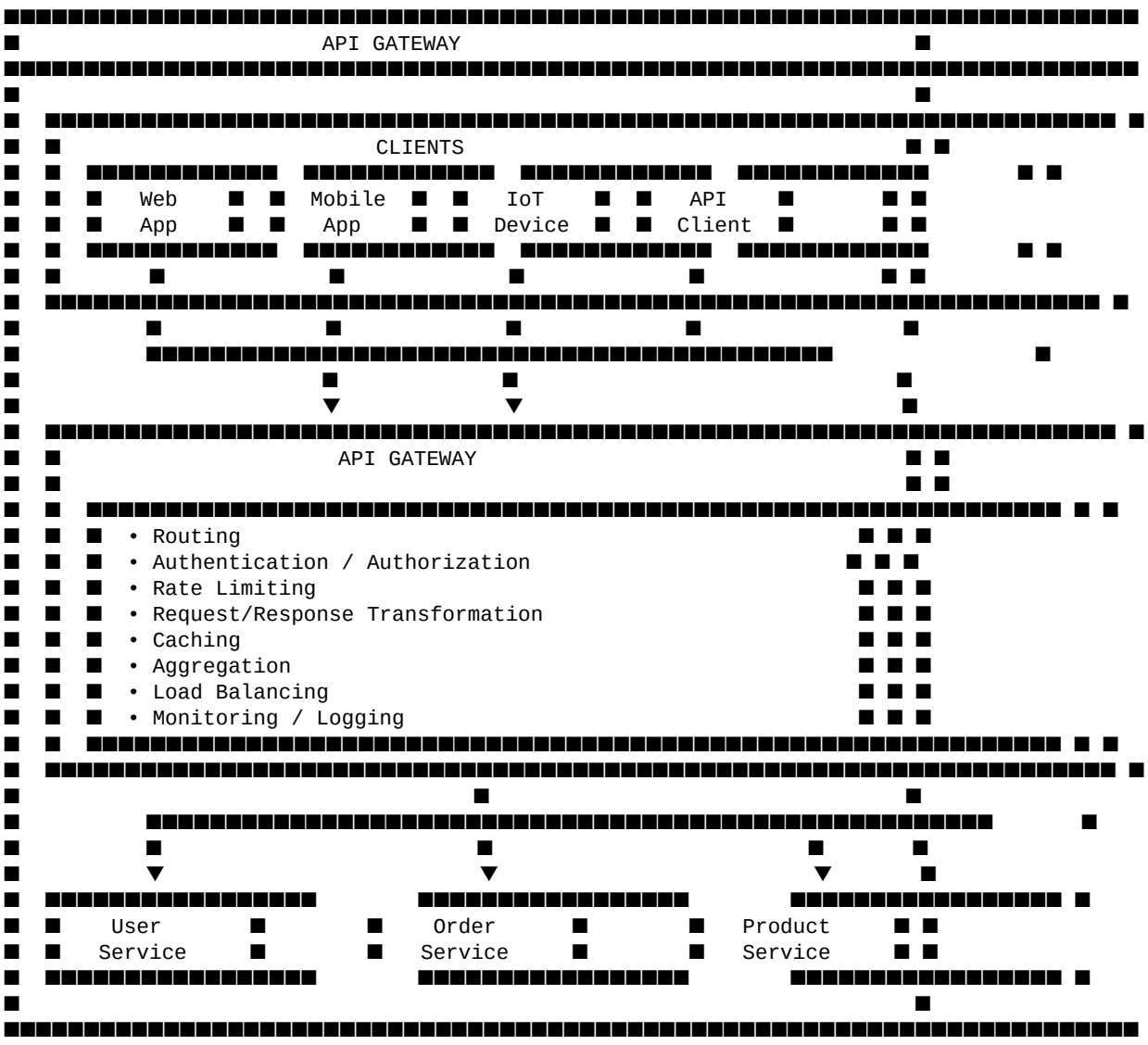
The Solution

API Gateway provides:

- **Single Entry Point:** One URL for all services
- **Routing:** Route to appropriate service
- **Authentication:** Centralized auth
- **Aggregation:** Combine multiple service responses

- **Cross-cutting Concerns:** Rate limiting, logging, caching

Diagram: API Gateway



When to Use API Gateway

1. For microservices architectures.
2. When you have multiple client types.
3. When you need centralized cross-cutting concerns.
4. When clients need aggregated responses.
5. When you want to decouple clients from services.

Real-World Use Cases:

- **Microservices:** Single entry point
- **Mobile Backends:** Optimized for mobile
- **API Management:** Monetization, developer portal
- **Edge Services:** Edge caching, compression

Advantages

- Single entry point simplifies clients
- Centralized cross-cutting concerns
- Protocol translation
- Aggregation reduces client round trips
- Security enforcement point

Disadvantages

- Single point of failure (mitigate with redundancy)
- Potential performance bottleneck
- Added complexity
- Development overhead

[Page 33]

Chapter 31: Strangler Pattern

Detailed Explanation

The Strangler Pattern incrementally migrates a monolithic application to microservices by gradually replacing parts of the monolith with new services, eventually "strangling" the old system.

Understanding the Problem

Rewriting a monolith all at once is:

- Risky

- Time-consuming

- Disruptive to development

- Hard to test

The Solution

Strangler Pattern steps:

1. Create API Gateway/Proxy

2. Identify functionality to replace

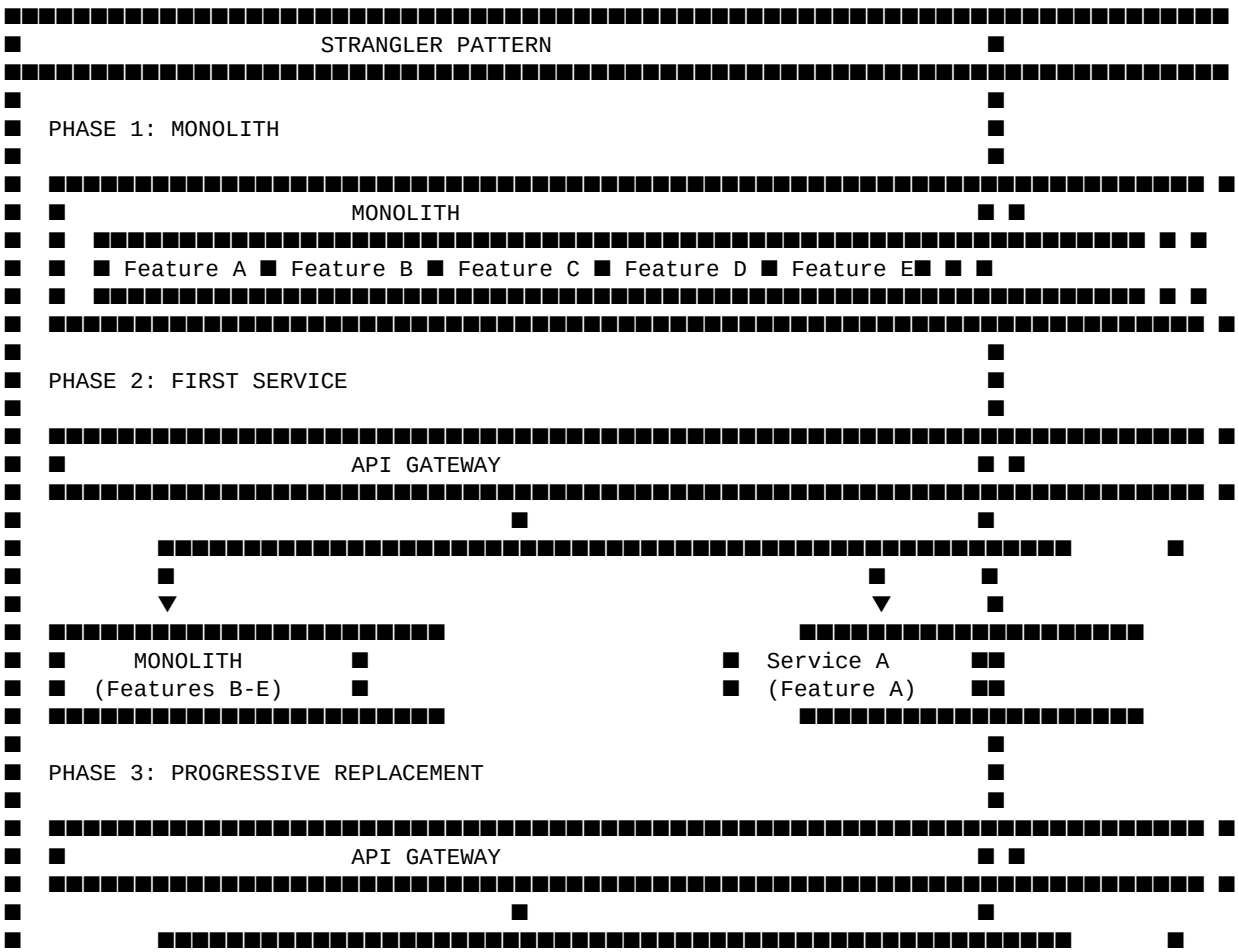
3. Build new microservice

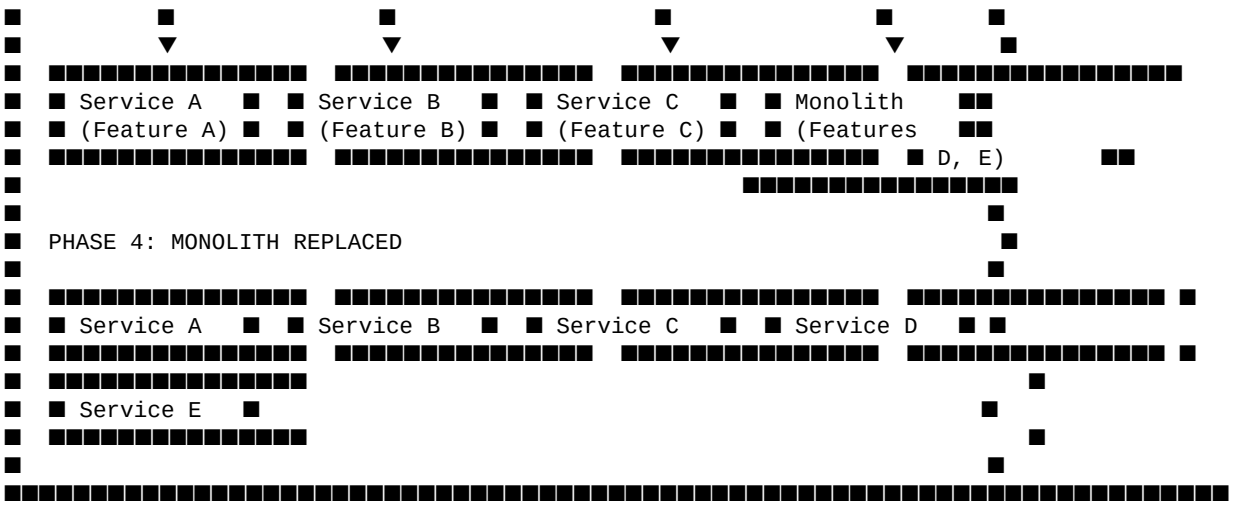
4. Route traffic to new service

5. Remove old code

6. Repeat until monolith is replaced

Diagram: Strangler Pattern





When to Use Strangler Pattern

1. When migrating from monolith to microservices.
2. When you can't rewrite everything at once.
3. When you need continuous feature delivery during migration.
4. When you want to reduce risk.
5. When you have a large, complex legacy system.

Real-World Use Cases:

- **Legacy Migration:** Mainframe to modern
- **Monolith to Microservices:** Amazon, Netflix
- **Framework Migration:** Moving between technologies

Advantages

- Low risk (incremental changes)
- Continuous delivery during migration
- Rollback capability
- Learning opportunity
- Business continuity

Disadvantages

- Prolonged migration (months/years)

- Dual maintenance
- Complex routing
- Data consistency challenges
- Coordination overhead

[Page 34]

Chapter 32: Sidecar Pattern

Detailed Explanation

The Sidecar pattern deploys auxiliary components alongside the main application container, providing supporting features like logging, monitoring, and configuration without modifying the main application.

Understanding the Problem

Applications need supporting features:
- Logging agents
- Monitoring
- Configuration management
- Service discovery
- Security proxies

Embedding these in the app causes:
- Code bloat
- Language constraints
- Upgrade difficulties
- Cross-cutting concerns mixing

The Solution

Sidecar runs in separate container/process:
- Co-located with main app

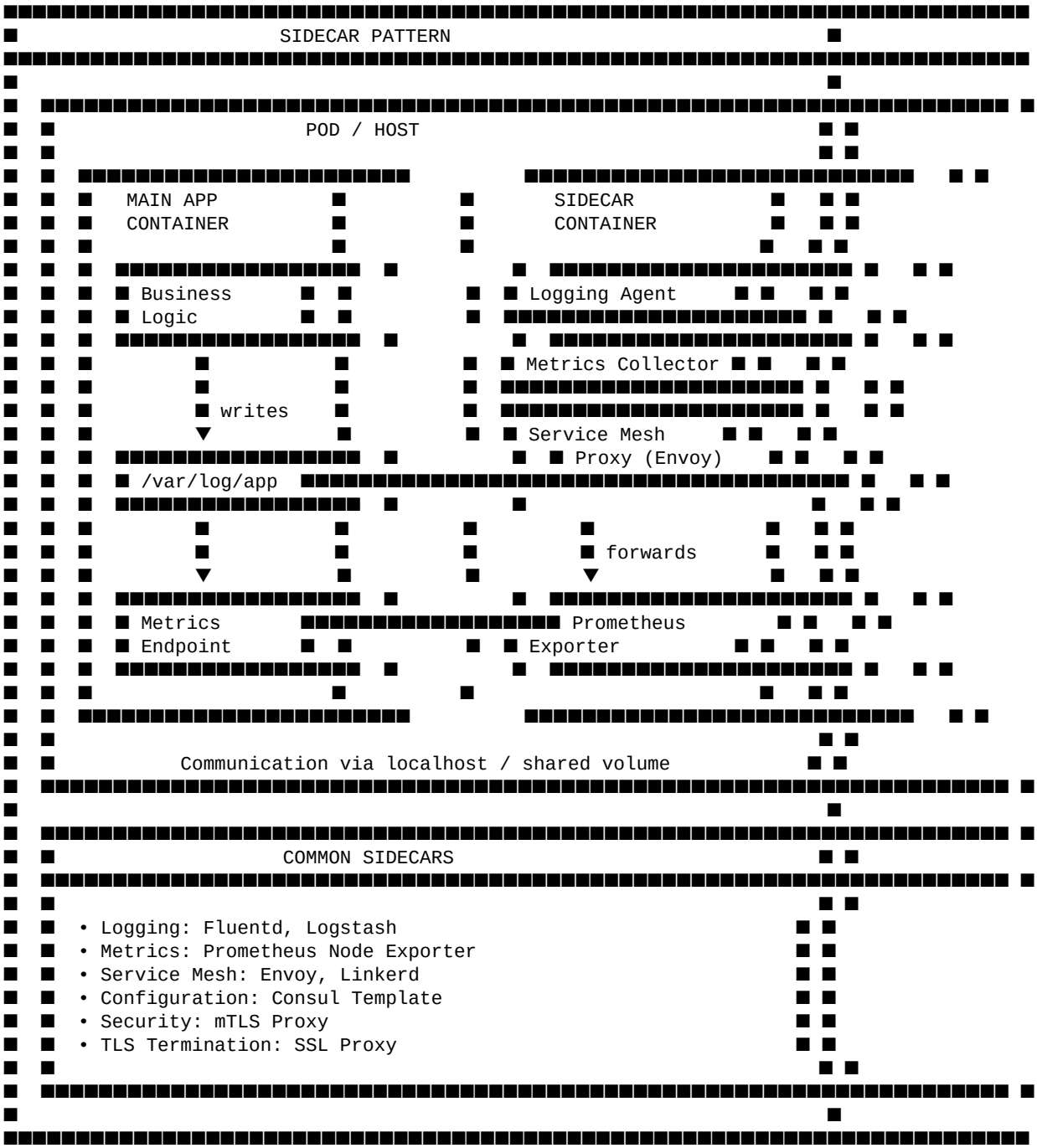
- Shares lifecycle (same pod/host)

- Local communication (localhost)

- Language-agnostic

- Independent upgrades

Diagram: Sidecar Pattern



When to Use Sidecar Pattern

1. When you need to add features without modifying main app.
2. When you want language-agnostic infrastructure.
3. When you need independent upgrades.
4. For service mesh implementations.
5. For logging and monitoring agents.

Real-World Use Cases:

- **Service Mesh:** Envoy, Linkerd sidecars
- **Logging:** Fluentd sidecar for log shipping
- **Monitoring:** Prometheus exporters
- **Configuration:** Dynamic config reloaders
- **Security:** Authentication/authorization proxies

Advantages

- Separation of concerns
- Language-agnostic
- Independent upgrades
- Reusable components
- Transparent to main app

Disadvantages

- Resource overhead
- Increased complexity
- Deployment coordination
- Communication overhead
- Debugging complexity

[Page 35]

Chapter 33: Ambassador Pattern

Detailed Explanation

The Ambassador pattern creates a helper service that handles network communication on behalf of the main application, managing retries, circuit breaking, service discovery, and authentication.

Understanding the Problem

Applications need to handle:

- Service discovery
- Retry logic
- Circuit breaking
- Load balancing
- Authentication
- Request tracing

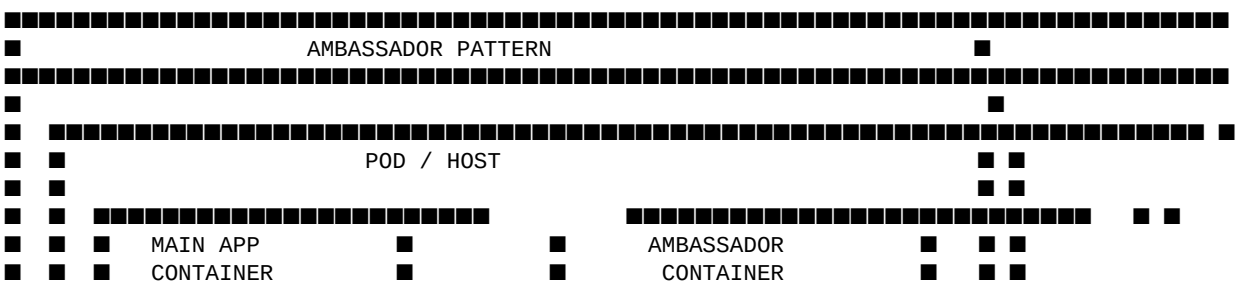
Implementing all this in each service is duplicative.

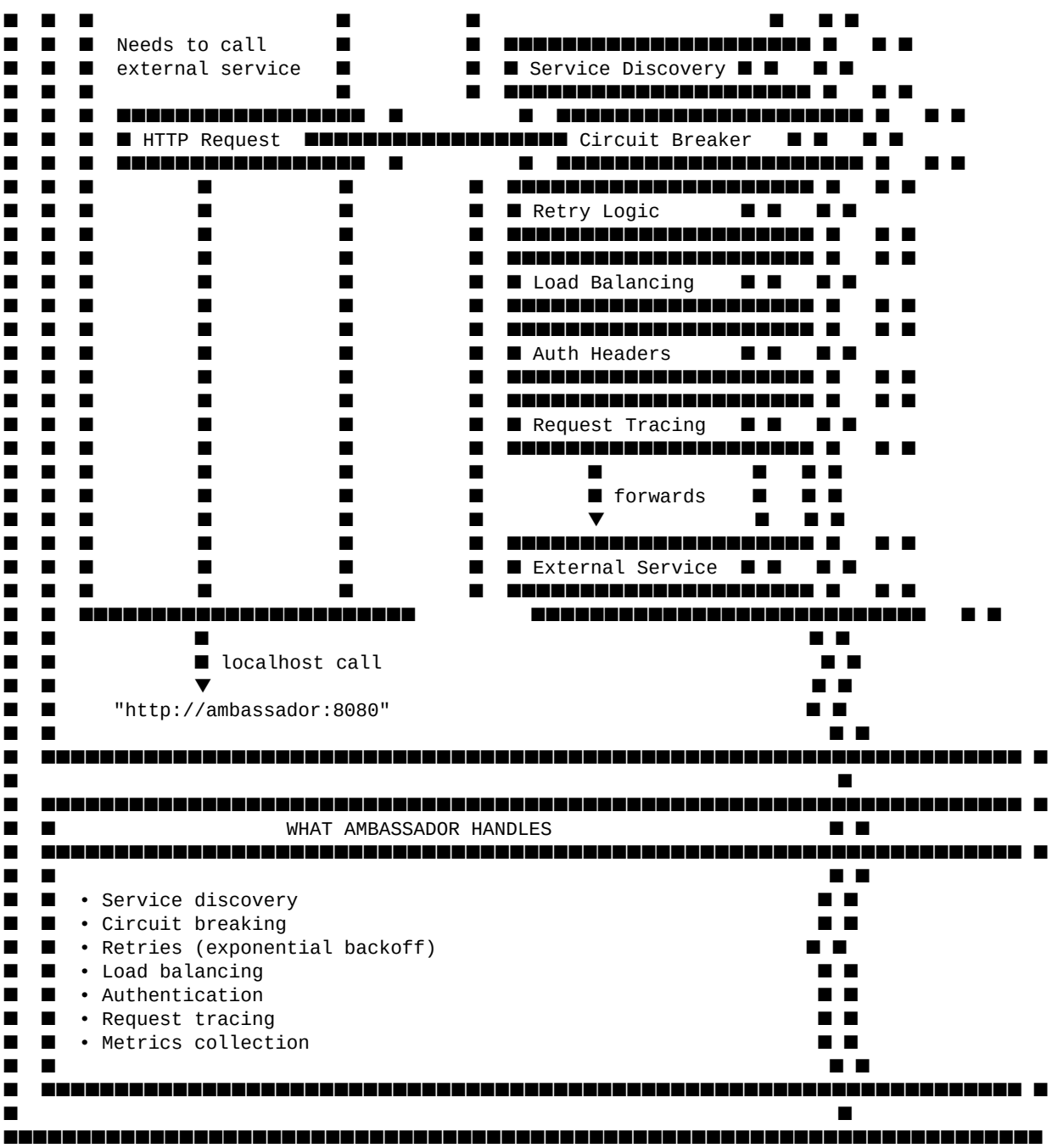
The Solution

Ambassador runs as a sidecar:

- Intercepts outbound requests
- Handles cross-cutting concerns
- App talks to ambassador on localhost
- Ambassador forwards to actual service
- Provides simplified interface to main app

Diagram: Ambassador Pattern





When to Use Ambassador Pattern

1. When services need to call external services.
2. When you want to centralize cross-cutting concerns.
3. When you need sophisticated retry and circuit breaking.
4. When you want to abstract service discovery.
5. For service mesh implementations.

Real-World Use Cases:

- **Database Proxies:** Connection pooling, read/write splitting
- **Service Mesh:** Envoy as ambassador
- **Third-party API:** Rate limiting, auth
- **Legacy Integration:** Protocol translation

Advantages

- Separation of concerns
- Consistent policies across services
- Language-agnostic
- Independent upgrades
- Simplified main app

Disadvantages

- Resource overhead
- Latency increase
- Configuration complexity
- Debugging difficulty

Sidecar vs Ambassador

Aspect	Sidecar	Ambassador
--------	---------	----------
Focus	Auxiliary features	Outbound communication
Direction	Inbound/outbound	Primarily outbound
Examples	Logging, metrics	Service proxy

[Page 36]

PART NINE: COMPARISON AND INTERVIEW PREPARATION

Chapter 34: Pattern Comparison Table

Pattern	Category	Purpose	When to Use
---	---	---	---
Layered	Structural	Separation of concerns	Simple enterprise apps
Hexagonal	Structural	Decouple domain	Complex domain logic
Onion	Structural	Domain-centric	Long-lived systems
Clean	Structural	Dependency inversion	Testability, frameworks
Monolith	Microservices	Single unit	Small teams, startups
Microservices	Microservices	Independent services	Large teams, scale
SOA	Microservices	Enterprise integration	Legacy integration
SCS	Microservices	Complete systems	Multiple teams, UI
Space-Based	Microservices	High scalability	Real-time, high load
EDA	Event-Driven	Loose coupling	Async processing
Event Sourcing	Event-Driven	Audit trail	History, replay
CQRS	Event-Driven	Read/write separation	Complex queries
Pub-Sub	Event-Driven	Broadcast	Multiple consumers
Message Bus	Message-Based	Central integration	Enterprise integration
Message Queue	Message-Based	Work distribution	Async processing
ESB	Message-Based	Service mediation	Complex integration
P2P	Distributed	Decentralized	Large scale, no central
Master-Slave	Distributed	Coordination	Read-heavy workloads
Shared-Nothing	Distributed	Linear scaling	Large data
Sharding	Distributed	Data distribution	Large datasets
Serverless	Modern	Event-driven	Bursty workloads
Event-Driven Microservices	Modern	Loose coupling	Microservices
Stream Processing	Modern	Real-time	Analytics, monitoring
Data Mesh	Modern	Data ownership	Data platforms
Cell-Based	Modern	Failure isolation	Multi-tenant SaaS
Circuit Breaker	Cross-Cutting	Resilience	Remote calls
API Gateway	Cross-Cutting	Entry point	Microservices
Strangler	Cross-Cutting	Migration	Legacy replacement
Sidecar	Cross-Cutting	Auxiliary features	Kubernetes
Ambassador	Cross-Cutting	Outbound proxy	Service mesh

Chapter 35: How to Answer Architecture Questions

Understanding What Interviewers Look For

When interviewers ask about architecture, they want to see:

1. **Trade-off Awareness:** Do you know pros and cons of each pattern?

2. **Context Sensitivity:** Can you choose the right pattern for the situation?

3. **Non-Functional Requirements:** Do you consider scalability, availability, maintainability?

4. **Experience:** Can you relate patterns to real projects?

5. **Communication:** Can you explain complex ideas simply?

The Structure for Architecture Answers

1. Start with the Problem

"What problem are we trying to solve? What are the constraints?"

2. Present Options

"We could use Pattern A, B, or C. Let me explain each."

3. Compare Trade-offs

"Pattern A gives us X but costs Y. Pattern B gives us Z but..."

4. Make Recommendation

"Based on our constraints (team size, scale, etc.), I recommend Pattern A because..."

5. Discuss Alternatives

"If we later find that..., we could evolve to Pattern B."

Example: Choosing Microservices

"Given we have 8 teams and need independent deployment cycles, microservices would be appropriate. However, we need to consider:

- **Consistency:** We'll need Saga pattern for distributed transactions
- **Communication:** We'll need API Gateway and service discovery
- **Operations:** We'll need container orchestration (Kubernetes)
- **Data:** Each service needs its own database

The trade-off: we get independent deployments and team autonomy, but we accept operational complexity and eventual consistency.

If our team were smaller (3-5 developers), I'd recommend a modular monolith instead to keep complexity manageable."

Common Architecture Interview Questions

1. "How would you design a system that needs to handle 1 million concurrent users?"

2. "How would you migrate a monolith to microservices?"

3. "How do you ensure data consistency across microservices?"

4. "What architecture would you choose for a real-time analytics platform?"

5. "How do you handle failure in a distributed system?"

6. "What's the difference between SOA and microservices?"

7. "When would you choose event-driven over request-response?"

8. "How do you scale a database?"

9. "What's your approach to API design?"

10. "How do you secure a microservices architecture?"

Chapter 36: Common Interview Questions with Examples

1. Q: How would you design a system for 1 million concurrent users?

A: I'd design a layered, scalable architecture:

- **Load Balancer:** Distribute traffic across multiple instances
- **CDN:** Serve static content from edge locations
- **API Gateway:** Single entry point with rate limiting
- **Microservices:** Independently scalable services
- **Database:** Read replicas for reads, sharding for writes
- **Caching:** Redis for frequently accessed data
- **Queue:** Async processing for non-critical operations
- **Auto-scaling:** Based on CPU/memory metrics

The key is to identify bottlenecks: databases are often the first. Use caching to reduce database load, and queue to smooth traffic spikes.

2. Q: How do you migrate a monolith to microservices?

A: I'd use the Strangler pattern:

1. **Add API Gateway:** Route all traffic through gateway
2. **Identify Service Boundaries:** Based on business capabilities
3. **Extract First Service:** Build new service, route to it
4. **Incrementally Extract:** Continue extracting services
5. **Data Migration:** Move data per service
6. **Remove Old Code:** When all functionality migrated

Key considerations:
- Maintain backwards compatibility during migration
- Use anti-corruption layer for legacy data
- Implement saga for distributed transactions
- Monitor carefully for any issues

3. Q: How do you ensure data consistency across microservices?

A: With the Saga pattern:

For order processing, I'd use orchestration saga:

- **Orchestrator:** Coordinates the saga
- **Commands:** Each step is a command
- **Compensation:** Rollback on failure

Example flow:

1. Create order (pending)
2. Reserve inventory
3. Process payment
4. Confirm order

If payment fails, compensate:

1. Release inventory
2. Cancel order

This gives eventual consistency without distributed transactions.

4. Q: What's the difference between SOA and microservices?

A: Key differences:

Aspect	**SOA**	**Microservices**
--------	-----	----------
Communication	ESB, SOAP	API gateway, REST
Granularity	Coarse	Fine
Governance	Centralized	Decentralized
Data	Shared databases	Database per service
Deployment	Often monolith per service	Independent containers

SOA is often used for enterprise integration with legacy systems. Microservices are for building new, cloud-native applications.

5. Q: When would you choose event-driven architecture?

A: Event-driven works well when:

- **Loose coupling needed:** Services shouldn't know about each other
- **Multiple consumers:** Many services need the same data
- **Async processing:** Can process later
- **Audit required:** Complete event history
- **Real-time not needed:** Eventual consistency acceptable

Example: Order processing with notifications, analytics, and inventory—all can react to the same events.

6. Q: How do you scale a database?

A: Multiple strategies:

1. **Read Replicas:** For read-heavy workloads
2. **Sharding:** Horizontal partitioning by shard key
3. **Caching:** Redis/Memcached for hot data
4. **Connection Pooling:** Efficient connection management
5. **Query Optimization:** Indexes, query tuning
6. **Denormalization:** Reduce joins
7. **CQRS:** Separate read/write paths

Choose based on workload:
- Read-heavy: Replicas
- Write-heavy: Sharding
- Mixed: Combination

7. Q: What architecture would you choose for real-time analytics?

A: Stream processing architecture:

- **Data Ingestion:** Kafka for event streaming
- **Stream Processing:** Apache Flink or Kafka Streams

- **State Management:** RocksDB for local state
- **Windowed Aggregations:** Tumbling/sliding windows
- **Serving Layer:** Druid or ClickHouse for queries
- **Visualization:** Grafana dashboards

Key considerations:
- Exactly-once semantics for accuracy
- Low latency (sub-second)
- Horizontal scaling
- Fault tolerance with state snapshots

8. Q: How do you handle failure in distributed systems?

A: Multiple patterns:

- **Circuit Breaker:** Stop calling failing services
- **Retry with Backoff:** Handle transient failures
- **Bulkhead:** Isolate failures to components
- **Health Checks:** Detect failures quickly
- **Timeout:** Fail fast, don't wait
- **Fallback:** Degraded responses
- **Supervision:** Restart failed components
- **Redundancy:** Multiple instances

And design for failure:
- Assume services will fail
- Test failure scenarios
- Monitor and alert
- Automate recovery

9. Q: What's your API design approach?

A: RESTful principles with pragmatism:

- **Resources:** Nouns, not verbs (/users, /orders)

- **HTTP Methods:** GET, POST, PUT, DELETE
- **Status Codes:** Appropriate HTTP codes
- **Versioning:** URL path (/v1/...)
- **Pagination:** Limit/offset or cursor
- **Authentication:** JWT or OAuth2
- **Documentation:** OpenAPI/Swagger

For microservices, add:
- **API Gateway:** Single entry point
- **Aggregation:** Combine multiple services
- **Rate Limiting:** Per client

10. Q: How do you secure a microservices architecture?

A: Multiple layers:

- **API Gateway:** Authentication, rate limiting
- **JWT Tokens:** Stateless auth with claims
- **Service-to-Service:** mTLS (service mesh)
- **Database:** Network isolation, encryption
- **Secrets Management:** Vault for credentials
- **Audit Logging:** Track access
- **Network Security:** Private networks, firewalls
- **Input Validation:** Prevent injection

Principle of least privilege: Each service gets minimal permissions.

[Page 39]

CONCLUSION

Key Takeaways

1. **Architecture Matters:** It shapes everything that follows.

2. **No Silver Bullet:** Each pattern has trade-offs.

3. **Context is Everything:** Choose based on your specific situation.

4. **Evolve Over Time:** Start simple, add complexity when needed.

5. **Consider the Team:** Architecture must match team capabilities.

6. **Document Decisions:** Capture why you chose what you chose.

Your Interview Preparation Checklist

- [ ] Understand the problem each pattern solves
- [ ] Can explain each pattern in simple terms
- [ ] Know when to use each pattern
- [ ] Know the trade-offs and disadvantages
- [ ] Can compare similar patterns
- [ ] Have real-world examples ready
- [ ] Can draw architecture diagrams
- [ ] Understand non-functional requirements (scalability, availability)

Final Tips

1. **Draw Diagrams:** Visual explanations show deep understanding.

2. **Start with the Problem:** Don't jump to a solution.

3. **Discuss Trade-offs:** Show you've considered alternatives.

4. **Be Honest:** If you haven't used a pattern, say so.

5. **Think at Scale:** Consider how the system would behave under load.

6. **Stay Practical:** Architecture must be implementable by teams.

Recommended Reading

- **Books:** "Building Microservices" (Sam Newman), "Software Architecture: The Hard Parts" (Ford, Richards)
- **Pattern Catalogs:** Chris Richardson's microservices patterns, Martin Fowler's architecture articles
- **Case Studies:** How Netflix, Amazon, Uber, Spotify built their architectures

Good luck with your interview!

About This Book

This book was created specifically for developers and architects preparing for technical interviews on architectural patterns. Every pattern is explained in clear, simple English with comprehensive examples and diagrams. The goal is not just to help you pass an interview, but to make you a better software architect.

May your architectures be scalable, your deployments smooth, and your interviews successful.

[End of Document]